

Advanced Software Engineering

Appunti del corso

Professore: Antonio Brogi

*Traduzione e rielaborazione in italiano degli appunti
di Emiliano Sescu (<https://github.com/Faxatos>), a.a. 2023/24*

22 settembre 2026

Indice

1	Prodotti software	1
1.1	Software engineering project-based	1
1.2	Software engineering product-based	2
1.2.1	Modelli di esecuzione	2
1.3	Product vision	3
1.4	Software product management	4
2	Ingegneria del software Agile	5
2.1	Extreme Programming (XP)	6
2.2	Scrum	7
2.2.1	Scrum Team	8
2.2.2	Artefatti	9
2.2.3	Eventi Scrum	11
2.2.4	Esecuzione: le attività sul backlog	13
3	Feature, scenari e storie	14
3.1	Personas	14
3.2	Scenari	15
3.3	User Stories	16
3.3.1	Dalle storie alle feature	17
3.3.2	Bilanciare i fattori e il feature creep	17
3.3.3	Derivare le feature	19
4	Architettura software	20
4.1	Attributi di qualità non funzionali	21
4.2	Decomposizione del sistema	22
4.2.1	Architettura a livelli	23
4.3	Architettura di distribuzione	24
4.4	Scelte tecnologiche	26
4.5	Enterprise Application Integration	27
4.5.1	Pattern	27
4.5.2	Pipes and Filters	29
5	Software cloud-based	31
5.1	Virtualizzazione e container	31
5.1.1	Docker	32
5.2	Everything as a Service	33

5.2.1	SaaS	34
5.2.2	Sistemi multi-tenant e multi-instance	35
5.3	Architettura del software nel cloud	37
5.3.1	Organizzazione del database	37
5.3.2	Scalabilità e resilienza	38
5.3.3	Struttura del software	38
5.4	Orchestrazione dei container	39
5.4.1	Kubernetes	39
5.4.2	Principi di design di Kubernetes	40
5.4.3	Oggetti di Kubernetes	41
5.4.4	Il control plane di Kubernetes	43
6	Architettura a microservizi	46
6.1	Microservizi	47
6.2	Architettura	48
6.2.1	Decomposizione del sistema	49
6.2.2	Comunicazione tra servizi	50
6.2.3	Distribuzione e condivisione dei dati	50
6.2.4	Coordinamento dei servizi	52
6.2.5	Gestione dei guasti	53
6.3	Servizi RESTful	54
6.4	Deploy dei servizi	55
6.4.1	Monitoraggio	56
6.5	Architectural smell	56
6.5.1	Smell e refactoring	57
6.5.2	Una toolchain per i microservizi	58
7	Sicurezza e privacy	60
7.1	Attacchi e difese	61
7.1.1	Injection	61
7.1.2	Session hijacking	61
7.1.3	Cross-site scripting (XSS)	62
7.1.4	Denial of Service (DoS)	62
7.1.5	Brute force	62
7.2	Autenticazione e autorizzazione	63
7.2.1	Autenticazione	63
7.2.2	Autorizzazione	64
7.3	Crittografia	65
7.3.1	TLS	66
7.3.2	Quando e dove cifrare	66
7.4	Privacy	67
7.5	Security smell	68
7.5.1	Le sfide della sicurezza nei microservizi	68

7.5.2	Smell e refactoring	69
7.5.3	Fare refactoring o no?	71
8	Business Process Modeling	72
8.1	BPMN	72
8.2	Workflow net	75
8.2.1	Reti di Petri	76
8.2.2	Workflow net e soundness	77
8.2.3	Verificare la soundness	78
9	Testing	80
9.1	Test funzionale	81
9.1.1	Unit testing	81
9.1.2	Feature testing	82
9.1.3	System testing	82
9.1.4	Release testing	83
9.2	Test di sicurezza	83
9.3	Automazione dei test	84
9.4	Test-Driven Development	85
9.5	Code review	86
10	DevOps	88
10.1	Gestione del codice	89
10.1.1	Sistemi centralizzati e distribuiti	90
10.2	Automazione DevOps	91
10.2.1	Continuous integration	92
10.2.2	Continuous delivery e deployment	93
10.2.3	Infrastructure as code	94
10.3	Misurazioni DevOps	95
11	Paradigmi emergenti	97
11.1	Cloud-Edge Continuum	97
11.1.1	Monitoraggio	97
11.1.2	Gestione decentralizzata	98
11.2	Quantum Software Engineering	98
11.2.1	Quantum Broker	98

Capitolo 1

Prodotti software

Capitolo originale: 1 — Software products • Antonio Brogi
Fonte: Appunti di Emiliano Sescu (github.com/Faxatos), cap. 1

1.1 Software engineering project-based

L'ingegneria del software **project-based** (orientata al progetto) è la forma più antica di ingegneria del software. Si parte da una richiesta del cliente e, alla fine del progetto, si consegna al cliente il software richiesto.

Il problema principale è che il cliente deve scrivere i **requisiti** (cioè la descrizione di cosa il software deve fare) in un formato preciso, parlando di cose che di solito non conosce: il cliente non ragiona in termini di requisiti software, ma è costretto a produrli. In più i requisiti **non sono fissi**: cambiano a ogni incontro, perché il cliente capisce via via cosa il software può fare per lui.

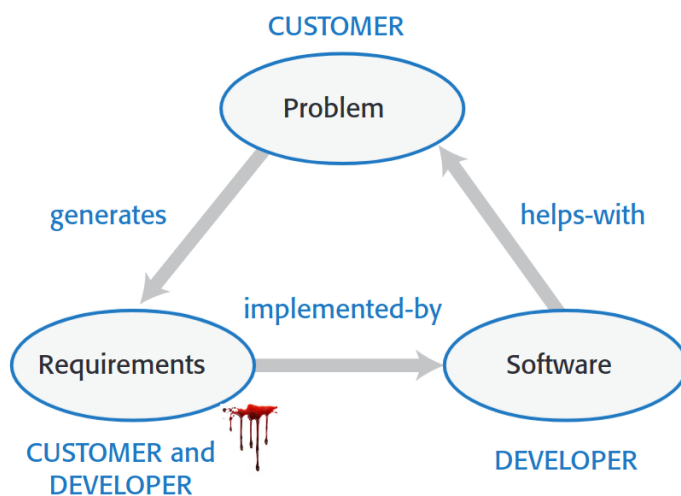


Figura 1.1: Ciclo di vita del software project-based.

Con il tempo gli sviluppatori tendono a passare da tanti prodotti fatti su misura per clienti diversi a **un unico prodotto** che vada bene per tutti.

Suggerimento — Software generico

Per la maggior parte delle aziende gli utenti non hanno bisogno di software su misura, ma di **software generico** che risolva problemi comuni.

1.2 Software engineering product-based

Nell'ingegneria del software **product-based** (orientata al prodotto) è il team di sviluppo a decidere quali sono le funzionalità (*feature*) del software e come cambierà nel tempo. Il cliente esce dal ciclo: sono gli sviluppatori a scegliere cosa costruire, e quindi devono individuare le feature che rispondono ai bisogni dei clienti.

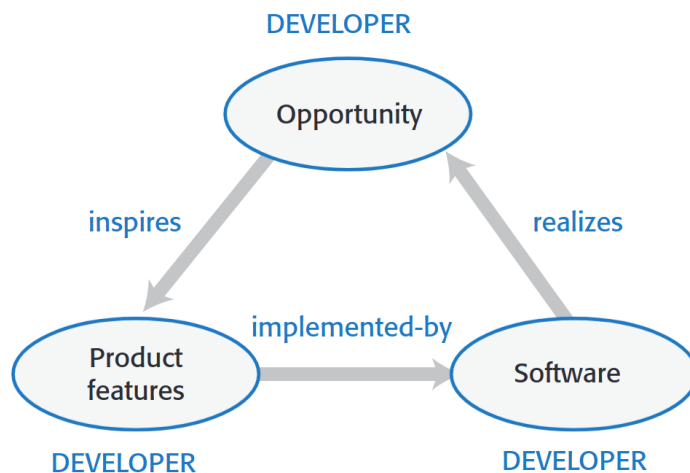


Figura 1.2: Ciclo di vita del software product-based.

Un requisito tipico dei prodotti è la **velocità**: bisogna sviluppare in fretta, altrimenti qualcun altro occupa quella fetta di mercato. Il tempo è critico, ed è per questo che si usano i **metodi agili**.

1.2.1 Modelli di esecuzione

Esistono tre modi in cui un prodotto software può essere eseguito:

- **Stand-alone**: interfaccia utente, funzionalità e dati stanno tutti sul computer dell'utente. Il fornitore (*vendor*) si occupa solo di creare e inviare gli aggiornamenti.
- **Ibrida** (*hybrid*): una parte dell'applicazione è sul dispositivo dell'utente (interfaccia, dati utente, aggiornamenti) e una parte sul server del fornitore (logica di business e backup dei dati).
- **Software as a Service (SaaS)**: l'utente non installa nulla, ha solo un'interfaccia (browser o app). Il programma gira sul server del fornitore, quindi è più facile gestirlo e aggiornarlo senza doverlo distribuire.

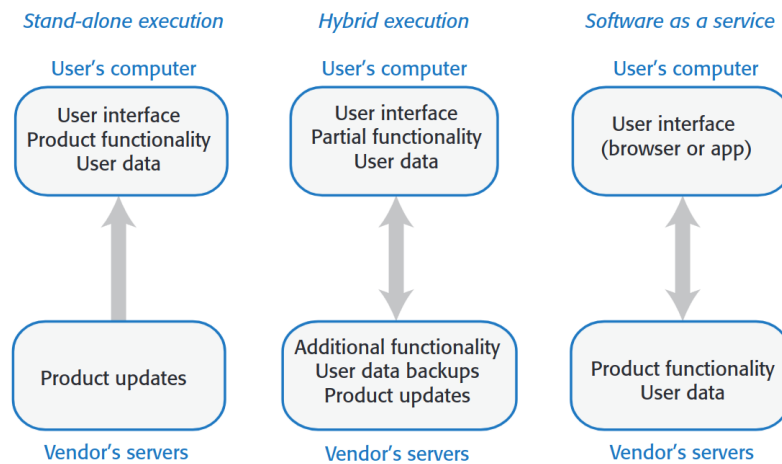


Figura 1.3: Modelli di esecuzione.

1.3 Product vision

Ogni nuovo prodotto software dovrebbe avere una **visione** (*product vision*) che risponde a tre domande: **chi** sono i clienti a cui ci rivolgiamo? **Cosa** stiamo sviluppando? **Perché** i clienti dovrebbero comprarlo?

Per gli sviluppatori è comodo riassumere la visione in una scheda con questa struttura:

Definizione — Schema della product vision

FOR (cliente target) **WHO** (bisogno o opportunità)

THE (nome del prodotto) **is a** (categoria del prodotto) **THAT** (beneficio chiave, motivo per comprarlo)

UNLIKE (principale alternativa concorrente) **OUR PRODUCT** (cosa ci distingue dai concorrenti)

Esempio — Il sistema iLearn (dal libro di Sommerville)

Per insegnanti ed educatori **che** hanno bisogno di aiutare gli studenti a usare risorse didattiche online, **iLearn** è un ambiente di apprendimento aperto **che** permette agli insegnanti stessi di configurare facilmente le risorse per le loro classi. **A differenza** degli ambienti come Moodle, centrati sulla gestione di materiali e valutazioni, **il nostro prodotto** mette al centro il processo di apprendimento.

Le fonti da cui nasce una buona visione sono:

- **Esperienza nel dominio** (*domain experience*): chi sviluppa lavora già in un certo settore (es. marketing e vendite), conosce il software che serve e ne vede i difetti, quindi intravede spazio per un sistema migliore.
- **Esperienza sul prodotto** (*product experience*): chi usa software esistente (es. un word processor) può trovare modi più semplici per offrire le stesse funzionalità. I nuovi prodotti possono anche sfruttare tecnologie recenti (es. interfacce vocali).
- **Esperienza con i clienti** (*customer experience*): parlando a lungo con i potenziali clienti si capiscono i loro problemi e i vincoli che li limitano (es. l'interoperabilità con altri sistemi), oltre alle caratteristiche per loro irrinunciabili.

- **Prototipazione e sperimentazione:** se l'idea c'è ma non è ancora chiara, un prototipo permette di esplorarla e di raccogliere feedback dai clienti (che ragionano in termini di funzionalità). Il feedback guida il prototipo successivo.

1.4 Software product management

Il **Product Manager** (PM) deve assicurarsi che il team implementi feature che portano **valore reale** ai clienti. Per farlo deve bilanciare tre forze (vedremo che alcune tecniche Agile lo aiutano):

- **Bisogni di business:** tutto lo sviluppo va gestito tenendo conto degli obiettivi sia del cliente sia dell'azienda.
- **Esperienza dei clienti:** raccogliere regolarmente il feedback dei clienti su come usano il prodotto.
- **Vincoli tecnologici:** tenere conto dei limiti tecnologici dell'azienda e dell'hardware/tecnologia che il cliente già usa.

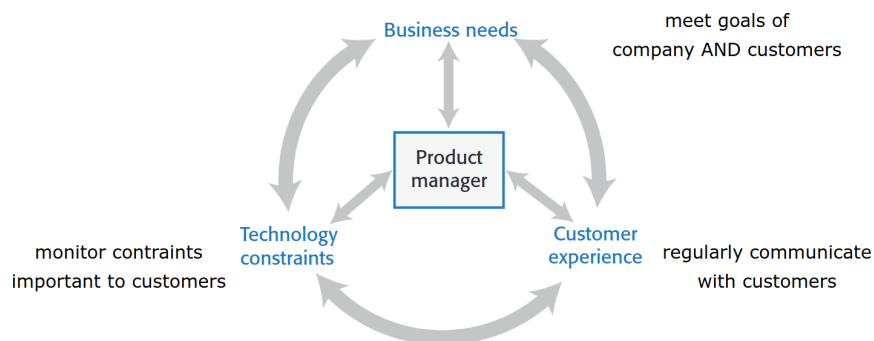


Figura 1.4: Compiti del Product Manager.

Sul piano tecnico il PM si occupa di:

- **Product roadmap:** obiettivi, milestone, criteri di successo e strade alternative se gli obiettivi non vengono raggiunti.
- **User story e scenari,** per individuare le feature del prodotto.
- **Product backlog:** la lista delle cose da fare per completare lo sviluppo.
- **Acceptance testing:** test usati lungo tutto il ciclo di vita per verificare che ogni release raggiunga gli obiettivi fissati.
- **Customer testing:** raccolta di feedback su usabilità e feature.
- **UI design,** per avere un'interfaccia semplice e naturale.

Capitolo 2

Ingegneria del software Agile

Capitolo originale: 2 — Agile Software Engineering • Antonio Brogi
Fonte: *Appunti di Emiliano Sescu (github.com/Faxatos), cap. 2*

Per circa 50 anni l'ingegneria del software è stata dominata dallo sviluppo **plan-driven** (guidato da un piano), che prevedeva:

- una pianificazione dettagliata del progetto (parte molto pesante);
- la specifica dei requisiti (anch'essa molto onerosa);
- metodi di analisi e progettazione;
- una documentazione completa del sistema;
- un controllo di qualità formale.

All'inizio degli anni 2000 viene pubblicato il **Manifesto per lo sviluppo agile del software**, che mette al primo posto:

- **Individui e interazioni** più che processi e strumenti.
- **Software funzionante** più che documentazione esaustiva. La documentazione serve a chi modificherà il codice in futuro, ma renderla dettagliata costa molto.
- **Collaborazione con il cliente** più che negoziazione del contratto: il cliente entra nel processo di sviluppo.
- **Rispondere al cambiamento** più che seguire un piano, perché nessuno può prevedere il futuro.

Nota — “Più che” non vuol dire “invece di”

Il manifesto non dice che processi, documentazione, contratti e piani siano inutili: dice che hanno valore, ma che gli elementi a sinistra ne hanno **di più**.

La vera novità di Agile è smettere di pensare le applicazioni come si faceva prima e allinearsi a come **gli utenti** vedono il sistema: un insieme di funzionalità. Uno dei principi chiave è lo **sviluppo incrementale**: si scelgono poche funzionalità, si implementano, e si ripete.

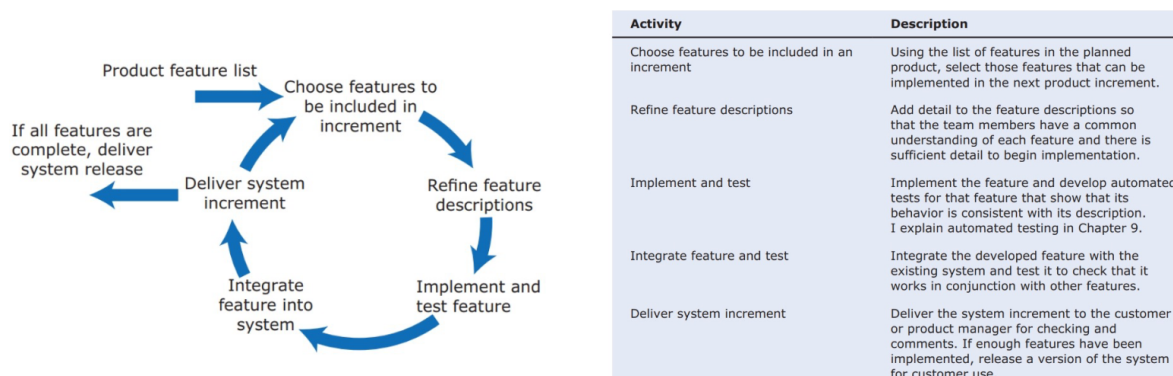


Figura 2.1: Il ciclo di Continuous Integration e Continuous Deployment.

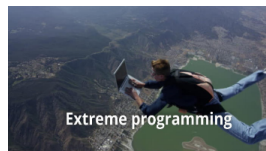
Il ciclo si ripete finché il sistema non è completo. Ogni giro può anche fallire, perché si sono capite male le funzionalità richieste o perché il cliente ha cambiato idea. Questo processo, che si può immaginare come una *pipeline*, si chiama **Continuous Integration e Continuous Delivery/Deployment (CI/CD)**.

Oltre ai quattro valori, il manifesto elenca **dodici principi** (tra parentesi il significato pratico):

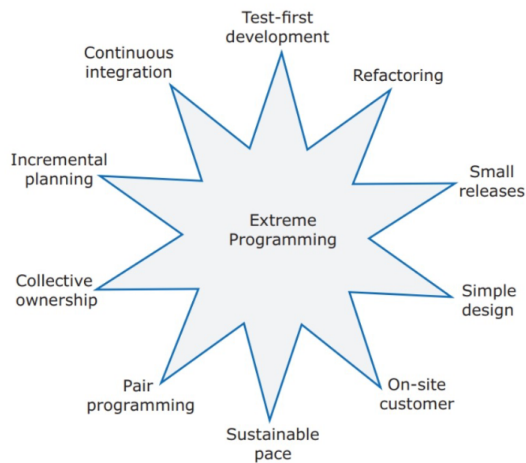
1. La priorità massima è **soddisfare il cliente** con consegne rapide e continue di software di valore (appena c'è un pezzo di software usabile, va rilasciato).
2. **I cambiamenti nei requisiti sono benvenuti**, anche a sviluppo avanzato: i processi agili li sfruttano come vantaggio competitivo per il cliente (è irrealistico avere tutte le funzionalità chiare fin dall'inizio: bisogna accettare che i requisiti cambino).
3. **Consegnare spesso software funzionante**, da ogni paio di settimane a ogni paio di mesi, preferendo i tempi più brevi (pipeline CI/CD).
4. Chi si occupa del business e gli sviluppatori devono **lavorare insieme ogni giorno** per tutto il progetto.
5. I progetti si costruiscono attorno a **persone motivate**: bisogna dare loro l'ambiente e il supporto di cui hanno bisogno e fidarsi di loro (principio assente negli appunti originali).
6. Il modo più efficace per passare informazioni al team e dentro il team è la **conversazione faccia a faccia**.
7. Il **software funzionante** è la misura principale del progresso (non la consegna di nuova documentazione o specifiche).
8. I processi agili promuovono uno **sviluppo sostenibile**: sponsor, sviluppatori e utenti devono poter mantenere un ritmo costante indefinitamente.
9. L'attenzione continua all'**eccellenza tecnica** e al buon design aumenta l'agilità.
10. La **semplicità**, cioè l'arte di massimizzare il lavoro *non* fatto, è essenziale.
11. Le migliori architetture, requisiti e design emergono da **team auto-organizzati** (invece che da team separati in stanze diverse).
12. A intervalli regolari il team **riflette** su come diventare più efficace e adatta il suo comportamento di conseguenza.

2.1 Extreme Programming (XP)

L'**Extreme Programming (XP)** è una delle prime tecniche nate all'interno della metodologia Agile. I suoi punti chiave sono riassunti nell'immagine:



Extreme Programming (XP)



Practice	Description
Incremental planning/ user stories	There is no "grand plan" for the system. Instead, what needs to be implemented (the requirements) in each increment are established in discussions with a customer representative. The requirements are written as user stories. The stories to be included in a release are determined by the time available and their relative priority.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the previous release.
Test-driven development	Instead of writing code and then tests for that code, developers write the tests first. This helps clarify what the code should actually do and that there is always a "tested" version of the code available. An automated unit test framework is used to run the tests after every change. New code should not "break" code that has already been implemented.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system and a new version of the system is created. All unit tests from all developers are run automatically and must be successful before the new version of the system is accepted.
Refactoring	Refactoring means improving the structure, readability, efficiency, and security of a program. All developers are expected to refactor the code as soon as potential code improvements are found. This keeps the code simple and maintainable.

Figura 2.2: I principi dell'Extreme Programming.

Le pratiche descritte nella tabella della figura sono:

- **Incremental planning / user stories:** non c'è un "grande piano". Cosa implementare in ogni incremento si decide con un rappresentante del cliente, scrivendo i requisiti come *user story*; le storie da includere in una release dipendono dal tempo disponibile e dalla loro priorità.
- **Small releases:** si parte dall'insieme minimo di funzionalità utile che porta valore, e le release successive, frequenti, aggiungono funzionalità un po' alla volta.
- **Test-driven development:** prima di scrivere il codice si scrivono i test per quella funzionalità. Questo chiarisce cosa deve fare il codice e fa sì che ci sia sempre una versione "testata" del codice. Un framework di test automatici controlla che il nuovo codice non rompa quello esistente.
- **Continuous integration:** appena un compito è finito, viene integrato nel sistema completo e si crea una nuova versione; tutti i test di unità devono passare prima che la nuova versione sia accettata.
- **Refactoring:** migliorare struttura, leggibilità, efficienza e sicurezza del programma. Tutti gli sviluppatori devono fare refactoring appena trovano possibili miglioramenti, per mantenere il codice semplice e manutenibile.

2.2 Scrum

Definizione — Scrum

Scrum è un framework leggero che aiuta persone, team e organizzazioni a generare valore attraverso soluzioni adattive a problemi complessi. Contiene un insieme di principi e regole da seguire per raggiungere un obiettivo comune.

La motivazione è questa: i manager di un'azienda software hanno bisogno di sapere quanto costerà sviluppare un prodotto, quanto tempo servirà e quando potrà uscire sul mercato. Lo

sviluppo plan-driven risponde con piani di lungo periodo che indicano i **deliverable** (cosa il team consegnerà e quando). Però i piani cambiano sempre: sono affidabili solo quelli di breve periodo.

Scrum si basa su due idee:

- l'**empirismo**: la conoscenza viene dall'esperienza e le decisioni si prendono in base a ciò che si osserva;
- il **pensiero Lean**: ridurre gli sprechi e concentrarsi sull'essenziale.

Usa un approccio **iterativo e incrementale** per rendere il lavoro più prevedibile e controllare il rischio. I tre **pilastr**i di Scrum sono:

- **Trasparenza**: l'avanzamento e il lavoro devono essere visibili sia a chi li svolge sia a chi li riceve.
- **Ispezione**: gli artefatti di Scrum e i progressi verso gli obiettivi concordati vanno controllati spesso e con attenzione, per scoprire problemi o deviazioni indesiderate.
- **Adattamento**: se qualcosa nel processo si discosta oltre i limiti accettabili (es. nuovi requisiti) o il prodotto non va bene, bisogna correggere il processo o ciò che si sta producendo, e farlo il prima possibile.

Scrum funziona se le persone vivono i suoi **cinque valori**: **Impegno** (*commitment*, rispettare le scadenze), **Focus**, **Apertura** (verso nuove soluzioni), **Rispetto** (per i compagni di team e gli altri) e **Coraggio** (di proporre soluzioni che allineino meglio il prodotto alla sua visione).

2.2.1 Scrum Team

Un team **auto-organizzato** coordina il lavoro discutendo i compiti e decidendo insieme chi fa cosa. Il team decide da solo tempi e deliverable, e gli sviluppatori sono coinvolti il meno possibile nelle **interazioni esterne**, cioè quelle con persone fuori dal team (management e clienti). L'idea di Scrum è che gli sviluppatori pensino solo a sviluppare: le interazioni esterne le gestiscono **Scrum Master** e **Product Owner**, così il team lavora senza interferenze o distrazioni.

Product Owner

Il **Product Owner** fa in modo che il team resti concentrato sulla costruzione del prodotto, senza perdersi in lavori tecnicamente interessanti ma poco rilevanti. Nello sviluppo di prodotti, di solito questo ruolo lo ricopre il Product Manager.

Suggerimento — Chi gestisce cosa

Le interazioni esterne **legate al prodotto** le gestisce il **Product Owner**.

Scrum Master

Lo **Scrum Master** è un esperto di Scrum che guida il team nell'usare bene il metodo. Non è un project manager tradizionale ma un **coach** del team, con autorità su *come* il team usa Scrum. In molte aziende però si occupa anche di parte della gestione del progetto.

Il motivo è pratico: tranne che nelle aziende molto piccole, il team deve riferire i progressi al management, e qualcuno deve farlo. Ruotare questo compito tra i membri non funziona, perché serve continuità nei rapporti con l'esterno. Lo Scrum Master è la persona che conosce meglio

lo stato del lavoro, quindi è quella più adatta a fornire informazioni e piani affidabili, anche se gli ideatori di Scrum non l'avevano previsto.

Suggerimento — Chi gestisce cosa

Le interazioni esterne **legate al team** le gestisce lo **Scrum Master**.

Developers

I team auto-organizzati prendono le decisioni discutendo e trovando un accordo. La **dimensione ideale** di un team Scrum è di **5-8 persone**: abbastanza per avere competenze diverse (reti, user experience, database, ...) e livelli di esperienza diversi, ma abbastanza poche per comunicare in modo informale ed efficace e mettersi d'accordo sulle priorità.

Il vantaggio di un team auto-organizzato è che diventa coeso e si adatta ai cambiamenti. Visto che la responsabilità del lavoro è del team e non dei singoli, il team regge anche l'uscita o l'ingresso di persone. Inoltre, comunicando molto, i membri imparano a conoscere le competenze degli altri.

Scrum assume che il team lavori nello **stesso spazio** e che il **daily scrum** (la riunione quotidiana) tenga tutti aggiornati. Ma due ipotesi dietro il daily scrum non sempre valgono:

- Scrum assume persone a tempo pieno nello stesso ufficio. In realtà si può lavorare part-time o da luoghi diversi (e, in un team di studenti, ognuno segue lezioni in orari diversi).
- Scrum assume che tutti possano partecipare a una riunione mattutina. In realtà qualcuno ha orari flessibili (es. per impegni familiari) o lavora a più progetti contemporaneamente.

2.2.2 Artefatti

Product Backlog

Definizione — Product backlog

Il **product backlog** è la lista delle cose da fare per sviluppare il prodotto. Viene rivisto e aggiornato prima di ogni sprint.

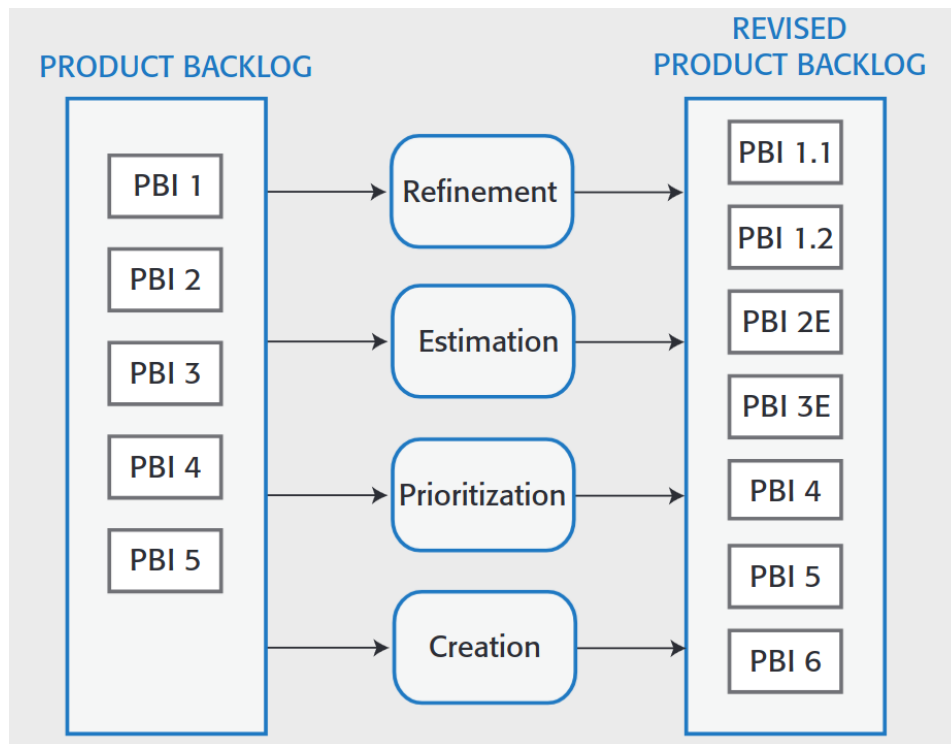


Figura 2.3: Il product backlog durante le attività sul backlog.

Come mostra la figura, gli elementi (**PBI**, *Product Backlog Item*) vengono prima selezionati e messi in ordine di priorità; in base alla priorità verranno implementati nello sprint successivo. Alcuni esempi di PBI:

- “Come insegnante, voglio poter configurare gli strumenti disponibili per ogni classe.” (*feature*)
- “Come genitore, voglio poter vedere i lavori dei miei figli e le valutazioni degli insegnanti.” (*feature*)
- “Come insegnante di bambini piccoli, voglio un’interfaccia a immagini per chi non sa ancora leggere bene.” (*richiesta di un utente*)
- “Cifrare tutti i dati personali degli utenti.” (*miglioramento tecnico*)

Un PBI può trovarsi in tre **stati**:

- **Ready for consideration:** idee e descrizioni di feature ad alto livello, ancora in valutazione. Sono provvisorie: possono cambiare o non entrare mai nel prodotto.
- **Ready for refinement:** il team è d’accordo che l’elemento è importante e va fatto nel ciclo corrente. Cosa serve è abbastanza chiaro, ma va ancora raffinato.
- **Ready for implementation:** il PBI è abbastanza dettagliato da poter stimare lo sforzo e iniziare a implementarlo. Sono note anche le dipendenze da altri elementi.

Sprint Backlog

Definizione — Sprint backlog

Lo **sprint backlog** è la lista ristretta di compiti, presi dal product backlog, che il team si impegna a completare durante lo sprint.

Si crea durante lo *sprint planning*: si scelgono gli elementi in base all'obiettivo dello sprint e li si divide in compiti più piccoli e concreti. A differenza del product backlog, che contiene tutte le possibili feature e migliorie, lo sprint backlog contiene **solo ciò che si può fare nel tempo dello sprint**. Durante lo sprint il team lo aggiorna per segnare i compiti completati e i cambiamenti.

Lo sprint backlog aiuta il team a restare allineato all'obiettivo e a gestire l'avanzamento. Per visualizzare il lavoro e il tempo rimanenti si usano spesso i **burn-down chart**.

2.2.3 Eventi Scrum

In Scrum il software si sviluppa in periodi di durata fissa chiamati **sprint**, di solito lunghi **2-4 settimane**. Durante lo sprint il team fa riunioni quotidiane (gli *scrum*) per controllare l'avanzamento e aggiornare la lista del lavoro ancora da fare. Ogni sprint deve produrre un **incremento di prodotto rilasciabile** (*shippable product increment*): il software sviluppato deve essere completo e pronto per il deploy.

Gli sprint sono **timeboxed**: lo sviluppo si ferma alla fine dello sprint, che il lavoro sia finito o no.

Le attività di uno sprint sono tre:

- **Sprint Planning**: il team sceglie gli elementi da completare nello sprint e, se serve, li raffina per creare lo sprint backlog. Non dovrebbe durare più di un giorno, all'inizio dello sprint.
- **Sprint Execution**: il team implementa gli elementi dello sprint backlog. Se non riesce a completarli tutti, lo sprint **non viene allungato**: gli elementi non finiti tornano nel product backlog per gli sprint successivi.
- **Sprint Review**: il team (ed eventualmente stakeholder esterni) rivede il lavoro fatto, ragionando su cosa è andato bene, cosa è andato male e come migliorare il processo.

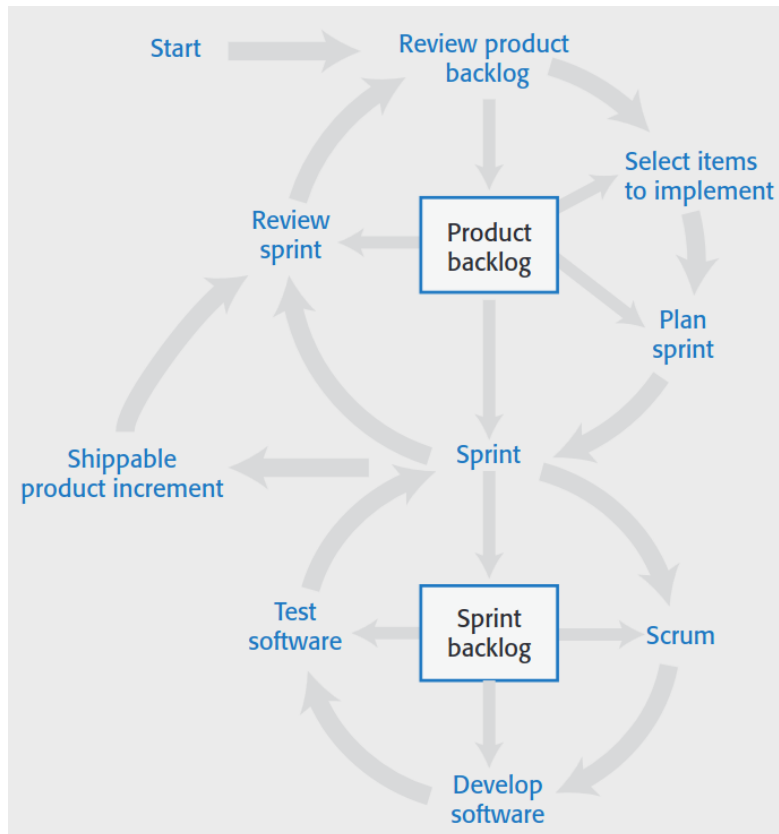


Figura 2.4: Le attività dello sprint all'interno del ciclo Scrum.

Sprint Planning

Nello sprint planning si concorda un **obiettivo dello sprint** (*sprint goal*), che può riguardare funzionalità, supporto, prestazioni o affidabilità. Per definirlo il team sceglie quali elementi del product backlog implementare. Il risultato è lo **sprint backlog**, una versione più dettagliata del product backlog con i compiti da svolgere nello sprint.

Durante lo sprint il team fa ogni giorno un **daily scrum** per coordinarsi: una riunione breve, di solito a inizio giornata, in cui ognuno racconta cosa ha fatto, che problemi ha incontrato e cosa farà oggi. Così tutti sanno cosa succede e si può ripianificare in fretta se serve.

Suggerimento — Stand-up meeting

Gli scrum devono restare brevi e concentrati. Per evitare discussioni lunghe a volte si fanno **in piedi**, senza sedie (*stand-up meeting*).

Durante lo scrum si rivede lo sprint backlog: si tolgono gli elementi completati e se ne aggiungono di nuovi se emergono nuove informazioni. Poi il team decide chi lavora su cosa quel giorno.

Sprint Execution

Scrum non impone pratiche tecniche specifiche durante lo sprint, ma due sono molto consigliate:

- **Test automation:** automatizzare più test possibile, creando una suite di test eseguibili in qualsiasi momento per controllare la qualità del software.

- **Continuous Integration:** ogni volta che si modifica un componente, lo si integra subito con gli altri per ottenere il sistema completo, che poi si testa per scoprire problemi imprevisti nelle interazioni tra componenti.

Sprint Review

Alla fine di ogni sprint c'è una **sprint review** con tutto il team. Si controlla se l'obiettivo dello sprint è stato raggiunto, si discutono i problemi nuovi emersi e si ragiona su come migliorare il modo di lavorare.

È il **Product Owner** a decidere se l'obiettivo è stato raggiunto, confermando che gli elementi scelti sono stati implementati completamente. La review include anche una **revisione del processo**: il team riflette su come ha usato Scrum e su come essere più produttivo nello sprint successivo.

2.2.4 Esecuzione: le attività sul backlog

A seconda dello stato di un PBI si possono fare diverse azioni:

- **Refinement** (raffinamento): si analizzano i PBI esistenti e li si divide in PBI più dettagliati. Può portare alla creazione di nuovi elementi.
- **Estimation** (stima): il team stima quanto lavoro serve per implementare un PBI e aggiunge la stima al PBI.
- **Creation** (creazione): si aggiungono nuovi elementi al backlog, ad esempio feature proposte dal product manager, modifiche a feature esistenti, miglioramenti tecnici o attività di processo (come valutare nuovi strumenti di sviluppo).
- **Prioritization** (priorità): i nuovi elementi vengono ordinati per priorità all'interno del backlog.

Gli effetti di queste azioni si vedono in Fig. 2.3. Per decidere lo stato o l'azione successiva di un PBI si usano alcune metriche:

- **Sforzo richiesto** (*effort*): in persone-ora o persone-giorno, cioè le ore o i giorni che servirebbero a **una** persona per implementare il PBI. Non coincide con il tempo di calendario, perché più persone possono lavorare sullo stesso elemento.
- **Story point**: una stima arbitraria dello sforzo che tiene conto di dimensione del compito, complessità, tecnologie necessarie e parti "sconosciute" del lavoro. Nascono dal confronto tra user story ma si possono usare per qualsiasi PBI. Si stimano in modo **relativo**: il team fissa i punti di un compito di riferimento e stima gli altri per confronto (più/meno complesso, più grande/più piccolo, ...).

Esempio — Sforzo vs tempo di calendario

Un PBI stimato 10 persone-giorno, se ci lavorano in due, può essere finito in circa 5 giorni di calendario.

Capitolo 3

Feature, scenari e storie

Capitolo originale: 3 — Features, Scenarios, and Stories • Antonio Brogi
Fonte: Appunti di Emiliano Sescu (github.com/Faxatos), cap. 3

Il design di un prodotto software nasce di solito da uno di questi fattori:

- l'ispirazione;
- bisogni di aziende o consumatori che i prodotti esistenti non soddisfano;
- insoddisfazione verso i prodotti esistenti;
- cambiamenti tecnologici che rendono possibili nuovi tipi di prodotto.

In questo capitolo vediamo come arrivare alla specifica del prodotto che vogliamo creare. Abbiamo già visto che discutere i requisiti con il cliente è difficile, soprattutto per la differenza di linguaggio e perché spesso il cliente non sa nemmeno quali funzionalità vuole.

Lo sviluppo product-based richiede meno documentazione di quello project-based. In Agile i requisiti non li fissa il cliente e cambiano spesso: per questo gli sviluppatori devono **individuare da soli le feature** del prodotto, capendo chi sono i potenziali utenti e come attrarli (con interviste, sondaggi, consultazioni informali, ecc.).

Per individuare le feature si usano rappresentazioni degli utenti (**personas**) e descrizioni in linguaggio naturale (**scenari e storie**).

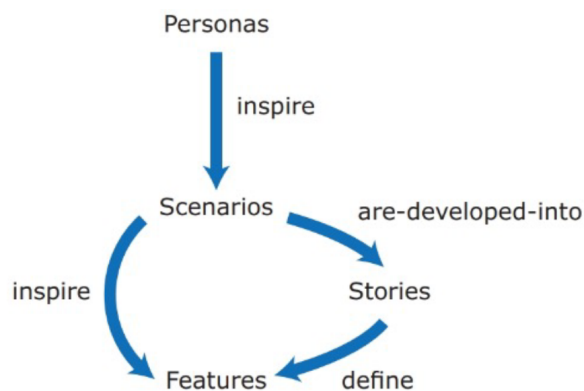


Figura 3.1: Il processo di generazione delle feature.

3.1 Personas

Definizione — Persona

Una **persona** è un utente tipo a cui si rivolge il prodotto, descritto come se fosse una persona reale.

Per progettare feature utili bisogna capire i potenziali utenti: il loro background, le loro competenze e la loro esperienza influenzano l'esperienza d'uso e l'interfaccia. Di solito bastano po-

che personas (1-2, al massimo 5) per individuare le feature principali. Le personas permettono agli sviluppatori di “mettersi nei panni degli utenti”.

Una persona si descrive con questi elementi:

- **Personalizzazione**: le si dà un nome e si descrive la sua situazione personale. A volte si usa anche una foto di repertorio: alcuni studi dicono che aiuta i team a usare meglio le personas.
- **Lavoro** (*job-related*): se il prodotto è per le aziende, si descrive il suo lavoro e (se serve) in cosa consiste. Per lavori noti a tutti, come l’insegnante, può non servire.
- **Rilevanza**: se possibile, si spiega perché potrebbe interessarle il prodotto e cosa vorrebbe farci.
- **Formazione**: si descrive il suo percorso di studi e il suo livello di competenze tecniche. È importante soprattutto per progettare l’interfaccia.

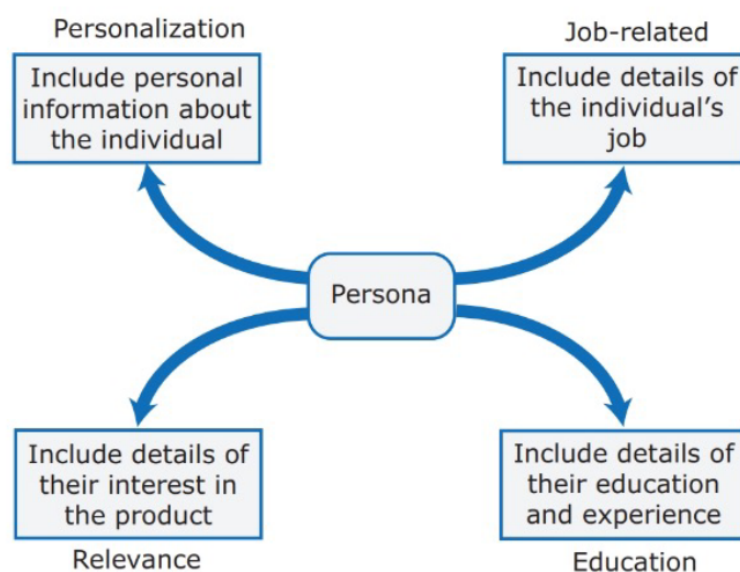


Figura 3.2: Gli elementi chiave di una persona.

L'uso della foto non è accettato da tutti: gli studi mostrano che la foto ci condiziona con dei pregiudizi. Quando non è possibile studiare gli utenti reali (ad esempio per prodotti del tutto nuovi) si creano delle **proto-personas**, cioè personas immaginate.

3.2 Scenari

Una volta create le personas, si passa agli scenari. Per scoprire le feature del prodotto si descrivono scenari di interazione tra l’utente e il prodotto.

Definizione — Scenario

Uno **scenario** è un racconto che descrive una situazione in cui un utente usa le feature del prodotto per raggiungere un obiettivo preciso.

Uno scenario contiene questi elementi:

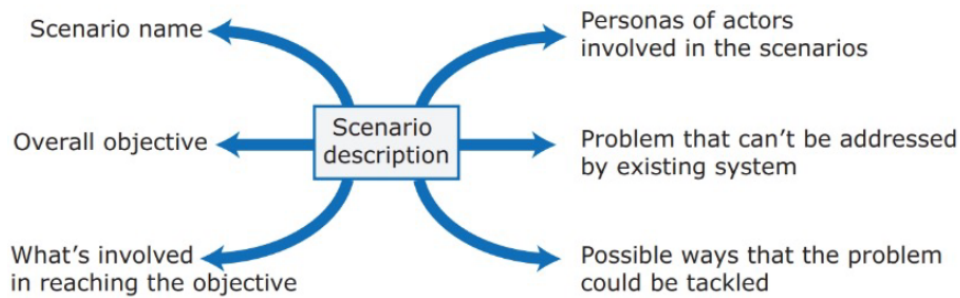


Figura 3.3: Gli elementi chiave di uno scenario.

Gli scenari narrativi e di alto livello facilitano la comunicazione e stimolano la creatività nel design. **Non sono però specifiche:** mancano di dettagli e possono essere incompleti. Di solito servono più scenari (es. 3-4) per ogni persona, che coprano le sue responsabilità principali. Gli scenari si scrivono **dal punto di vista dell'utente**.

Suggerimento — Come si lavora sugli scenari

Ogni membro del team scrive da solo alcuni scenari, poi li discute con il resto del team e, se possibile, con gli utenti.

3.3 User Stories

Gli scenari sono storie d'uso ad alto livello, mentre le **user story** sono racconti più fini e seguono una struttura precisa:

Definizione — User story

As a <ruolo> **I want to** <fare qualcosa> **so that** <motivo / valore>.
(Come <ruolo> voglio <fare qualcosa> così da <motivo / valore>.)

Le user story non fanno parte del manifesto Agile, ma aiutano a rispettarne i principi:

- la priorità massima è soddisfare il cliente con consegne rapide e continue di software di valore;
- il software funzionante è la misura principale del progresso;
- la semplicità, l'arte di massimizzare il lavoro non fatto, è essenziale.

Le user story aiutano anche a **visualizzare l'avanzamento**: si possono scrivere su post-it e attaccarle a una bacheca delle cose da fare. Va ricordato però che:

- le storie che **non creano valore** per il cliente sono lavoro che non conta come progresso;
- creare storie che **dipendono l'una dall'altra** può portare a situazioni di stallo (*deadlock*).

Il product backlog di Scrum è spesso un insieme di user story. Le storie lunghe (**epic**) vanno spezzate in storie più semplici. A ogni storia si associa una **priorità** (e magari una stima dello sforzo), e le storie si ordinano per priorità (*requirements triage*).

È possibile esprimere come user story tutte le funzionalità descritte in uno scenario, ma gli scenari si leggono in modo più naturale: rendono le storie più facili da capire, danno più contesto e sono più facili da “vendere” agli stakeholder.

3.3.1 Dalle storie alle feature

L'obiettivo finale delle user story è individuare le **feature** che definiscono il prodotto. In linea di principio una feature dovrebbe avere queste proprietà:

- **Indipendenza:** non deve dipendere da come sono implementate le altre feature, né dall'ordine in cui vengono attivate.
- **Coerenza:** deve corrispondere a una sola funzionalità. Non deve fare più di una cosa e non deve mai avere effetti collaterali.
- **Rilevanza:** deve riflettere il modo in cui gli utenti svolgono normalmente un compito. Niente funzionalità oscure che servono raramente.

Per progettare le feature servono quattro tipi di conoscenza:

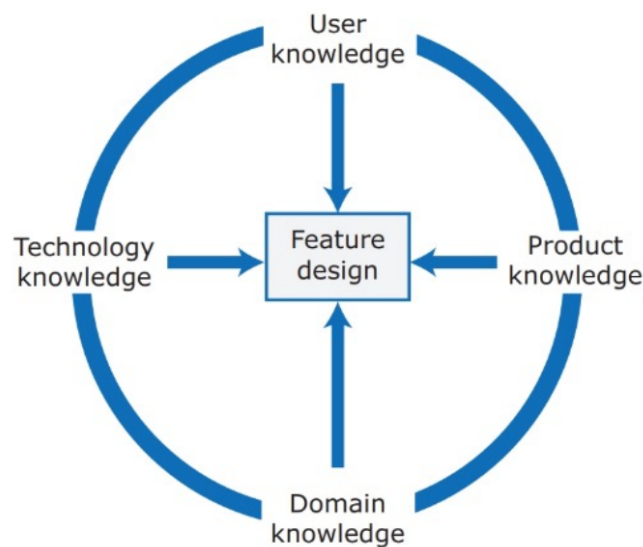


Figura 3.4: Le conoscenze necessarie per il design delle feature.

- **Conoscenza degli utenti:** scenari e user story dicono al team cosa vogliono gli utenti e come potrebbero usare le feature.
- **Conoscenza dei prodotti:** si può avere esperienza di prodotti esistenti, o studiarli durante lo sviluppo. A volte le feature devono replicare quelle dei prodotti esistenti, perché offrono funzionalità di base sempre necessarie.
- **Conoscenza del dominio:** conoscere il settore in cui opera il prodotto (es. finanza, prenotazione di eventi) permette di pensare a modi nuovi per aiutare gli utenti a raggiungere i loro obiettivi.
- **Conoscenza della tecnologia:** spesso i nuovi prodotti nascono per sfruttare tecnologie arrivate dopo i concorrenti. Chi conosce le tecnologie più recenti può progettare feature che le usano.

3.3.2 Bilanciare i fattori e il feature creep

Nel design delle feature bisogna trovare un equilibrio tra fattori opposti.

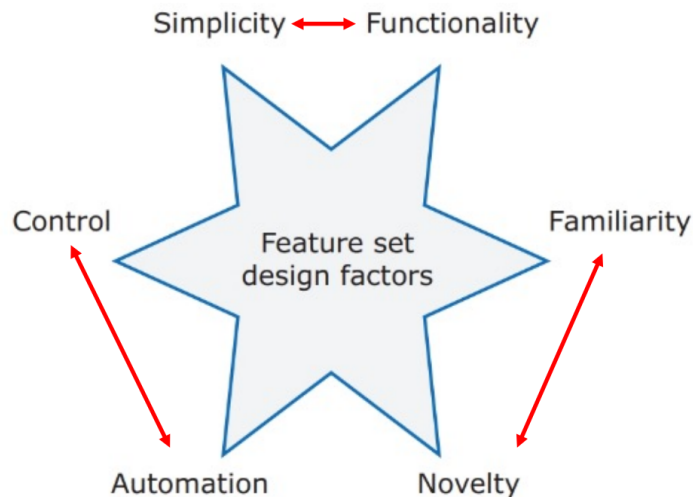


Figura 3.5: I fattori nel design delle feature.

Suggerimento — Le tre coppie in conflitto

- **Semplicità vs funzionalità:** più funzioni aggiungi, più il prodotto diventa complicato da usare.
- **Familiarità vs novità:** gli utenti vogliono ritrovare cose che conoscono, ma un prodotto nuovo deve anche offrire qualcosa di diverso.
- **Automazione vs controllo:** automatizzare semplifica la vita, ma alcuni utenti vogliono decidere da soli cosa succede.

Il numero di feature tende a crescere man mano che si immaginano nuovi potenziali utenti: questo fenomeno si chiama **feature creep**. La crescita è spinta dal marketing, che vuole accontentare tutte le richieste degli utenti, aggiungere le feature dei concorrenti e supportare sia utenti esperti sia principianti. Per evitare il feature creep conviene porsi alcune domande su ogni feature, che aiutano a capire quali sono inutili.

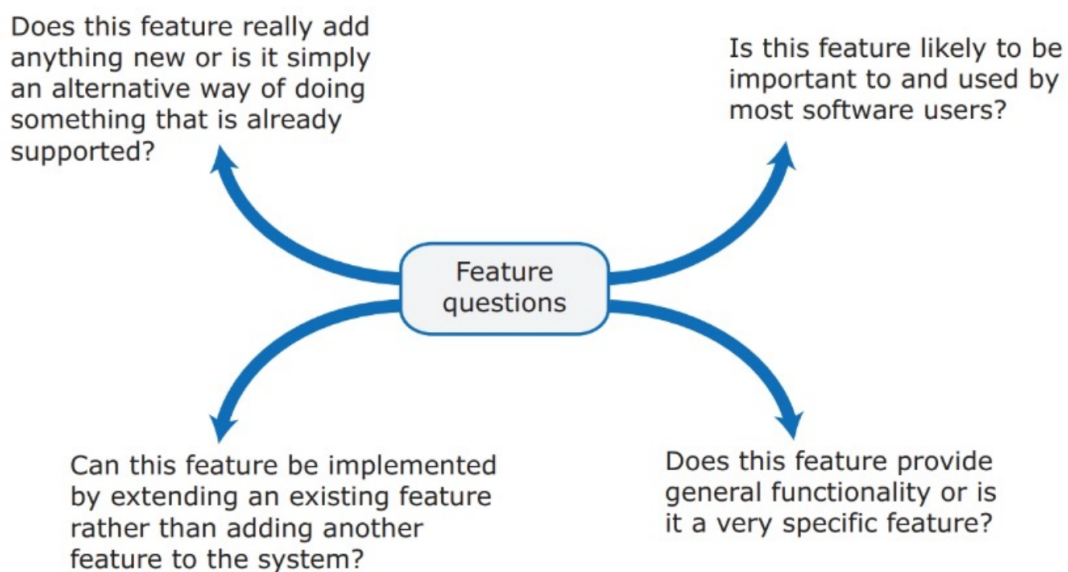


Figura 3.6: Le domande sulle feature per evitare il feature creep.

3.3.3 Derivare le feature

Il team di sviluppo si riunisce per discutere scenari e storie e ricavare la lista delle descrizioni delle feature. Per ogni feature si individuano l'**input** e il modo in cui viene **attivata**, si descrive l'**azione** (come vengono elaborati i dati) e infine si scrive l'**output** atteso.

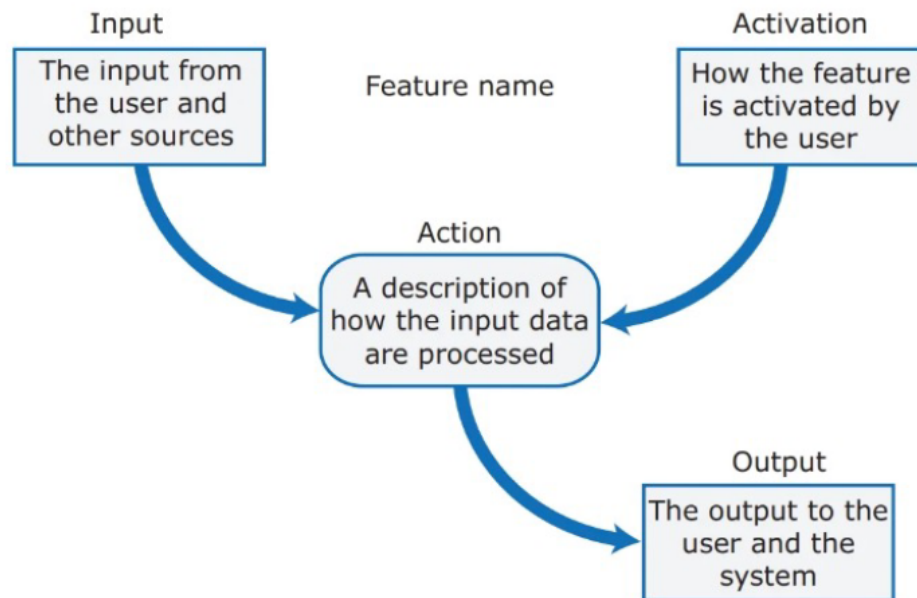


Figura 3.7: Derivazione di una feature.

Capitolo 4

Architettura software

Capitolo originale: 4 — Software Architecture • Antonio Brogi

Fonte: *Appunti di Emiliano Sescu (github.com/Faxatos), cap. 4*

Definizione — Architettura software

L'**architettura** è l'organizzazione fondamentale di un sistema software: i suoi **componenti**, le loro relazioni tra loro e con l'ambiente, e i principi che ne guidano il design.

L'architettura influenza prestazioni, usabilità, sicurezza, affidabilità, manutenibilità e così via. È fatta di **componenti**, dove ogni componente implementa un insieme coerente di funzionalità (**servizi** che altri componenti possono usare). Ecco un esempio di come si usano i componenti:

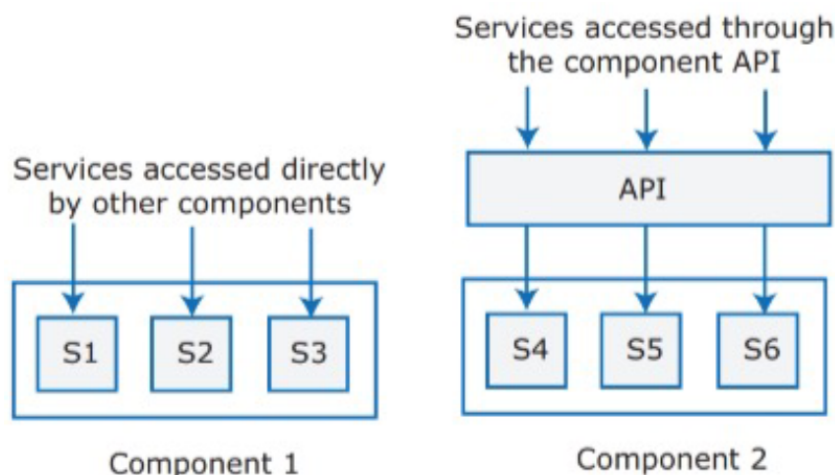


Figura 4.1: Chiamare i servizi di un componente tramite API.

Quando il software è grande bisogna sviluppare delle **API** (*Application Programming Interface*) per far comunicare i componenti, che potrebbero anche essere realizzati con tecnologie diverse.

Nel progettare il software si tengono in considerazione diversi aspetti:

- **Caratteristiche non funzionali:** sicurezza, prestazioni ecc. riguardano tutti gli utenti. Se si sbagliano, difficilmente il prodotto avrà successo commerciale. Ognuna di queste caratteristiche ha un **costo** (disponibilità, sicurezza, ...).
- **Durata del prodotto:** se si prevede che il prodotto duri a lungo, bisognerà rilasciare revisioni regolari. Serve quindi un'architettura che possa **evolvere** e accogliere nuove feature e tecnologie.
- **Riuso del software:** riusare grandi componenti di altri prodotti o software open source fa risparmiare molto tempo, ma **limita le scelte** architetturali, perché il design deve adattarsi al software riusato.

- **Numero di utenti:** per software consumer distribuito via Internet il numero di utenti può cambiare molto in fretta. Se l'architettura non permette di **scalare** rapidamente in su e in giù, le prestazioni crollano.
- **Compatibilità:** per alcuni prodotti è importante restare compatibili con altri software, così gli utenti possono adottarlo usando dati preparati con un altro sistema. Questo può limitare le scelte (es. il database da usare).

4.1 Attributi di qualità non funzionali

Le **caratteristiche non funzionali** descrivono le capacità operative del sistema e i suoi vincoli, e cercano di migliorarne il funzionamento. Le più comuni sono:

- **Responsiveness** (reattività): il sistema restituisce i risultati in un tempo ragionevole?
- **Reliability** (affidabilità): le feature si comportano come si aspettano sviluppatori e utenti?
- **Availability** (disponibilità): il sistema riesce a fornire i suoi servizi quando gli utenti li chiedono?
- **Security** (sicurezza): il sistema protegge se stesso e i dati degli utenti da attacchi e intrusioni?
- **Usability** (usabilità): gli utenti riescono ad accedere alle feature che servono e a usarle in fretta e senza errori?
- **Maintainability** (manutenibilità): il sistema si può aggiornare e ampliare con nuove feature senza costi eccessivi?
- **Resilience** (resilienza): il sistema continua a fornire servizi se una parte si guasta o subisce un attacco esterno?

Ogni attributo non funzionale che si implementa va anche **testato**, e questo aumenta il costo del prodotto. Inoltre ottimizzarne uno di solito ne peggiora altri: ad esempio, per avere più sicurezza con chiavi più lunghe si sacrificano un po' di prestazioni o di usabilità.

Nota — I prototipi

I prototipi non devono rispettare queste caratteristiche: l'obiettivo è costruirli in fretta.

Una buona pratica per la **manutenibilità** è dividere il sistema in parti piccole e autonome ed **evitare strutture dati condivise**. Senza un database centralizzato, se un componente deve cambiare l'organizzazione dei suoi dati gli altri non ne risentono, e il sistema può continuare a funzionare in parte anche se un database si guasta.

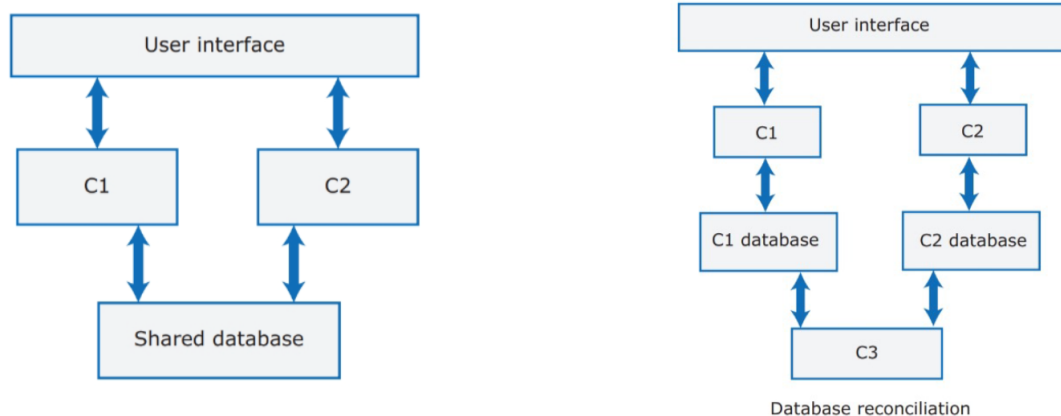


Figura 4.2: Partizionamento del database.

Il prezzo da pagare è la **consistenza finale** (*eventual consistency*): se il componente C1 fa una modifica (vedi Fig. 4.2), prima o poi quella modifica deve arrivare anche al database di C3, e questo è un costo in più. Lo stesso vale per la disponibilità: per averne di più si possono usare più database, ma poi vanno sincronizzati. Quando qualcosa va storto in un sistema distribuito è difficile annullare (*rollback*) le transazioni: per questo di solito **si rinuncia alla consistenza per ottenere disponibilità**.

Suggerimento — Collegamento: teorema CAP

È lo stesso compromesso descritto dal **teorema CAP**: in presenza di partizioni di rete un sistema distribuito non può garantire insieme consistenza e disponibilità, quindi deve sceglierne una.

4.2 Decomposizione del sistema

Ricapitolando i termini:

- un **servizio** è un'unità coerente di funzionalità;
- un **componente** è un'unità software che offre uno o più servizi;
- un **modulo** è un insieme di componenti.

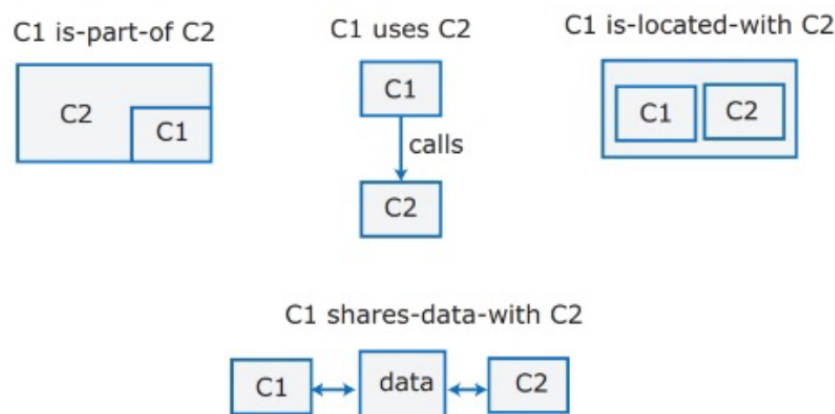


Figura 4.3: Esempi di relazioni tra componenti.

Quando i componenti aumentano, le relazioni tra loro aumentano ancora più in fretta. Per questo bisogna decomporre **quanto serve, non di più**.

Si decompone per avere parti più semplici, da mantenere separatamente. Una buona guida è la **separazione delle responsabilità** (*separation of concerns*): lo vedremo con i microservizi, ognuno dei quali implementa una sola funzionalità di business. Si vogliono anche **interfacce stabili**, da cambiare solo se davvero necessario. Il principio “**implementa una volta sola**” è in contrasto con l’idea di team indipendenti che lavorano su sistemi indipendenti: non ci sono regole fisse, bisogna sempre trovare un compromesso.

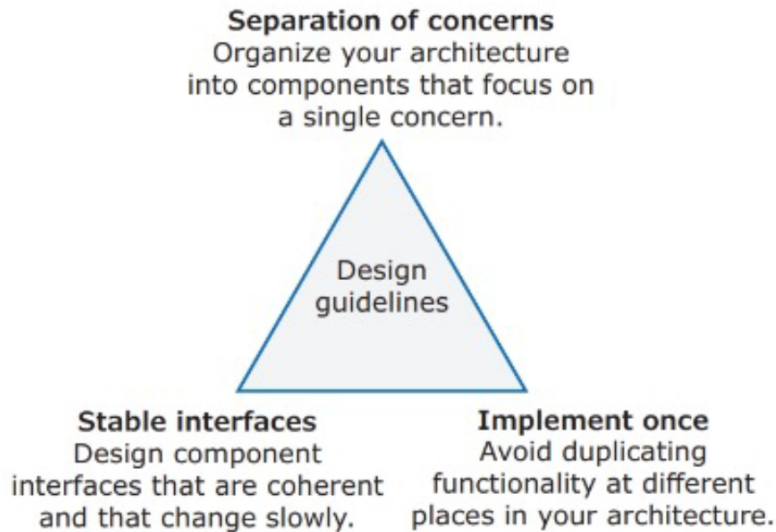


Figura 4.4: Linee guida di design per controllare la complessità.

4.2.1 Architettura a livelli

Un altro aspetto della decomposizione è l'**architettura a livelli** (*layered architecture*). Ogni livello è un'area di responsabilità e si considera separatamente dagli altri. Dentro ogni livello i componenti sono indipendenti e non si sovrappongono nelle funzionalità. Il modello architetturale è di alto livello e **non contiene dettagli implementativi**. Di solito gli architetti partono da livelli generici e li adattano, aggiungendone o togliendone, finché l'architettura non va bene per i suoi obiettivi.

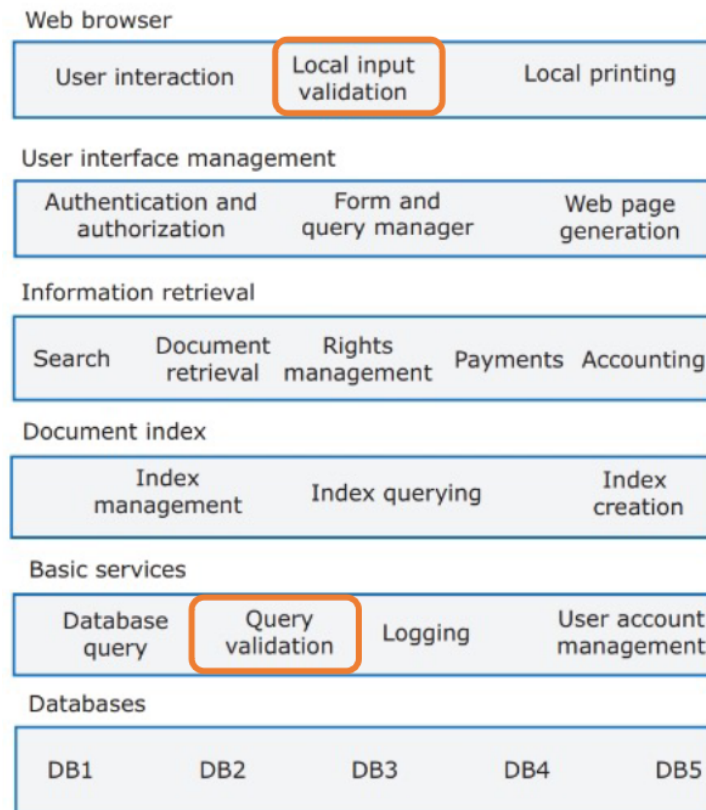


Figura 4.5: Esempio di architettura a livelli.

Alcune questioni riguardano l'intero sistema, quindi ogni livello deve tenerne conto. Queste **cross-cutting concerns** (sicurezza, prestazioni, affidabilità, validazione, ...) creano interazioni tra i livelli. Nell'esempio di Fig. 4.5 la validazione compare in più livelli.

Attenzione — Il dilemma dell'implementazione

Quando si progetta l'architettura non si vuole parlare di implementazione, per non vincolare il design con dettagli tecnici. Però bisogna anche pensare alle possibili implementazioni per capire come progettare: ad esempio, scegliere un database relazionale influenza i componenti dei livelli superiori.

4.3 Architettura di distribuzione

Definizione — Architettura di distribuzione

L'**architettura di distribuzione** definisce i server e come i componenti vengono assegnati ai server, cioè dove il software sarà installato ed eseguito in produzione.

Il tipo più comune è l'**architettura client-server**, adatta alle applicazioni in cui i client accedono a un database condiviso ed eseguono logica di business su quei dati. I client interagiscono con dei **load balancer** (bilanciatori di carico), installati su un cluster di server.

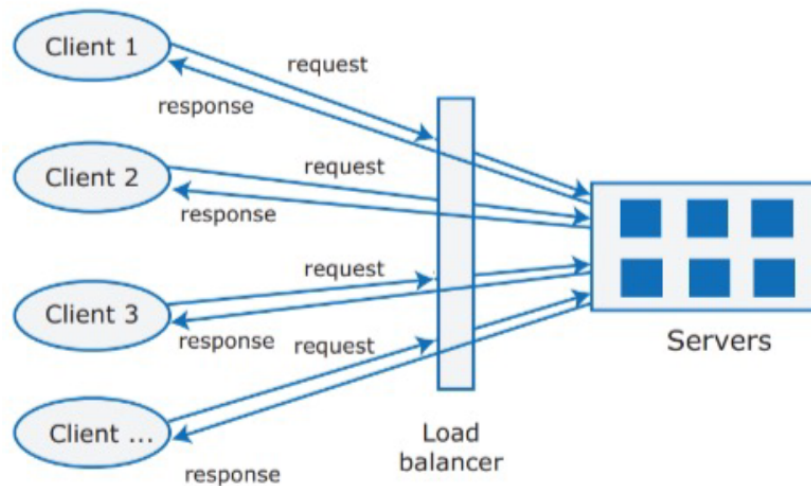


Figura 4.6: Esempio di architettura client-server.

Un pattern molto usato è il **Model-View-Controller (MVC)**, che struttura il codice dividendolo in tre parti: **Model**, **View** e **Controller**. In molti casi aiuta anche a capire come implementare la comunicazione tra le parti, e questo influisce sulla semplicità ed efficienza dell'architettura (spesso la comunicazione avviene con **HTTP + JSON**).

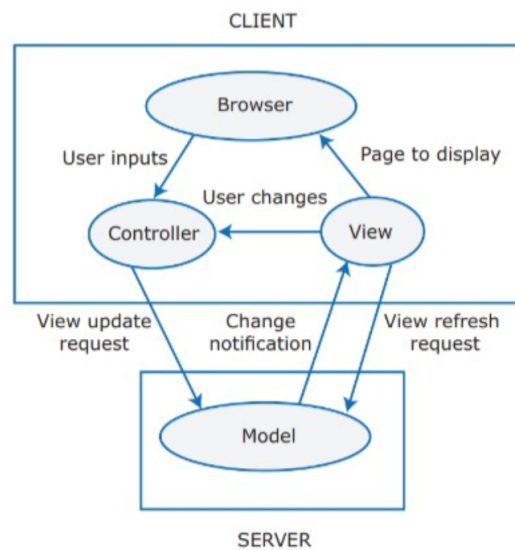


Figura 4.7: Model-View-Controller.

Il pattern di Fig. 4.7 **separa la logica di presentazione dei dati dalla logica di business**. Si colloca al livello logico/di business e si usa nelle architetture multi-tier.

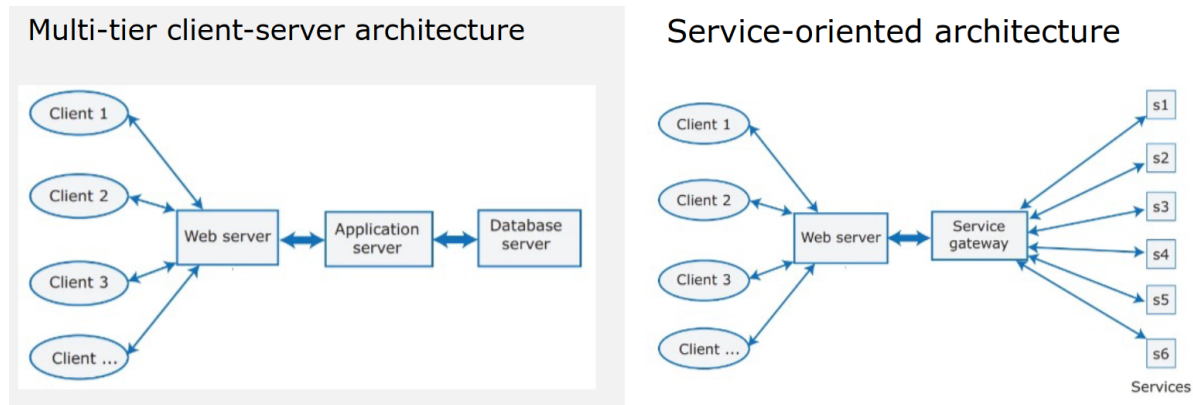


Figura 4.8: Tipi comuni di architettura client-server.

L'architettura client-server si può realizzare in vari modi (Fig. 4.8). L'architettura orientata ai servizi (*Service-Oriented Architecture*, SOA) è leggermente diversa: ha un **service gateway** che smista le richieste al servizio giusto.

La scelta dell'architettura di distribuzione dipende da:

- **Tipo di dati e aggiornamenti:** se si usano soprattutto dati strutturati modificati da più feature, di solito conviene un **unico database condiviso** che gestisca lock e transazioni. Se i dati sono distribuiti tra i servizi, serve un modo per tenerli consistenti, e questo appesantisce il sistema.
- **Piattaforma di esecuzione:** se il sistema girerà nel cloud con utenti che accedono via Internet, di solito conviene un'architettura a servizi, perché scalare è più semplice. Se è un sistema aziendale che gira su server locali, può andare meglio un'architettura **multi-tier**.
- **Frequenza dei cambiamenti:** se si prevede che alcuni componenti vengano cambiati o sostituiti spesso, isolarli come servizi separati (dentro container) rende i cambiamenti più semplici.

4.4 Scelte tecnologiche

Alcune scelte tecnologiche sono particolarmente importanti, perché se il design del prodotto cambia bisogna cambiare anche loro, con più complessità e costi durante lo sviluppo. Le principali sono:

- **Database:** si può scegliere un database relazionale **SQL** (dati organizzati in tabelle strutturate: adatto quando servono le transazioni e le strutture dati sono prevedibili e semplici) oppure un database non strutturato **NoSQL** (organizzazione dei dati più flessibile e definita dall'utente, anche gerarchica: più efficiente per l'analisi dei dati e per l'elaborazione concorrente di "big data").
- **Piattaforma di rilascio** (*delivery platform*): dove girerà il prodotto, web o mobile. Ogni piattaforma ha i suoi problemi (sul mobile: connessione intermittente, potenza del processore, gestione della batteria, schermo piccolo, tastiera a schermo). Conviene decidere presto per iniziare prima lo sviluppo.
- **Server:** il prodotto girerà in un cloud pubblico o su server interni? La maggior parte dei prodotti consumer usa **microservizi** in container, installati nel cloud (architettura SOA). I prodotti aziendali invece si preoccupano di più della sicurezza nel cloud (dove e come vengono salvati i dati dei clienti). Deciso di andare nel cloud, bisogna scegliere il provider: attenzione, poi è difficile spostarsi da un provider all'altro (**vendor lock-in**).

- **Open source:** esistono componenti open source adatti da integrare nel prodotto? Sono abbastanza sicuri?
- **Strumenti di sviluppo:** le tecnologie di sviluppo (toolkit mobile, framework web, ...) influenzano l'architettura. Anche le tecnologie che gli sviluppatori conoscono già finiscono per influenzarla indirettamente.

4.5 Enterprise Application Integration

Le **applicazioni enterprise** sono grandi sistemi software pensati per lavorare in un'organizzazione (aziende, pubblica amministrazione). L'utente vede solo il front-end, ma dietro c'è molta funzionalità di back-end fatta di tanti servizi. Questi servizi eterogenei si possono classificare come:

- **source:** servizi che fanno solo richieste;
- **sink:** servizi che rispondono solo alle richieste;
- un misto dei due.

I servizi comunicano con tipi di dati eterogenei (con rappresentazioni diverse anche per dati simili), e queste informazioni possono essere condivise via rete con altre organizzazioni. Tutta questa complessità fa delle applicazioni enterprise delle **applicazioni distribuite multi-servizio**, i cui servizi devono lavorare insieme ed essere integrati in modo adeguato.

La domanda architetturale è: come integrare tutto in modo **coerente, estendibile e manutenibile**? Un modo per gestire la complessità è usare i **pattern**.

Definizione — Pattern

Un **pattern** è un'astrazione di alto livello di una soluzione accettata e riusabile per un problema ricorrente. Si descrive con:

- **problema:** quale problema risolve;
- **contesto:** in quale situazione (per lo stesso problema possono esistere pattern diversi);
- **forze:** perché conviene usare il pattern per quel problema;
- **soluzione:** come applicare il pattern.

L'idea dei pattern è non reinventare la ruota e usare soluzioni già consolidate. L'**Enterprise Application Integration (EAI)** è un'astrazione riusabile di soluzioni collaudate per i problemi tipici che nascono quando si integrano i componenti e i servizi di un'applicazione enterprise.

4.5.1 Pattern

Questi pattern sono stati studiati più di 20 anni fa. Tra tutti, i più usati sono i **pattern di messaggistica** (*messaging patterns*), mostrati in Fig. 4.9 (catalogo completo su enterpriseintegrationpatterns.com).

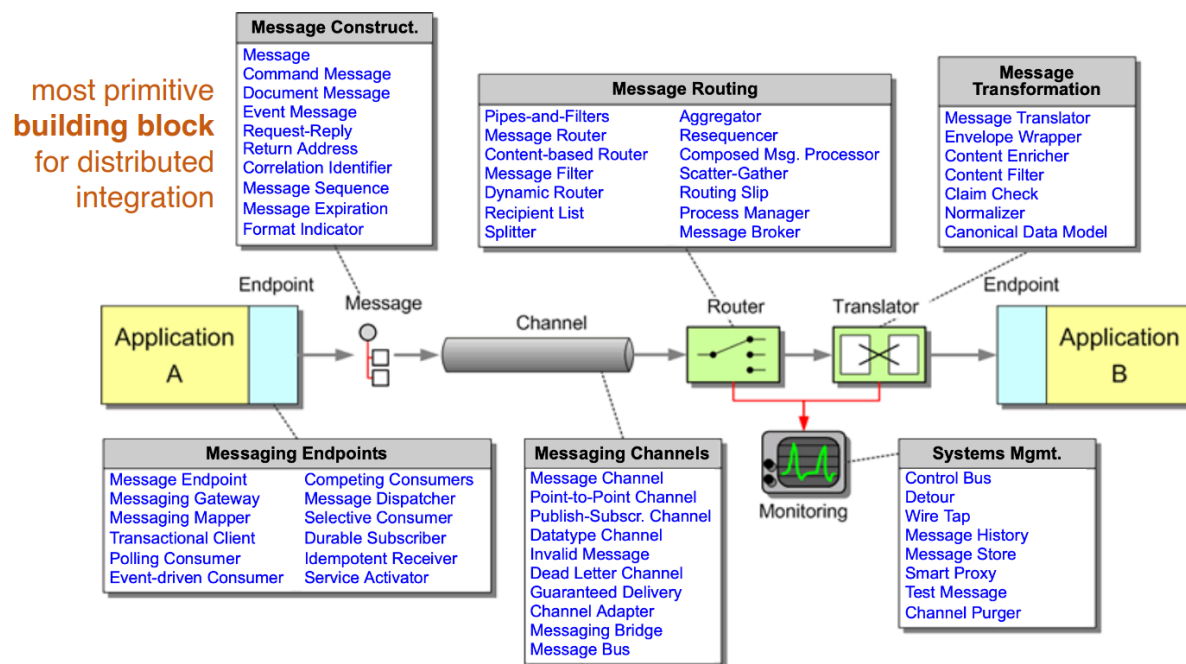


Figura 4.9: I pattern di messaggistica.

Definizione — Messaggio

Un **messaggio** è un pezzo di dati discreto inviato da un servizio a un altro. Di solito è formato da un **header** (metadati) e un **body** (il contenuto, *payload*).

Esempi concreti sono i **document message** (solo dati), gli **event message** (notifiche di eventi) e i **command message** (comandi per eseguire un'azione).



Figura 4.10: Esempi di messaggi concreti.

Scambiandosi messaggi, i servizi non sono legati strettamente tra loro: la comunicazione a messaggi permette l'**accoppiamento debole** (*loose coupling*). Si realizza con i **canali** (*channel*): astrazioni che portano i messaggi da una sorgente a una destinazione (implementate in vari modi: RPC, HTTP, TCP, ...). I canali sono a **senso unico**, quindi la comunicazione è naturalmente **asincrona**; una comunicazione sincrona (richiesta/risposta) usa due canali.

I servizi applicativi di solito sono indipendenti dal sistema di messaggistica, quindi si usano degli **adapter** per mandare sui canali i dati specifici dell'applicazione. I **message endpoint** permettono ai servizi di inviare e ricevere messaggi dai canali. Esistono vari tipi di canale:

- **point-to-point**: garantisce che un messaggio venga ricevuto da **un solo** destinatario (è quello di Fig. 4.11);
- **publish-subscribe**: il publisher consegna una copia di ogni messaggio a **ogni** subscriber.

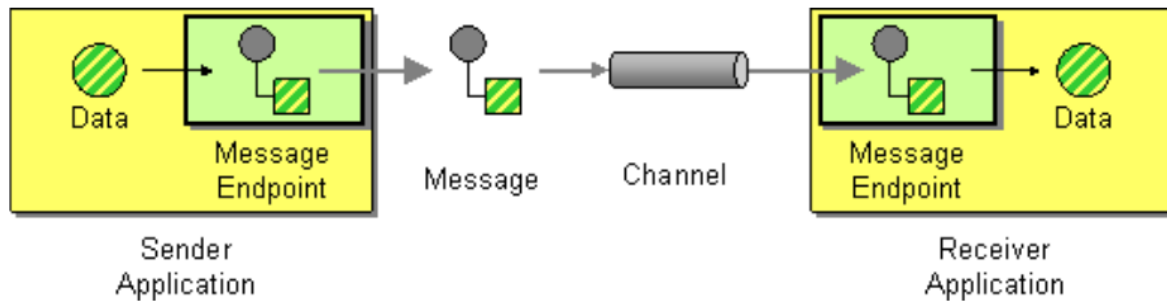


Figura 4.11: Integrazione semplice con comunicazione a messaggi.

Questo da solo però non basta: cosa succede se il servizio che riceve si aspetta un formato di dati diverso? Un **message translator** adatta il messaggio al servizio destinatario. Possono servire anche altri passaggi, ad esempio decidere come instradare i messaggi verso destinazioni diverse o multiple, come dividerli e come aggregarli.

4.5.2 Pipes and Filters

Lo stile architetturale **pipes and filters** (insieme agli altri pattern EIP) permette di strutturare le integrazioni più complesse. I messaggi attraversano più passi di elaborazione (i **filter**), e ogni componente invia i messaggi lungo i canali (le **pipe**) a cui è collegato. I messaggi scorrono nelle pipe e, quando arrivano a un filter, possono essere trasformati prima di raggiungere il **sink** (la destinazione finale).

Esempio — Loan Broker

Un esempio classico di EAI è il **Loan Broker**: arriva una richiesta di prestito, il **Credit Bureau** la valuta (punteggio di credito), la **Rule Base** decide a quali banche inoltrarla, le **banche** accettano o rifiutano il prestito. Tutte le risposte vengono aggregate e rimandate a chi ha fatto la richiesta. L'area grigia in figura rappresenta il servizio di prestito.

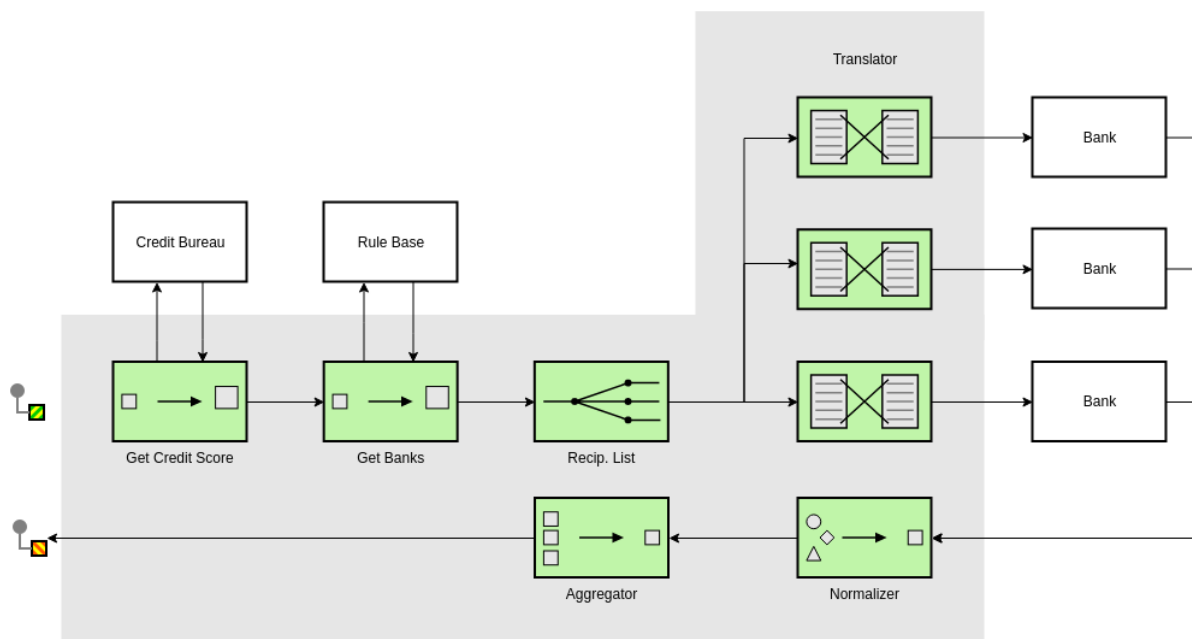


Figura 4.12: Il design del Loan Broker.

Altri pattern usati nell'esempio:

- **Content Enricher**: usa informazioni del messaggio in arrivo (es. campi chiave) per recuperare dati da una fonte esterna e li aggiunge al messaggio. Le informazioni originali si possono tenere o scartare. In Fig. 4.12 le chiamate *Get Credit Score* e *Get Banks* sono esempi di arricchimento.
- **Router**: ci sono due categorie principali. I **content-based router** instradano in base al tipo di messaggio (nell'header) o al suo contenuto (nel body). I **context-based router** instradano in base a informazioni di contesto prese da una configurazione centrale. In generale un message router è collegato a più canali e contiene la logica per decidere su quale inviare.

Message routing pattern	Consumed msgs	Published msgs	Stateful?
Message Filter	1	0 or 1	No*
Content-based Router	1	1	No*
Recipient List	1	multiple (incl. 0)	No
Splitter	1	multiple	No
Aggregator	multiple	1	Yes

Figura 4.13: Pattern di routing dei messaggi.

Una **recipient list** esamina il messaggio in arrivo, decide la lista dei destinatari e lo inoltra su tutti i canali associati a quei destinatari. Un **content-based router** invia ogni messaggio al destinatario giusto in base al suo contenuto.

- **Normalizer**: traduce i messaggi in un **formato di dati comune**. Di solito si realizza combinando più pattern: si possono comporre pattern per crearne di composti, come nel caso del normalizer.

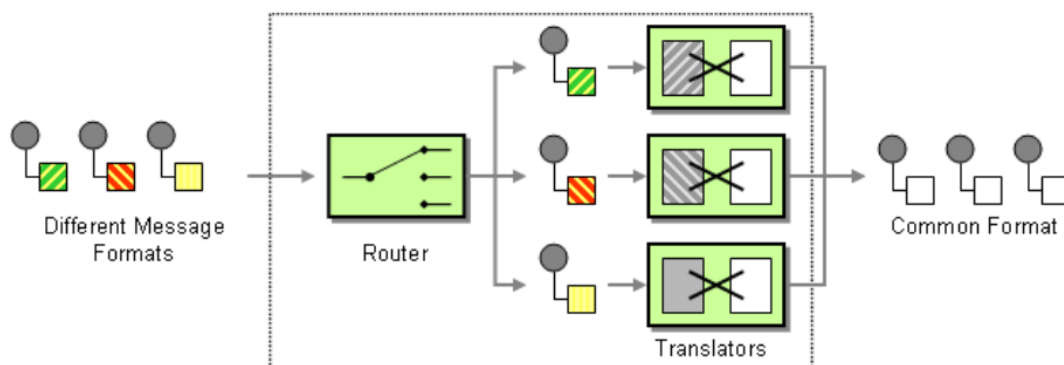


Figura 4.14: Schema del Normalizer.

- **Aggregator**: un aggregator (**con stato**) raccoglie e memorizza i singoli messaggi finché non ha ricevuto un insieme completo di messaggi collegati (vedi Fig. 4.13). Si usa quando alcuni passi dell'integrazione possono avvenire **in parallelo**: processi indipendenti girano in parallelo e i loro risultati vengono aggregati per decidere come proseguire.

Capitolo 5

Software cloud-based

Capitolo originale: 5 — Cloud-based Software • Antonio Brogi

Fonte: Appunti di Emiliano Sescu (github.com/Faxatos), cap. 5

La rivoluzione del software cloud-based nasce dall'unione di due cose: hardware potente e reti ad alta velocità, che hanno portato al **cloud computing**, e **risorse virtualizzate** (calcolo, storage, piattaforme) disponibili su richiesta.

I vantaggi principali del software nel cloud sono:

- **Scalabilità** (*scalability*): mantenere le prestazioni quando il carico aumenta (utile quando non si sa quanti clienti avrà il prodotto).
- **Elasticità** (*elasticity*): adattare la configurazione dei server alla domanda che cambia, evitando sia di avere **troppa** risorse (*overprovisioning*) sia **troppo poche** (*underprovisioning*).
- **Resilienza** (*resilience*): continuare a fornire il servizio anche se alcuni server si guastano.
- **Costo**: la maggior parte dei servizi cloud si paga a consumo (*pay-per-use*), senza comprare hardware.

5.1 Virtualizzazione e container

Uno degli ingredienti del cloud è la virtualizzazione delle risorse. Nello stesso server fisico si possono ospitare molti **server virtuali**, cioè **macchine virtuali** (VM).

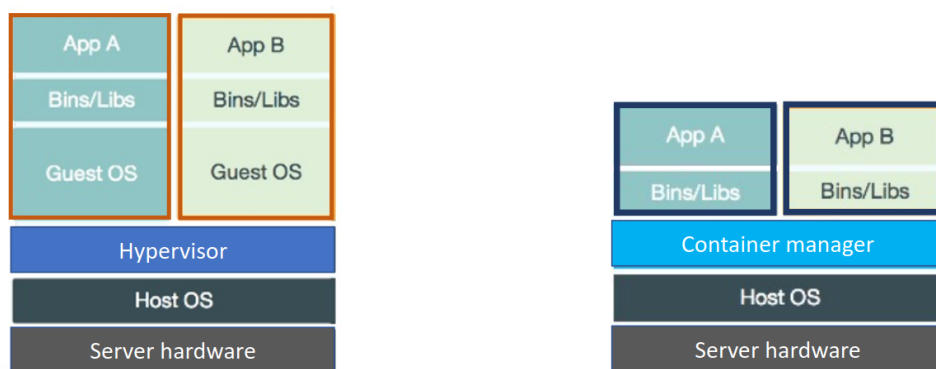


Figura 5.1: Macchine virtuali e container.

Lo stack a sinistra in Fig. 5.1 è una macchina con un sistema operativo e un **hypervisor**, il software che rende possibile la virtualizzazione. Quello in figura è un hypervisor di **tipo 2** (*hosted*), che gira sopra il sistema operativo; quello di **tipo 1** (*bare-metal*) gira direttamente sull'hardware. I riquadri arancioni sono le macchine virtuali, cioè partizioni della macchina fisica, ognuna con il proprio sistema operativo (*guest OS*).

Lo stack a destra usa i **container**: invece di isolare tutto come fanno le VM, i container **condividono lo stesso sistema operativo** e sfruttano la capacità del kernel di creare più istanze

isolate dello spazio utente. I container girano su un **container manager** che sta sopra il sistema operativo.

Suggerimento — VM vs container

Condividere il sistema operativo rende i container **molto più leggeri e veloci da avviare** delle VM. Il prezzo è una **sicurezza minore**: c'è meno isolamento, e ad esempio un container può “contaminare” la memoria di un altro.

5.1.1 Docker

La **portabilità** delle applicazioni è sempre stata uno dei grandi problemi dell'ingegneria del software. Tradizionalmente c'era un ambiente per sviluppo e test e un altro per la produzione (quello in cui l'applicazione viene installata e distribuita agli utenti). I due ambienti non erano uguali, quindi ogni volta che un'applicazione passava da uno all'altro servivano tempo e soldi.

Definizione — Docker

Docker è una piattaforma che permette di eseguire applicazioni in un ambiente isolato, usando i **container**. Un'applicazione, una volta sviluppata, viene chiusa in un container: aperto in un nuovo ambiente, il container non ha problemi di portabilità grazie al **Docker Engine** (che crea ed esegue i container). La piattaforma include anche il **Docker Hub**, per distribuire i container.

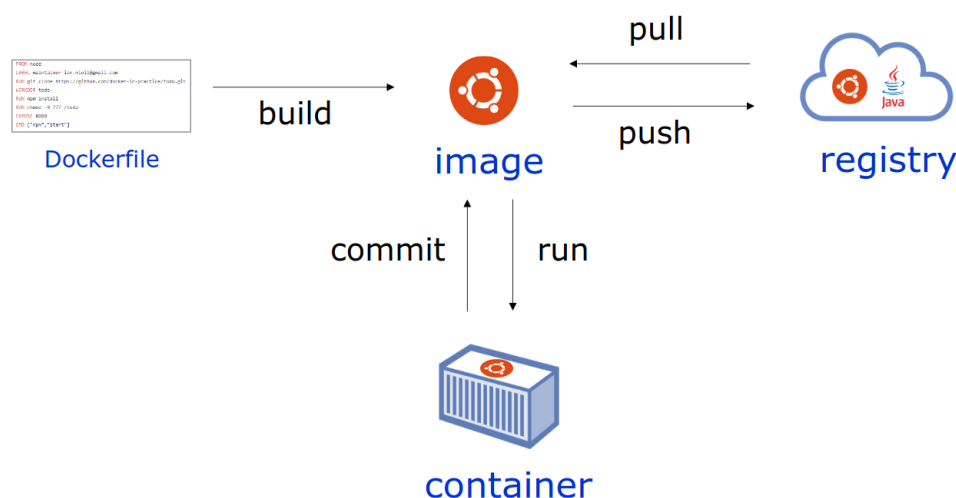


Figura 5.2: Il ciclo di vita di Docker.

I componenti software vengono impacchettati in **immagini** (*image*), cioè modelli **in sola lettura** usati per creare ed eseguire i container. Siccome le immagini sono in sola lettura, per salvare dati in modo persistente servono dei **volumi esterni**.

Le immagini sono conservate in un **Docker registry** (privato o pubblico), organizzato in **repository**: ogni repository contiene le immagini delle diverse versioni di un software, identificate dalla coppia `repository:tag`. Le immagini sono fatte a **livelli** (*layer*), e ogni livello è a sua volta un'immagine (quello più in basso si chiama **immagine base**).

I comandi principali (Fig. 5.2) sono:

- `pull`: scarica un'immagine dal registry;

- **push**: carica un'immagine nel registry;
- **build**: costruisce un'immagine a partire da un **Dockerfile** ([documentazione](#));
- **run**: esegue un'immagine in un container;
- **commit**: salva le modifiche fatte a un container come nuova versione dell'immagine.

Se un'immagine genera più processi dentro un container, la vita del container è legata al **processo principale**. Per questo è buona pratica usare **un container per processo**: quando un processo termina, gli altri non ne risentono. Se l'applicazione è fatta di più servizi, Docker offre **Docker Compose** ([documentazione](#)), che avvia un container per ogni servizio e gestisce anche la rete (i servizi locali si chiamano per nome) e la comunicazione tra i container.

5.2 Everything as a Service

Il cloud computing offre diversi modelli di servizio:

- **IaaS** (*Infrastructure as a Service*): fornisce server (virtualizzati), storage e rete. Il provider gestisce l'infrastruttura, il cliente tutto il resto (sistema operativo, applicazione, ...). Esempi: Amazon EC2 e S3.
- **PaaS** (*Platform as a Service*): fornisce una piattaforma completa (VM, sistema operativo, servizi, SDK, ...). Il provider gestisce infrastruttura, sistema operativo e software di supporto; il cliente installa e gestisce solo l'applicazione. Esempi: Heroku, Azure, Google App Engine.
- **SaaS** (*Software as a Service*): fornisce software pronto all'uso, accessibile tramite *thin client* (es. browser) o API. Il provider gestisce infrastruttura, sistema operativo e applicazione; il cliente **non gestisce nulla**, ed è per questo il modello con la quota di mercato più grande. Esempio: Salesforce.com.

La Fig. 5.3 mostra con un'analogia cosa offre ogni modello e cosa invece deve gestire il cliente.

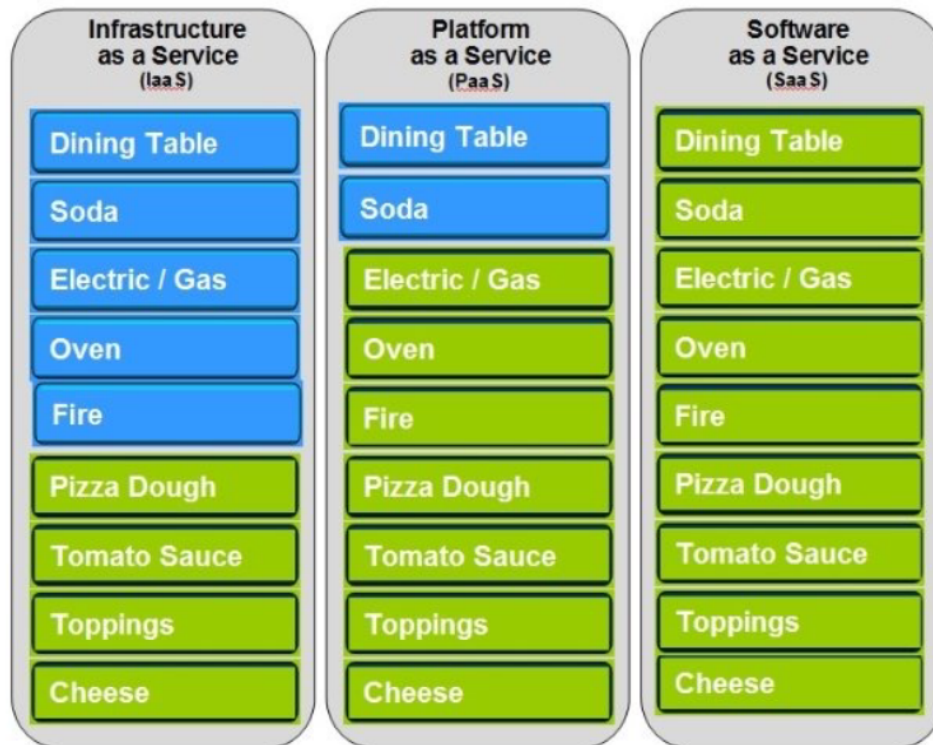


Figura 5.3: Pizza as a Service (in blu ciò che gestisce il cliente, in verde ciò che gestisce il fornitore).

5.2.1 SaaS

Prima del SaaS i prodotti software si installavano sui computer dei clienti. I clienti dovevano configurarli e gestirne gli aggiornamenti, e le aziende dovevano mantenere diverse versioni del prodotto. Con il SaaS il prodotto è fornito come servizio: il cliente non installa nulla, paga un **abbonamento** e usa il prodotto da remoto.

Vantaggi del SaaS (per il fornitore):

- **Flusso di cassa:** entrate regolari, perché i clienti pagano abbonamenti periodici o a consumo.
- **Gestione degli aggiornamenti:** è il fornitore a controllare gli aggiornamenti, e tutti i clienti li ricevono insieme. Non si devono mantenere più versioni, quindi si risparmia.
- **Continuous deployment:** si possono rilasciare nuove versioni appena le modifiche sono pronte e testate.
- **Flessibilità nei pagamenti:** diverse opzioni di pagamento attirano più clienti (es. piccole aziende o privati evitano grandi costi iniziali).
- **Prova prima di comprare** (*try before you buy*): si può offrire il prodotto gratis o a basso costo all'inizio per raccogliere feedback (es. gratis per qualche settimana).
- **Raccolta dati:** è facile raccogliere dati su come viene usato il prodotto e su come interagiscono i clienti.

Svantaggi del SaaS:

- **Rispetto delle leggi sulla privacy:** ogni Paese ha leggi diverse, spesso severe, su dove e come conservare i dati personali.

- **Sicurezza:** i clienti possono non voler cedere il controllo dei propri dati a un fornitore esterno, che però deve gestirli. E se il fornitore subisce un furto di dati (*data breach*), avviserà i clienti?
- **Limiti della rete:** possono rallentare le risposte quando si trasferiscono molti dati.
- **Scambio di dati:** se il cloud non offre API adatte, scambiare dati è difficile. Dipende dal contratto: a seconda dell'offerta si può accedere a insiemi di dati diversi.
- **Perdita di controllo sugli aggiornamenti:** il fornitore può cambiare troppo e troppo spesso (es. nuove funzioni ogni giorno che cambiano l'interfaccia).
- **Service lock-in:** è difficile spostarsi da un fornitore SaaS a un altro.

Scelto il SaaS, bisogna affrontare alcune questioni di design:

- **Elaborazione locale o remota:** alcune feature si possono eseguire in locale, riducendo il traffico di rete e velocizzando le risposte, ma aumentando il consumo di batteria sui dispositivi mobili.
- **Autenticazione:** di solito ci sono tre possibilità: un sistema di autenticazione proprio del prodotto; l'**autenticazione federata**, basata su un rapporto di fiducia tra un *Service Provider* (SP, es. chi vende l'applicazione) e un *Identity Provider* (IdP) esterno; le credenziali personali di servizi esterni (es. Google, LinkedIn).
- **Fuga di informazioni** (*information leakage*): se il servizio è offerto a più organizzazioni, c'è il rischio che un membro di un'organizzazione acceda a dati di un'altra.
- **Database multi-tenant o multi-instance:** i sistemi **multi-tenant** usano un solo archivio (con tecniche avanzate per partizionare i dati e controllare le partizioni), quelli **multi-instance** hanno copie separate del sistema e del database per ogni cliente.

5.2.2 Sistemi multi-tenant e multi-instance

Sistemi multi-tenant

Definizione — Sistema multi-tenant

Un sistema **multi-tenant** usa un **unico schema di database** condiviso da tutti gli utenti. Ogni riga è etichettata con un **identificatore del tenant** (il cliente): così si ottiene un **isolamento logico**, cioè ogni utente vede solo i dati del proprio tenant.

Stock management					
Tenant	Key	Item	Stock	Supplier	Ordered
T516	100	Widg 1	27	S13	2017/2/12
T632	100	Obj 1	5	S13	2017/1/11
T973	100	Thing 1	241	S13	2017/2/7
T516	110	Widg 2	14	S13	2017/2/2
T516	120	Widg 3	17	S13	2017/1/24
T973	100	Thing 2	132	S26	2017/2/12

Figura 5.4: Esempio di schema di database multi-tenant.

Vantaggi:

- **Uso delle risorse:** il fornitore controlla tutte le risorse usate dal software e può ottimizzarle.
- **Sicurezza:** siccome i dati di tutti i clienti stanno nello stesso database, il database deve essere progettato fin dall'inizio per la sicurezza, quindi probabilmente avrà meno vulnerabilità di un database standard. Inoltre c'è una sola copia del software da correggere se si scopre una vulnerabilità.
- **Gestione degli aggiornamenti:** aggiornare un'unica istanza è più facile che aggiornarne tante, e tutti i clienti usano sempre l'ultima versione.

Svantaggi:

- **Poca flessibilità:** tutti i clienti devono usare lo stesso schema, con poche possibilità di adattarlo.
- **Sicurezza:** con i dati di tutti nello stesso database, c'è il rischio teorico che i dati passino da un cliente all'altro. Peggio ancora, un attacco al database colpisce **tutti** i clienti.
- **Complessità:** dover gestire tanti utenti rende questi sistemi più complessi dei multi-instance, e quindi più esposti a bug.

Le aziende medie e grandi raramente vogliono software multi-tenant generico; preferiscono una versione adattata alle loro esigenze. Due soluzioni possibili:

1. **Aggiungere campi extra** a ogni tabella, che il cliente usa come vuole. Però è difficile prevedere quante colonne servono (troppo poche non bastano, troppe sprecano spazio) e clienti diversi vorranno tipi di colonne diversi.
2. **Aggiungere a ogni tabella un campo che punta a una “tabella di estensione”**, che il cliente crea secondo le sue esigenze. Questo però aggiunge complessità a un sistema già complesso.

La **sicurezza** è la preoccupazione principale dei clienti aziendali. Poiché tutto sta nello stesso database, un bug o un attacco potrebbe esporre i dati di alcuni o di tutti i clienti. Per ridurre il rischio si usano:

- il **controllo degli accessi multilivello:** l'accesso ai dati è controllato sia a livello di organizzazione sia a livello del singolo utente;
- la **cifatura dei dati:** così, se il sistema si guasta, i dati di un'azienda non sono leggibili da persone di altre aziende. Di solito si cifrano solo i dati sensibili.

Sistemi multi-instance

Definizione — Sistema multi-instance

In un sistema **multi-instance** ogni cliente ha il **proprio sistema**, adattato alle sue esigenze, con il proprio database e i propri controlli di sicurezza.

È concettualmente più semplice di un sistema multi-tenant ed elimina problemi come la fuga di dati tra organizzazioni. Si può realizzare in due modi:

- **Basato su VM:** l'istanza del software e il database di ogni cliente girano in una propria macchina virtuale. Tutti gli utenti dello stesso cliente accedono al database condiviso di quel sistema.
- **Basato su container:** ogni utente ha una versione isolata del software e del database, in un insieme di container. È la soluzione migliore per prodotti in cui gli utenti lavorano per lo più da soli, condividendo pochi dati (es. utenti privati).

Le due cose si possono anche combinare: un'azienda può avere il proprio sistema su VM e, sopra, eseguire container per i singoli utenti.

Vantaggi:

- **Flessibilità:** ogni istanza si può adattare alle esigenze del cliente.
- **Sicurezza:** ogni cliente ha il suo database, quindi niente fughe di dati tra clienti.
- **Scalabilità:** ogni istanza si scala secondo le esigenze del suo cliente (qualcuno avrà bisogno di server più potenti di altri).
- **Resilienza:** un guasto software probabilmente colpisce un solo cliente, gli altri continuano a lavorare.

Svantaggi:

- **Costo:** affittare tante VM nel cloud e gestire tanti sistemi costa di più. Le VM si avviano lentamente, quindi spesso vanno tenute accese sempre, anche quando il servizio è poco usato.
- **Gestione degli aggiornamenti:** aggiornare tante istanze è complicato, soprattutto se sono state personalizzate per clienti diversi.

5.3 Architettura del software nel cloud

Nelle prime fasi del design di un'architettura cloud bisogna rispondere ad alcune domande:

- Database **multi-tenant o multi-instance?** (organizzazione del database)
- Quali sono i requisiti di **scalabilità e resilienza?**
- Struttura **monolitica o a servizi?** (struttura del software)

Le risposte guidano la scelta della piattaforma cloud su cui installare e distribuire il prodotto.

5.3.1 Organizzazione del database

Ci sono tre modi per fornire il database dei clienti in un sistema cloud:

1. **Multi-tenant**, condiviso da tutti i clienti e ospitato nel cloud su server grandi e potenti.
2. **Multi-instance**, con il database di ogni cliente in una propria macchina virtuale.
3. **Multi-instance**, con ogni database in un proprio container. Il database di un cliente può anche essere distribuito su più container.

Fattori per scegliere:

- **Clienti target:** i clienti hanno bisogno di schemi di database diversi e di personalizzazioni? Se sì → **multi-instance**.
- **Requisiti sulle transazioni:** è fondamentale supportare transazioni **ACID**, cioè avere dati sempre consistenti? Se sì → **multi-tenant o multi-instance su VM**.
- **Dimensione e connessioni del database:** quanto è grande il database tipico? Quante relazioni ci sono tra i dati? Per database molto grandi di solito conviene il **multi-tenant**, perché si possono concentrare gli sforzi sull'ottimizzazione delle prestazioni.
- **Interoperabilità:** i clienti vorranno trasferire dati da database esistenti? Quanto sono diversi i loro schemi da un possibile schema multi-tenant? Che supporto si aspettano per il trasferimento? Se i clienti hanno schemi molto diversi → **multi-instance**.
- **Struttura del sistema:** si usa un'architettura a servizi? Il database del cliente si può dividere in tanti database, uno per servizio? Se sì → **multi-instance in container**.

5.3.2 Scalabilità e resilienza

Scalabilità

La **scalabilità** (la capacità di adattarsi automaticamente ai cambiamenti di carico) si ottiene in due modi quando il carico cresce:

- **scaling out** (scalare in orizzontale): aggiungere nuovi server virtuali;
- **scaling up** (scalare in verticale): aumentare la potenza di un server esistente.

Nel cloud si fa tipicamente **scaling out**: il prodotto deve essere organizzato in modo che i singoli componenti si possano replicare ed eseguire in parallelo, mentre dei meccanismi di **load balancing** distribuiscono le richieste tra le istanze (ad esempio con un PaaS).

Resilienza

La **resilienza** (la capacità di continuare a fornire i servizi critici in caso di guasti o attacchi) si ottiene con la tecnica dell'**hot standby**: si tengono **repliche** del software e dei dati in luoghi diversi (Fig. 5.5). Gli aggiornamenti del database vengono copiati sull'altro database (*mirroring*) e un **system monitor** controlla di continuo lo stato del sistema.

Un'alternativa più economica è il **cool standby**: in caso di guasto i dati si ripristinano da un backup, quindi il sistema resta non disponibile finché il ripristino non è finito.

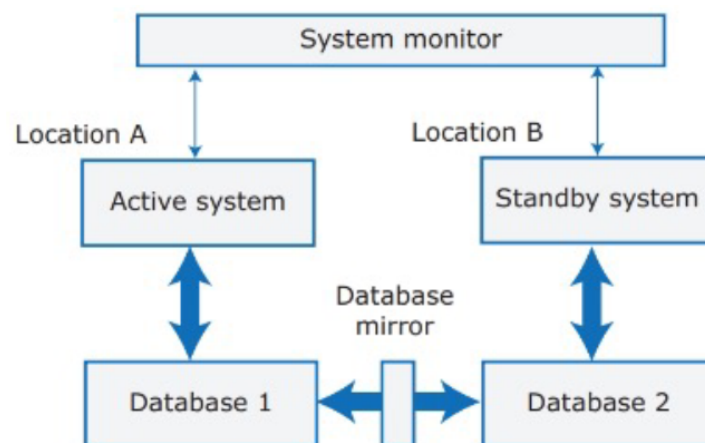


Figura 5.5: Schema della tecnica hot standby.

5.3.3 Struttura del software

La scelta principale è tra un sistema **monolitico** e **servizi piccoli e senza stato** (*fine-grained, stateless*). I servizi indipendenti si possono replicare, distribuire e spostare, quindi sono particolarmente adatti al software cloud con servizi in container. Il monolite si usa spesso per costruire i **prototipi**, anche quando alla fine un sistema a servizi sarebbe più adatto.

Per scegliere la piattaforma cloud si considerano due gruppi di aspetti:

1. **Aspetti tecnici**: carico previsto e quanto è prevedibile, resilienza, servizi cloud offerti dal provider, privacy e protezione dei dati (alcuni Paesi UE hanno regole severe su come e dove si salvano i dati, quindi il provider deve garantire dove vengono conservati).

2. **Aspetti di business:** esperienza degli sviluppatori (linguaggi e tecnologie dipendono da cosa sa il team), **Service Level Agreement** (SLA, le garanzie offerte dal provider), portabilità e migrazione (avere un “piano d’uscita” contro il vendor lock-in), clienti target (che magari hanno sistemi vecchi che devono interagire con i nuovi) e costi (inclusi quelli per sviluppatori e tecnologie).

5.4 Orchestrazione dei container

Come visto, i container sono un modo leggero per isolare l’ambiente di un’applicazione. Le immagini dei container si possono eseguire in modo affidabile su qualsiasi macchina, garantendo portabilità dallo sviluppo al deploy (noi abbiamo usato Docker).

Il deploy automatico che offre Docker, però, ha poche capacità di ottimizzare l’uso della memoria e può non sfruttare bene le risorse. E restano varie domande aperte:

- Cosa succede se un container muore?
- Cosa succede se si guasta la macchina su cui gira il container?
- Come comunicano più container tra loro, anche su nodi diversi?
- Come si gestisce la rete tra container?
- Se la produzione ha più macchine, come si decide su quale far girare un container?

La soluzione è l’**orchestrazione dei container**: l’architetto definisce quali container devono girare e poi lascia alla piattaforma di orchestrazione il compito di realizzare quella configurazione.

5.4.1 Kubernetes

Una delle piattaforme di orchestrazione più usate è **Kubernetes** (abbreviato **K8s**). Gestisce l’intero ciclo di vita dei container, creando e spegnendo risorse quando serve: se un container si spegne all’improvviso, K8s ne lancia un altro al suo posto. Offre anche un meccanismo per far comunicare le applicazioni tra loro anche mentre i container sottostanti vengono creati e distrutti. Dato un insieme di carichi di lavoro (container) e un insieme di macchine in un **cluster**, l’orchestratore esamina ogni container e sceglie la macchina migliore su cui eseguirlo.

Kubernetes riceve tramite la sua API lo **stato desiderato** (*Desired State Management*), descritto in un **manifest** YAML. Il manifest dice quali sono i **Pod** (gruppi di container) e quante **repliche** di ciascuno servono. Kubernetes mette sullo stesso nodo tutti i container di uno stesso Pod e crea tante repliche quante ne sono richieste; un nodo può ospitare più Pod. Se un nodo (*worker*, cioè una macchina che ospita container) si guasta, Kubernetes se ne accorge e avvia una nuova replica altrove (Fig. 5.6). I servizi del cluster Kubernetes gestiscono anche la comunicazione tra i nodi, sia quelli esistenti sia quelli nuovi.

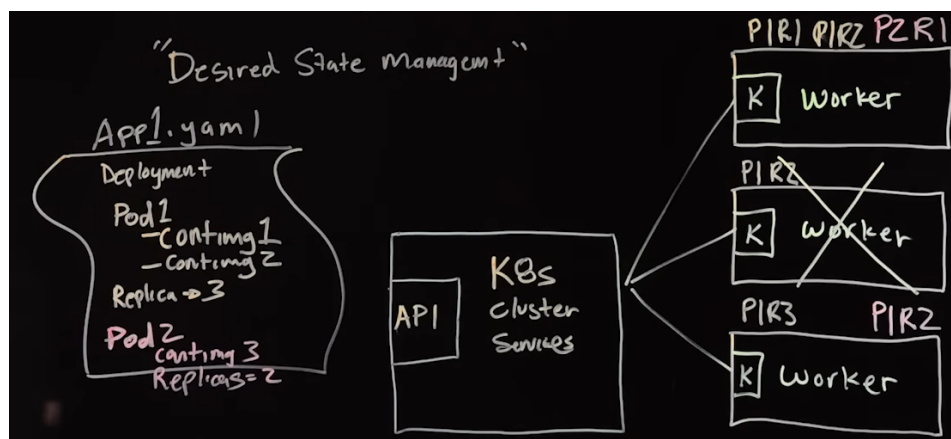


Figura 5.6: L'idea di gestione di Kubernetes.

Esempio — Un manifest minimo (Deployment)

Questo manifest chiede a K8s di tenere sempre attive 3 repliche di un Pod con un container web:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels: { app: web }
  template:
    metadata:
      labels: { app: web }
    spec:
      containers:
        - name: web
          image: nginx:1.27
```

Se un Pod muore, K8s nota che le repliche sono 2 invece di 3 e ne crea una nuova.

5.4.2 Principi di design di Kubernetes

- **Dichiaratività:** con l'approccio dichiarativo si descrive solo lo **stato desiderato** del sistema (nel manifest). K8s si accorge quando lo stato reale non corrisponde a quello desiderato e interviene per correggerlo: il sistema diventa **self-healing** (si ripara da solo). Lo stato desiderato è definito da un insieme di **oggetti**: ogni oggetto ha una **specifica** (*spec*, lo stato desiderato) e uno **stato** (*status*, lo stato attuale). K8s controlla di continuo che lo stato di ogni oggetto sia uguale alla specifica: se un oggetto non risponde ne avvia una nuova versione; se il suo stato si è allontanato dalla specifica, esegue i comandi necessari per riportarlo allo stato desiderato.
- **Distribuzione:** Kubernetes offre un'unica **interfaccia** per interagire con un intero cluster di macchine, così gli sviluppatori non devono comunicare con ogni macchina singolarmente.

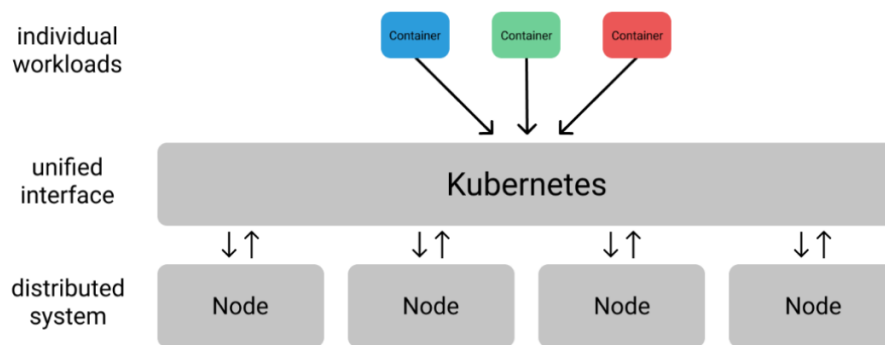


Figura 5.7: Schema della distribuzione in Kubernetes.

- **Disaccoppiamento:** ogni container dovrebbe occuparsi di **una sola cosa**, come nelle architetture a microservizi. K8s supporta in modo naturale servizi disaccoppiati, che si possono scalare e aggiornare in modo indipendente.
- **Infrastruttura immutabile:** invece di entrare in un container per aggiornare una libreria, si costruisce una **nuova immagine**, si installa la nuova versione e si spegne quella vecchia. Il motivo è che i container sono pensati per essere **effimeri**, pronti a essere sostituiti in qualsiasi momento. Un'infrastruttura immutabile rende anche facile tornare a uno stato precedente (es. dopo un errore): basta cambiare la configurazione per usare un'immagine più vecchia.

5.4.3 Oggetti di Kubernetes

Kubernetes definisce molti oggetti che si possono descrivere nei manifest (in JSON o YAML). I principali sono:

- **Pod:** la **più piccola unità di deploy** di Kubernetes. È formato da uno o più container strettamente collegati, con una rete e dei volumi (file system) condivisi. Un Pod non si distribuisce su più macchine. Ogni Pod ha un **indirizzo IP** unico per comunicare con lui.

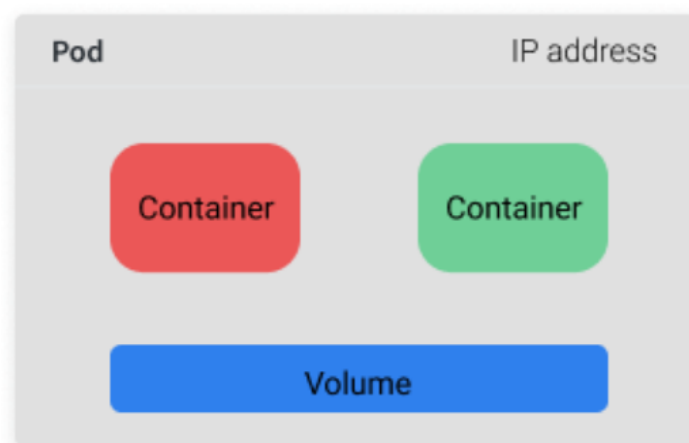


Figura 5.8: Schema di un Pod.

- **Deployment:** un insieme di Pod definito da un **template** e da un **numero di repliche n** (quante copie del template far girare). Il cluster cerca sempre di avere n Pod disponibili.

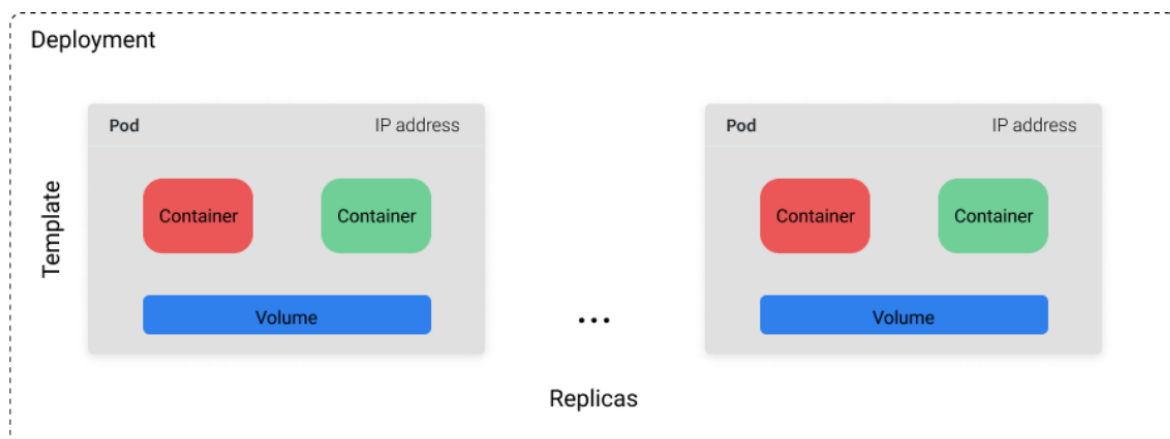


Figura 5.9: Schema di un Deployment.

- **Service:** offre un **punto di accesso stabile** per mandare il traffico ai Pod giusti, anche quando i Pod cambiano per aggiornamenti, scaling o guasti. Il Service sa a quali Pod inviare il traffico grazie alle **label** (coppie chiave-valore) definite nei metadati dei Pod.

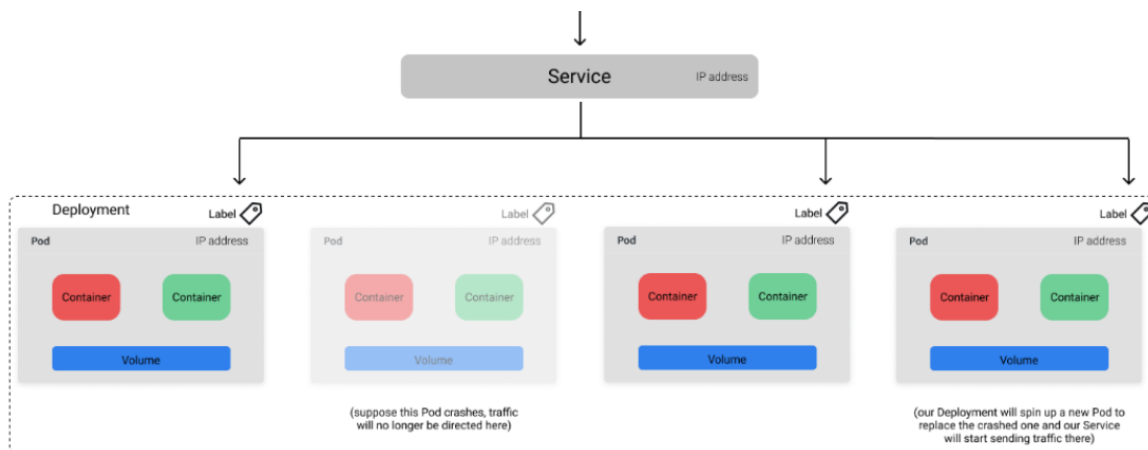


Figura 5.10: Schema di un Service.

- **Ingress:** i Service sono raggiungibili solo dall'interno del cluster. Per esporre l'applicazione al **traffico esterno** si definisce un oggetto Ingress, che sceglie quali Service rendere pubblici e li espone dietro un punto di accesso stabile (un URL esterno).

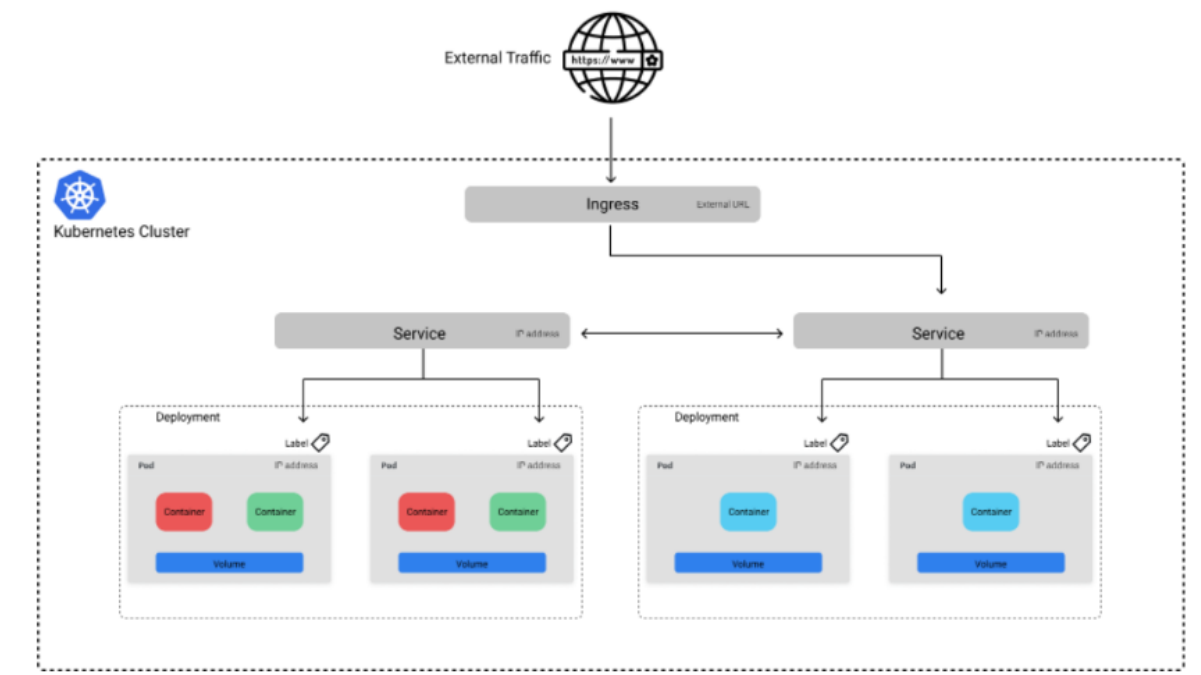


Figura 5.11: Schema di un Ingress.

5.4.4 Il control plane di Kubernetes

Vediamo ad alto livello come funziona Kubernetes “sotto il cofano”. Un cluster (insieme di macchine) ha due tipi di macchine:

- il **master node** (spesso uno solo), che contiene la maggior parte dei componenti del **control plane**;
- i **worker node**, che eseguono i carichi di lavoro delle applicazioni.

Master node

Gli utenti inviano le specifiche nuove o aggiornate degli oggetti (i manifest) all'**API server** del master node. L'API server valida le richieste (controllando l'input) e fa da **interfaccia unica** per qualsiasi domanda sullo stato attuale del cluster. Lo stato del cluster (configurazione, specifiche e stati degli oggetti, nodi del cluster, assegnazione degli oggetti ai nodi, ...) è salvato in un archivio chiave-valore distribuito chiamato **etcd**.

Il master node (Fig. 5.12) è formato da:

- l'**API server**;
- **etcd**, che contiene tutto lo stato del cluster;
- il **controller-manager**;
- lo **scheduler**.

L'API server fa da intermediario per rispettare il principio di **disaccoppiamento**: ogni componente ha un solo compito, e questo rende anche più facile ripartire dopo un guasto.

Il **controller-manager** controlla lo stato del cluster tramite l'API server. Se lo stato reale è diverso da quello desiderato, chiede modifiche all'API server per portare il cluster verso lo stato desiderato.

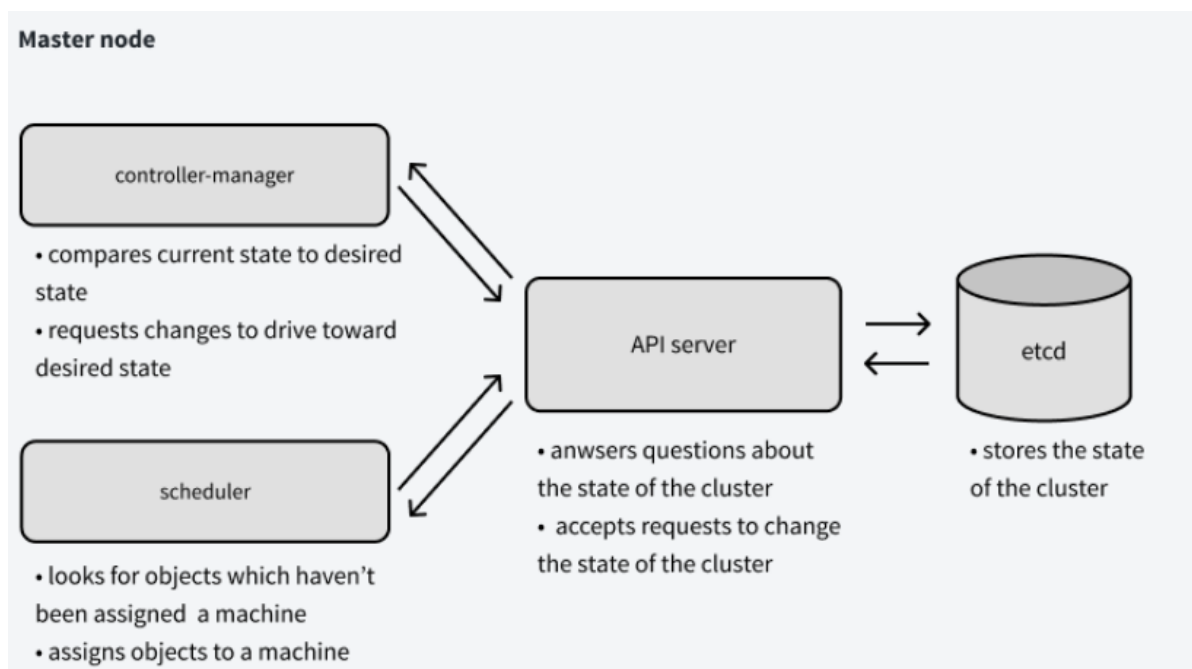


Figura 5.12: Struttura del master node di Kubernetes.

Lo **scheduler** decide dove eseguire gli oggetti (Fig. 5.13):

1. chiede all'API server quali oggetti non sono ancora assegnati a una macchina;
2. l'API server gli restituisce gli oggetti non assegnati (letti da etcd);
3. lo scheduler decide a quali macchine assegnarli e lo comunica all'API server;
4. l'API server registra l'assegnazione in etcd.

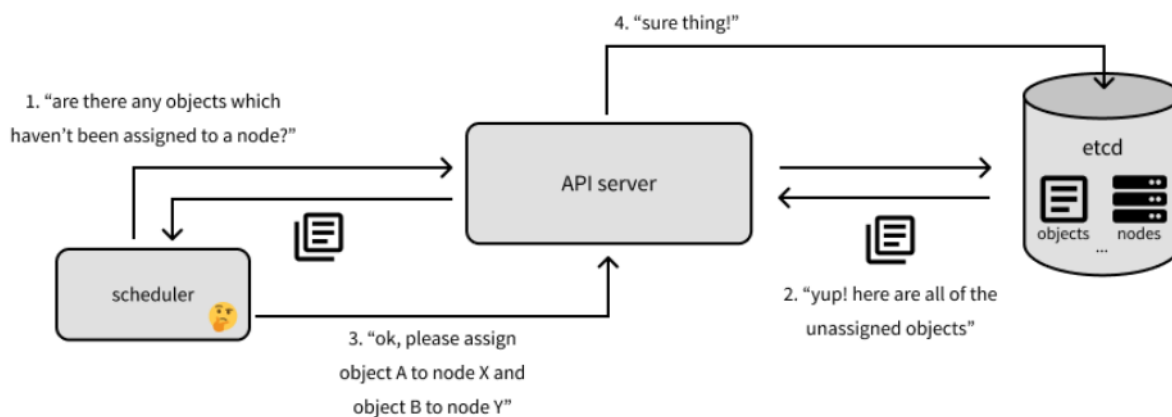


Figura 5.13: Il ciclo di vita dello scheduler.

Worker node

Ogni worker node contiene un **kubelet**, l'“agente” del nodo: comunica con l'API server per sapere quali container sono stati assegnati al nodo ed è responsabile di avviare i Pod che li eseguono. Quando un nodo entra nel cluster, il kubelet annuncia la sua esistenza all'API server, così lo scheduler può assegnargli dei Pod. Ogni worker contiene anche **kube-proxy**, che permette ai container di comunicare tra loro attraverso i vari nodi del cluster. La Fig. 5.14 mostra la struttura completa.

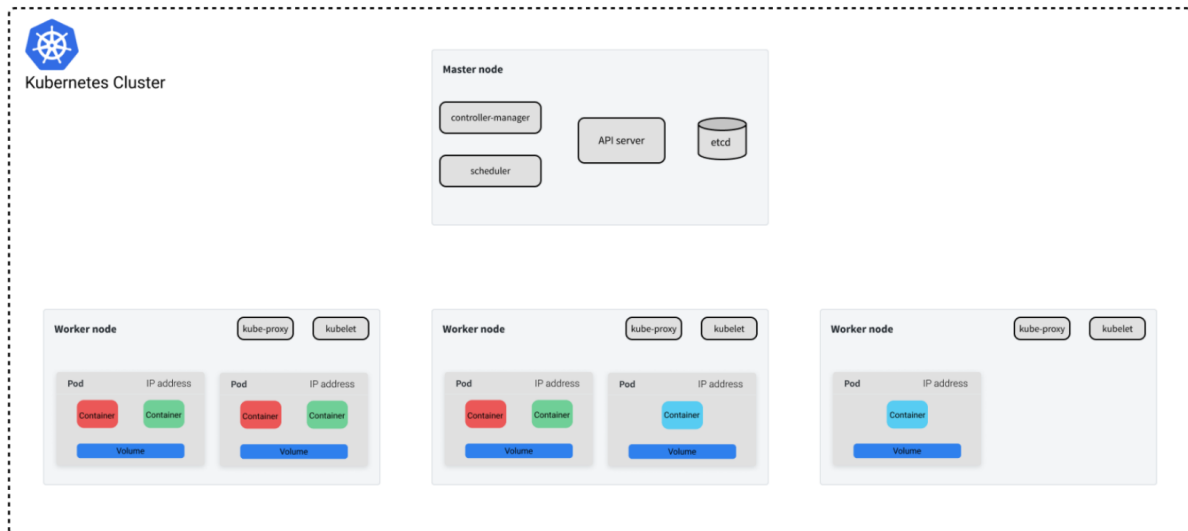


Figura 5.14: La struttura completa di Kubernetes.

Attenzione — Quando Kubernetes non conviene

- Se il carico di lavoro può girare su **una sola macchina**.
- Se serve **poca potenza di calcolo**.
- Se non serve **alta disponibilità** e si possono tollerare periodi di fermo.
- Se non si prevede di **cambiare spesso** i servizi installati.
- Se si ha un **monolite** e non si pensa di dividerlo in microservizi.

Esiste anche un modo per orchestrare container con Docker, chiamato **Docker Swarm**, ma Kubernetes è un orchestratore molto più potente. Le due tecnologie comunque collaborano, quindi non c'è lock-in tra Docker e Kubernetes.

Capitolo 6

Architettura a microservizi

Capitolo originale: 6 — Microservices Architecture • Antonio Brogi
Fonte: *Appunti di Emiliano Sescu (github.com/Faxatos), cap. 6*

Questo capitolo tratta le **architetture a microservizi**: idee di base, principi e vantaggi che le rendono così diffuse. Partiamo da una domanda: **come si divide un sistema in componenti?**

Dividere il sistema in componenti è il primo passo per adottare i microservizi. I motivi per farlo sono diversi:

- team diversi possono lavorare **in parallelo**;
- favorisce il **riuso**;
- rende più facile **distribuire** il sistema su più computer;
- permette di replicare, eseguire in parallelo e spostare i componenti, sfruttando al massimo le capacità del cloud (scalabilità, affidabilità, flessibilità).

Tra i principi dei microservizi è centrale l'uso di **servizi senza stato** (*stateless*), che salvano le informazioni persistenti in **database locali**. Questo rende il sistema agile e scalabile (i servizi si possono spostare tra server virtuali quando serve) e aiuta a costruire sistemi resilienti e tolleranti ai guasti.

Definizione — Servizio software

Un **servizio software** è un componente software accessibile via Internet. Si usa tramite la sua **interfaccia pubblica**, che nasconde tutti i dettagli di implementazione. Dato un input, produce l'output corrispondente **senza effetti collaterali**.

I servizi **non mantengono uno stato interno**: lo stato è salvato in un database esplicito oppure lo conserva chi chiama il servizio. Quando serve uno stato, la richiesta contiene lo stato e la risposta contiene il risultato insieme allo stato aggiornato.

Un po' di storia che ha portato ai microservizi:

- **Service-Oriented Architecture** (SOA, fine anni '90): applicazioni fatte di tanti blocchi, ognuno una classe o un oggetto. I blocchi erano indipendenti e potevano essere scritti con tecnologie diverse, perché comunicavano tra loro (o con Internet) tramite interfacce pubbliche.
- **Web Services** (primi anni 2000): servizi basati su standard (XML, SOAP, WSDL e molti altri), usati per descrivere i servizi (input, output, tipi dei parametri, ...). Gli standard erano utili, ma erano così tanti da diventare difficili da gestire.
- **Sistemi a servizi moderni** (oggi): protocolli di interazione e interfacce molto più semplici, con formati più efficienti per codificare i messaggi. Risultato: meno overhead ed esecuzione più veloce.

Amazon Web Services ha avuto un grande impatto su questo mondo, con un “manifesto” di linee guida su come implementare i servizi:

- un servizio deve riguardare **una sola funzione di business**;

- i servizi devono essere **completamente indipendenti**, ognuno con il **proprio database**;
- un servizio deve gestire la **propria interfaccia utente**;
- deve essere possibile sostituire o replicare un servizio **senza cambiare gli altri**.

Da qui nascono i **microservizi**: servizi in genere piccoli, senza stato e con **una sola responsabilità**.

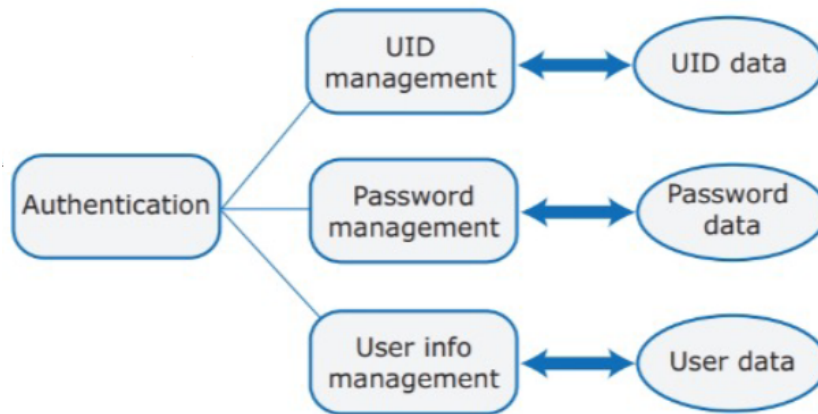


Figura 6.1: Esempio di sistema che usa un modulo di autenticazione.

Esempio — Dividere un modulo di autenticazione

Prendiamo un sistema con un modulo di autenticazione che offre: registrazione, autenticazione con UID/password, autenticazione a due fattori, gestione delle informazioni utente, reset della password. La Fig. 6.1 mostra una possibile divisione in microservizi. Per trovare i microservizi si spezzano le feature “grosse” in funzioni più dettagliate, si guardano i dati usati e si crea **un microservizio per ogni dato logico da gestire** (ogni servizio ha i propri dati). Se un servizio ha bisogno dei dati di un altro, li chiede tramite interfaccia oppure replica solo alcuni dati critici (accettando problemi di consistenza), cercando di replicare il meno possibile.

6.1 Microservizi

Definizione — Microservizio

Un **microservizio** è un servizio **indipendente** (la sua interfaccia non è toccata dai cambiamenti degli altri servizi) e di solito **piccolo**, che si combina con altri per creare applicazioni. Deve essere possibile modificarlo e ripubblicarlo **senza cambiare o fermare gli altri servizi**.

Caratteristiche che un microservizio deve avere:

- **Autonomo** (*self-contained*): poche dipendenze esterne. Gestisce i propri dati e la propria interfaccia utente, quindi si può installare e cambiare subito.
- **Leggero** (*lightweight*): comunica con protocolli leggeri, così l’overhead di comunicazione è basso.
- **Indipendente dall’implementazione**: ogni microservizio può essere scritto in un linguaggio diverso e usare tecnologie diverse.

- **Installabile da solo** (*independently deployable*): ogni microservizio gira nel suo processo e si installa in modo indipendente con sistemi automatici.
- **Orientato al business**: deve implementare capacità e bisogni di business, non solo offrire un servizio tecnico. Questo si sposa bene con i framework Agile.

Due misure importanti per i microservizi sono:

- l'**accoppiamento** (*coupling*): quante relazioni ci sono **tra** componenti diversi;
- la **coesione** (*cohesion*): quante relazioni ci sono **dentro** un componente.

L'obiettivo è **basso accoppiamento e alta coesione**: basso accoppiamento vuol dire servizi e aggiornamenti indipendenti, alta coesione vuol dire meno comunicazione (e quindi meno overhead) tra servizi.

Suggerimento — Quanto deve essere grande un microservizio?

Ogni servizio deve fare **una sola cosa, e farla bene** (*Single Responsibility Principle*). Per le dimensioni si usa la **regola del due**: un servizio deve poter essere sviluppato, testato e installato da un team in **due settimane**, e il team deve poter essere sfamato con **due pizze grandi** (8-10 persone).

Servono così tante persone perché il team deve: implementare le funzionalità del servizio; scrivere il codice che lo rende completamente indipendente; elaborare i messaggi in entrata e in uscita; gestire i guasti (ci saranno quasi sicuramente guasti dei servizi e delle interazioni); gestire la consistenza dei dati usati anche da altri servizi (problema serio quando i dati sono replicati); mantenere l'interfaccia del servizio; testare il servizio e le sue interazioni; supportare il servizio dopo il rilascio ("**you build it, you run it**", chi lo costruisce lo gestisce).

Anni fa c'erano team diversi per lo sviluppo e per la produzione; oggi si usa molto il **DevOps**, in cui le stesse persone costruiscono e mantengono il servizio.

6.2 Architettura

Le due motivazioni principali per usare un'architettura a microservizi sono:

- **ridurre i tempi** per rilasciare nuove feature e aggiornamenti;
- **scalare**.

Mettendo i microservizi in container separati, ognuno si può fermare e riavviare velocemente senza toccare gli altri, e le repliche si possono installare in fretta. Se questi due aspetti sono importanti per l'applicazione, i microservizi sono la scelta giusta; altrimenti (per un'applicazione semplice) un'**applicazione monolitica** va benissimo.

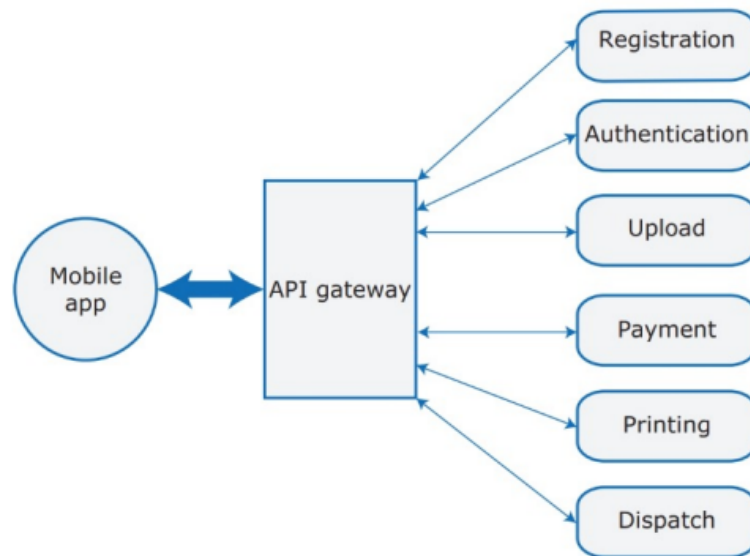


Figura 6.2: Sistema di stampa foto per dispositivi mobili.

La Fig. 6.2 mostra che i microservizi aiutano a individuare i servizi di base dell'applicazione: c'è un servizio separato per ogni area di funzionalità. Mostra anche una buona pratica: usare un **API gateway** come unico punto di contatto tra l'app mobile e i microservizi. È il gateway a inoltrare ogni richiesta al microservizio giusto.

Nelle architetture a microservizi bisogna prendere **cinque decisioni di design**:

1. Quali microservizi compongono il sistema?
2. Come comunicano tra loro?
3. Come si distribuiscono e condividono i dati?
4. Come si coordinano i microservizi?
5. Come si rilevano, segnalano e gestiscono i guasti?

Le vediamo una per una.

6.2.1 Decomposizione del sistema

La decomposizione è una delle scelte più importanti, ma anche delle più difficili. Con **troppi** microservizi, funzionalità collegate finiscono in servizi diversi che devono parlarsi di continuo: tanta comunicazione, quindi molto overhead. Con **troppo pochi**, ogni servizio fa troppe cose e i servizi diventano poco indipendenti negli aggiornamenti e nel deploy. Alcuni consigli:

- trovare il giusto equilibrio tra funzionalità molto fini e prestazioni del sistema;
- seguire il **common closure principle**: gli elementi che probabilmente cambieranno insieme devono stare nello stesso servizio;
- associare i servizi alle **capacità di business**;
- ogni servizio deve accedere **solo ai dati che gli servono** (con meccanismi per propagare i dati quando serve).

Suggerimento — Da dove partire

Ragionare direttamente a microservizi è difficile: per questo si parte spesso da un **monolite** e lo si divide in seguito. Un buon modo per capire come dividere è guardare i **dati** che i

servizi devono gestire.

6.2.2 Comunicazione tra servizi

La seconda scelta riguarda come comunicano i servizi.

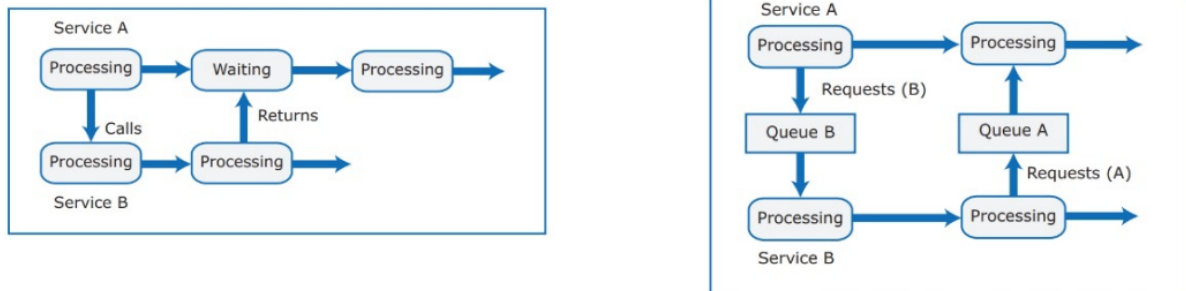


Figura 6.3: Interazione sincrona e asincrona tra servizi.

La prima scelta (Fig. 6.3) è tra comunicazione **sincrona** e **asincrona**:

- la **sincrona** (a sinistra) è più facile da scrivere e da capire;
- l'**asincrona** (a destra) ha un accoppiamento più basso ed è più efficiente, ma è più difficile da scrivere (serve una coda per le richieste) e da capire.

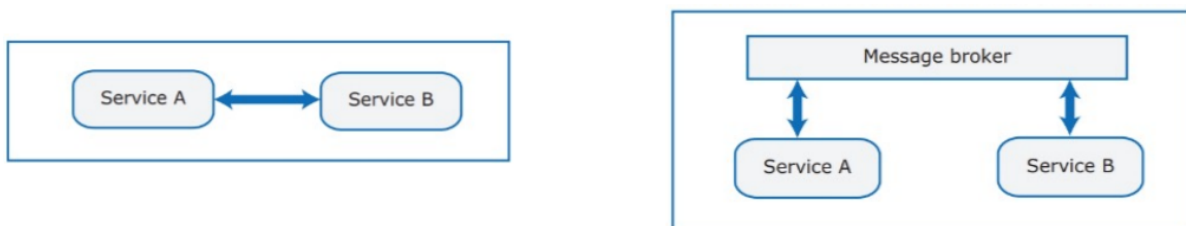


Figura 6.4: Comunicazione diretta e indiretta tra servizi.

La seconda scelta (Fig. 6.4) è tra comunicazione **diretta** e **indiretta**:

- nella **diretta** un servizio manda messaggi direttamente a un altro. È più semplice e veloce, ma chi chiama deve conoscere gli URI degli altri servizi;
- nella **indiretta** si usa un **message broker** (o una coda) che trova l'indirizzo del servizio richiesto e si occupa di tradurre i messaggi. Supporta sia interazioni sincrone sia asincrone e rende più facile modificare e sostituire i servizi, ma è più complessa e lenta.

6.2.3 Distribuzione e condivisione dei dati

Ogni microservizio deve **gestire i propri dati**, perché vogliamo che i microservizi siano indipendenti (gli altri non devono sapere come sono organizzati i suoi dati). Ci saranno comunque dipendenze tra i dati di servizi diversi. Per gestirle:

- condividere **il meno possibile**;
- condividere **in sola lettura**, con pochi servizi responsabili degli aggiornamenti;
- prevedere un meccanismo per tenere **consistenti** le copie del database usate dai servizi replicati.

Un meccanismo classico per la consistenza sono le **transazioni ACID** (*Atomicity, Consistency, Isolation, Durability*): gli aggiornamenti vengono serializzati, cioè il database passa da uno stato consistente a un altro, evitando inconsistenze. Nei sistemi distribuiti però bisogna scegliere tra consistenza e prestazioni, e con i microservizi il sistema va progettato per **tollerare un po' di inconsistenza**. Ci sono due tipi di inconsistenza da gestire:

- **Inconsistenza di dati dipendenti**: le azioni o i guasti di un servizio possono rendere inconsistenti i dati gestiti da un altro servizio.
- **Inconsistenza tra repliche**: più repliche dello stesso servizio girano insieme, ognuna con la propria copia del database che aggiorna per conto suo. Le copie devono diventare **prima o poi** consistenti (*eventual consistency*).

Il teorema CAP

Teorema — Teorema CAP (congettura di Brewer, PODC 2000)

È impossibile per un servizio web garantire **contemporaneamente**:

- **Consistency** (consistenza): ogni servizio restituisce la risposta corretta a ogni richiesta;
- **Availability** (disponibilità): ogni richiesta prima o poi riceve una risposta;
- **Partition tolerance** (tolleranza alle partizioni): i servizi possono essere divisi in più gruppi, e la rete può ritardare o perdere quanti messaggi vuole tra i servizi.

Una riformulazione successiva (Gilbert e Lynch, 2012) dice: *in una rete soggetta a guasti di comunicazione, è impossibile per qualsiasi servizio web implementare una memoria condivisa atomica in lettura/scrittura che garantisca una risposta a ogni richiesta*. In pratica, visto che le partizioni di rete capitano, bisogna **scegliere tra consistenza e disponibilità**.

Suggerimento — Idea della dimostrazione (versione semplificata)

Siano S1 e S2 due repliche dello stesso servizio in due partizioni di rete diverse, e supponiamo che ogni messaggio tra S1 e S2 possa essere ritardato o perso. Un client scrive il valore V1 su S1. Poi un altro client legge da S2. S2 non può sapere se S1 ha ricevuto una scrittura (il messaggio potrebbe essersi perso). Allora ha due scelte:

- **risponde subito** con il valore che ha: la risposta può essere vecchia, quindi si perde la **consistenza**;
- **aspetta** di sentire S1 per essere sicuro: ma il messaggio potrebbe non arrivare mai, quindi si perde la **disponibilità**.

Soluzioni pratiche:

- garantire la **disponibilità** e offrire consistenza “al meglio possibile” (*best-effort*): è la soluzione più comune;
- garantire la **consistenza forte** e offrire disponibilità *best-effort*: si usa quando la consistenza è indispensabile (es. applicazioni finanziarie);
- trovare un **compromesso** (es. tollerare dati vecchi di un'ora, ma non di un giorno).

Il pattern Saga

Come si implementano le transazioni distribuite con un pattern?

Definizione — Saga

Il **pattern Saga** implementa ogni transazione di business che coinvolge più servizi come una **saga**, cioè una **sequenza di transazioni locali** (Fig. 6.5). Ogni transazione locale aggiorna un database e fa partire la transazione locale successiva. Se una transazione locale fallisce, la saga esegue una serie di **transazioni di compensazione** (invece di fare rollback, che con i microservizi a volte non è possibile).

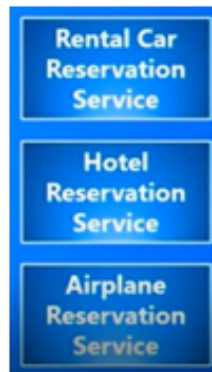


Figura 6.5: Esempio di saga.

Le saghe si possono coordinare in due modi:

- **Coreografia:** ogni transazione locale pubblica degli eventi che fanno partire le transazioni successive.
- **Orchestrazione:** un **orchestratore** dice ai partecipanti quali transazioni locali eseguire.

Anche le transazioni di compensazione si possono fare in due modi:

- **modello backward** (all'indietro): si annullano le modifiche fatte dalle transazioni locali già eseguite;
- **modello forward** (in avanti): si applica il principio “**riprova più tardi**”.

Esempio — L'approccio di Netflix

Netflix replica i dati su n nodi con **Apache Cassandra** per ottenere la consistenza finale: il sistema prova ad aggiornare più repliche possibile, e richiede che rispondano almeno $\lfloor n/2 \rfloor + 1$ repliche (la maggioranza). È un esempio di compromesso ottenibile con l'eventual consistency.

6.2.4 Coordinamento dei servizi

Il coordinamento dei servizi si può fare in due modi:

- **Orchestrazione:** un orchestratore dice ai microservizi come devono “suonare”.
- **Coreografia:** i microservizi si gestiscono da soli.

Suggerimento — Un'immagine per ricordarlo

L'orchestrazione è come un **semaforo** (qualcuno decide per tutti), la coreografia è come una **rotonda** (ognuno si regola da sé seguendo le regole).

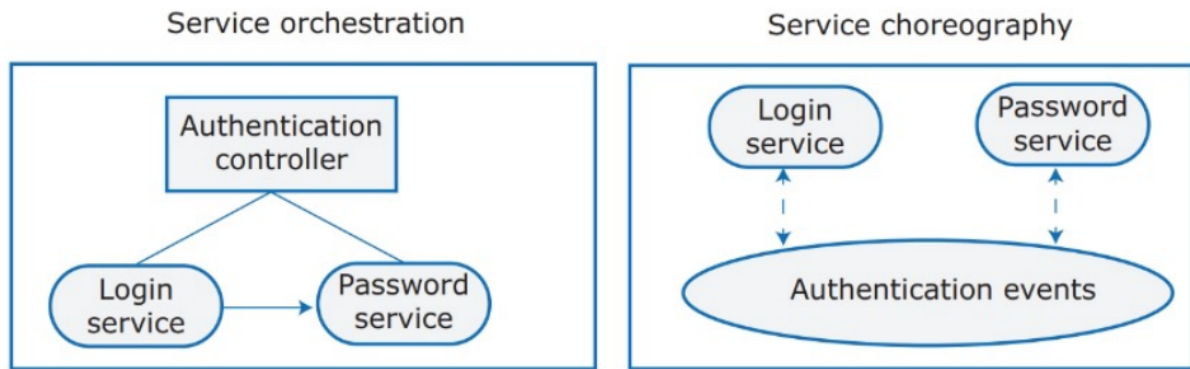


Figura 6.6: Esempio di orchestrazione e coreografia.

Nella Fig. 6.6, con l'**orchestrazione** c'è un controller esplicito che coordina il sistema (l'*Authentication controller*); con la **coreografia** si usa una sincronizzazione basata su **eventi** (un meccanismo publish & subscribe chiamato *Authentication events*). La coreografia evita un controller in più (che sarebbe un **single point of failure**), ma è più difficile da debuggare e da ripristinare in caso di guasto.

Suggerimento — Buona pratica

Partire sempre con l'**orchestrazione** (più facile da progettare) e passare alla coreografia solo se il prodotto diventa rigido o difficile da aggiornare.

6.2.5 Gestione dei guasti

Prima o poi qualcosa andrà storto, inevitabilmente.

Esempio — Quanto costa dipendere da altri servizi

Un servizio offre una funzionalità F chiamando altri due servizi, ognuno disponibile il 99% del tempo e indipendenti tra loro. F è disponibile solo quando lo sono entrambi: $0,99 \times 0,99 = 0,9801$, cioè circa il 98%. Il restante 2% di un giorno (1440 minuti) sono circa **30 minuti al giorno** di non disponibilità.

Soprattutto negli ambienti distribuiti, bisogna gestire i guasti: i servizi devono essere **progettati per sopravvivere ai guasti**. Ci sono tre tipi principali di guasto:

- **Guasto interno** (*internal service failure*): il servizio stesso rileva il problema e può segnalarlo a chi lo ha chiamato con un messaggio di errore. Esempio: un servizio riceve un URL in input e scopre che il link non è valido.
- **Guasto esterno** (*external service failure*): una causa esterna compromette la disponibilità del servizio, che può smettere di rispondere; bisogna intervenire per riavviarlo.
- **Guasto di prestazioni** (*service performance failure*): le prestazioni del servizio scendono sotto un livello accettabile, per carico eccessivo o per un problema interno. Un monitoraggio esterno può rilevare sia i guasti di prestazioni sia i servizi che non rispondono.

Circuit breaker

Una delle soluzioni più tipiche per i microservizi è il **circuit breaker** (interruttore automatico): un design pattern che rende i microservizi resilienti limitando l'impatto di guasti e ritardi dei servizi.

Funziona così (Fig. 6.7): tra il client e il servizio remoto (il *supplier*) si aggiunge un componente, il circuit breaker.

1. Il circuit breaker riceve la richiesta dal client, la inoltra al servizio remoto e fa partire un timer.
2. Se il servizio remoto risponde in tempo, il circuit breaker inoltra la risposta al client.
3. Se scade il timer (il servizio è guasto o troppo lento), il circuit breaker dice al client che il servizio remoto ha fallito, così il client non resta bloccato ad aspettare.
4. Dopo più fallimenti il circuit breaker **scatta** (*trip*): per un certo periodo tutte le chiamate al servizio remoto falliscono **subito**, senza nemmeno provarci (circuit aperto).
5. Finito il periodo, lascia passare un numero limitato di richieste di prova. Se vanno a buon fine torna al funzionamento normale, altrimenti il periodo di attesa ricomincia.

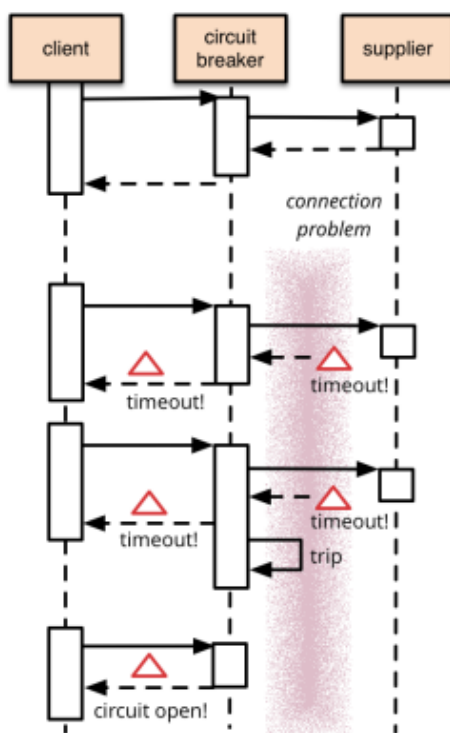


Figura 6.7: Esempio di diagramma di sequenza di un circuit breaker.

Esempio — Il circuit breaker di Spotify

Il servizio di ricerca di Spotify a volte fallisce (deve cercare in un indice enorme di canzoni). Allora Spotify ricarica velocemente la pagina e usa il tempo che l'utente impiega a scrivere il nome della canzone come periodo di attesa. Nessun messaggio di errore compare all'utente.

Ci sono molti modi per testare la gestione dei guasti. Uno interessante è **Chaos Monkey** di Netflix (dal *chaos engineering*): uno strumento che spegne a caso istanze di VM e container **in produzione**. Visto che si fa direttamente in produzione, è un test "coraggioso".

6.3 Servizi RESTful

I microservizi sono **servizi RESTful**. **REST** (*REpresentational State Transfer*) ha quattro principi:

- **Identificazione delle risorse tramite URI:** il servizio espone un insieme di risorse, ognuna identificata da un **URI** (*Uniform Resource Identifier*, una sequenza di caratteri che identifica una risorsa logica o fisica).
- **Interfaccia uniforme:** i client usano i metodi HTTP per creare, leggere, aggiornare e cancellare le risorse: **POST** e **PUT** per creare e aggiornare lo stato di una risorsa, **DELETE** per cancellarla, **GET** per leggerne lo stato attuale.
- **Messaggi auto-descrittivi:** le richieste contengono abbastanza informazioni di contesto per essere elaborate, anche perché la stessa risorsa può essere rappresentata in vari formati (HTML, XML, JSON, testo, PDF, JPEG, ...). Per questo le risorse sono **separate dalla loro rappresentazione**.
- **Interazioni senza stato tramite hyperlink:** ogni interazione con una risorsa è **stateless**, cioè il server non conserva lo stato del client e l'eventuale stato di sessione è tenuto dal client. Le interazioni senza stato si basano sul **trasferimento esplicito dello stato**.

Nota — Nota

Gli altri dettagli su REST sono dati per noti dalla laurea triennale.

6.4 Deploy dei servizi

Per circa 70 anni, in ogni progetto software un gruppo di sviluppatori gestiva tutto il ciclo di vita secondo l'approccio project-based (requisiti, specifiche, prototipo, test, integrazione, qualità dell'esperienza, ...). Finito il software, il **team operativo** (*operations*), che non aveva partecipato allo sviluppo, lo metteva in produzione (magari in un ambiente diverso da quello di sviluppo) e poi rispondeva ai problemi dei clienti.

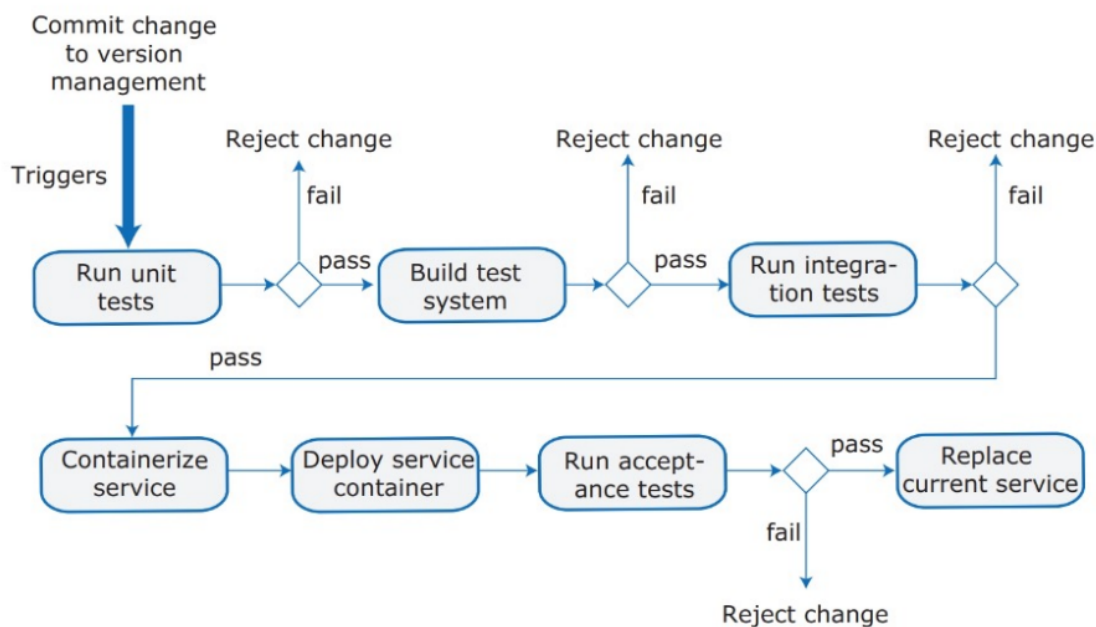


Figura 6.8: Pipeline di Continuous Deployment.

La nuova era dell'ingegneria del software è il **DevOps** (da *development* + *operations*): **lo stesso team** si occupa di sviluppo, deploy e gestione del servizio. La Fig. 6.8 mostra la pipeline che parte quando c'è un commit: test di unità, build, test di integrazione, creazione del con-

tainer, deploy, test di accettazione e, se tutto va bene, sostituzione del servizio attuale con la nuova versione.

6.4.1 Monitoraggio

I test non possono prevenire il 100% dei problemi imprevisti, quindi bisogna **monitorare** i servizi in produzione. Senza monitoraggio non si conosce lo stato dei servizi, ma un monitoraggio eccessivo peggiora le prestazioni: come spesso accade con i microservizi, serve un compromesso.

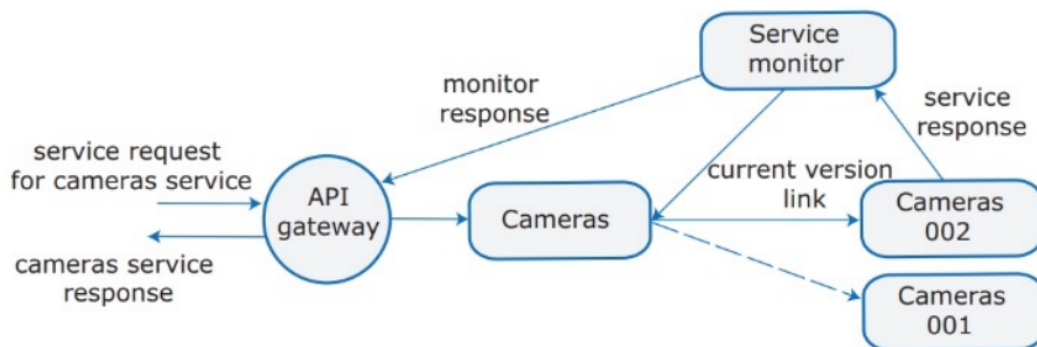


Figura 6.9: Esempio di monitoraggio.

Il monitoraggio aiuta anche l'affidabilità (Fig. 6.9): quando si introduce una nuova versione di un servizio si può tenere attiva quella vecchia e spostare il “**current version link**” sulla nuova. Se il monitoraggio rileva un problema nella nuova versione “Cameras 002”, il link torna alla versione 001 del servizio.

6.5 Architectural smell

Definizione — Architectural smell

Un **architectural smell** è una decisione architetturale di uso comune che però **peggiora le qualità** del sistema durante il suo ciclo di vita.

Definiti i principi dei microservizi, come si rilevano gli smell che li violano e come si risolvono tramite **refactoring**? Questa sezione presenta, sulla base di una *multivocal review* (una rassegna che usa sia articoli scientifici sia siti tecnici), gli smell più riconosciuti per i microservizi e i refactoring per eliminarli.

I principi di design considerati sono:

- **Deploy indipendente** (*independent deployability*): ogni microservizio deve essere installabile in modo indipendente.
- **Scalabilità orizzontale** (*horizontal scalability*): ogni microservizio deve poter essere scalato in orizzontale, cioè si devono poter aggiungere o togliere sue repliche.
- **Isolamento dei guasti** (*isolation of failures*): i guasti devono restare isolati, evitando l'effetto a cascata.
- **Decentralizzazione** (*decentralization*): va applicata in tutti gli aspetti, dalla gestione dei dati alla governance.

6.5.1 Smell e refactoring

Gli smell sono raggruppati per principio violato; per ognuno vediamo i refactoring che lo risolvono.

Deploy indipendente

- **Smell:** **più servizi in un container** (*multiple services in one container*). Idealmente si vuole un container per microservizio. Si può mettere nello stesso container qualche microservizio che interagisce molto, ma può creare vari problemi.
- **Refactoring:** mettere **ogni servizio in un container separato**.

Scalabilità orizzontale

- **Smell:** **interazione basata su endpoint** (*endpoint-based service interaction*). Se si interagisce con **una specifica istanza** di un servizio (Fig. 6.10), aggiungere repliche non fa scalare l'applicazione, perché si parla sempre con la stessa replica.
- **Refactoring:**
 - aggiungere un **service discovery** che dice a chi chiama quale replica usare (usato nel 55% dei casi);
 - aggiungere un **message router** che inoltra la richiesta a una delle repliche, es. un load balancer (31%);
 - aggiungere un **message broker** che raccoglie le richieste, poi elaborate dalle repliche, es. una coda di messaggi (14%). È il meno usato perché richiede di modificare il codice del microservizio.

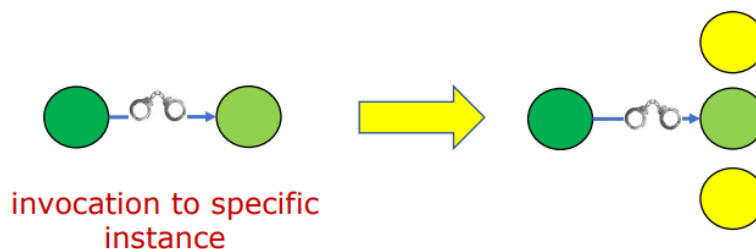


Figura 6.10: Interazione basata su endpoint.

- **Smell:** **assenza di un API gateway**: i client chiamano direttamente i servizi (simile allo smell precedente).
- **Refactoring:** aggiungere un **API gateway**, che oltre a risolvere lo smell è utile anche per autenticazione, limitazione del traffico (*throttling*), ecc.

Isolamento dei guasti

- **Smell:** **interazione instabile** (*wobbly service interaction*): l'interazione di m1 con m2 è "instabile" quando un guasto di m2 può causare un guasto di m1.
- **Refactoring:** aggiungere un **circuit breaker** (42% dei casi), usare dei **timeout** (22%), aggiungere un **bulkhead** (20%) o un **message broker** (16%).

Suggerimento — Cos'è un bulkhead

Il nome viene dalle paratie stagne delle navi: si dividono le risorse (thread, connessioni, ...) in compartimenti separati, così se una parte si blocca non trascina con sé tutto il resto.

Decentralizzazione

- **Smell: persistenza condivisa** (*shared persistence*): più servizi usano lo stesso database.
- **Refactoring** (Fig. 6.11):
 - **dividere il database** (50% dei casi);
 - aggiungere un **data manager** che fa da gateway tra i servizi e il database (22%);
 - **unire i servizi** (9%).

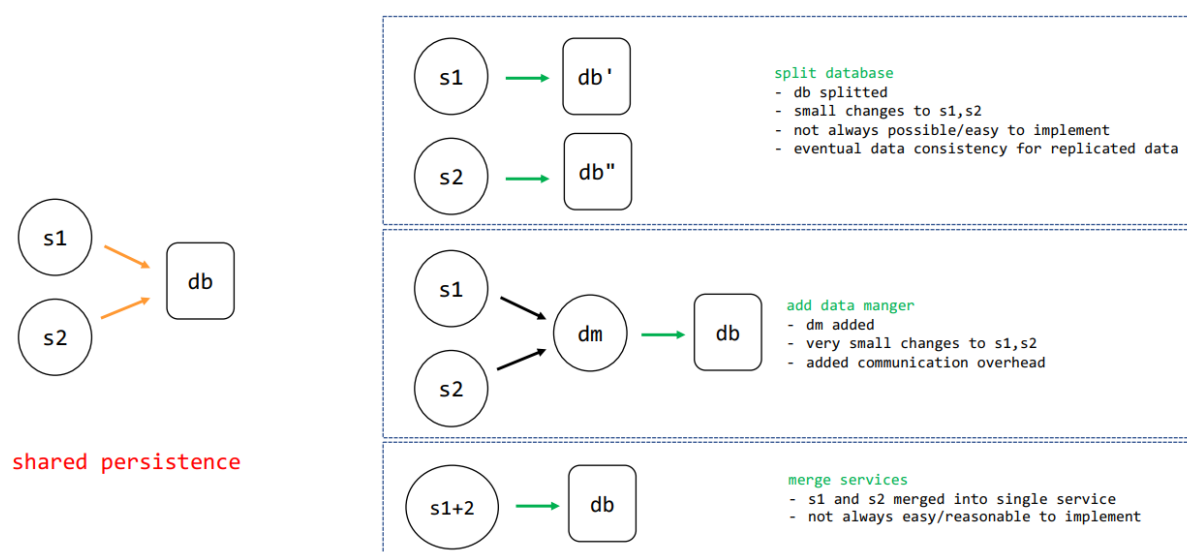


Figura 6.11: Soluzioni per la persistenza condivisa.

- **Smell: team a livello unico** (*single-layer teams*): team divisi per livello tecnico (es. team del database, team dell'interfaccia) invece che per servizio. Va contro l'idea di Agile.
- **Refactoring: dividere i team per servizio.**

6.5.2 Una toolchain per i microservizi

Ora che conosciamo gli smell, vogliamo trovarli e risolverli. Il primo passo è creare una **rappresentazione grafica** dell'architettura (Fig. 6.12), per capire meglio la struttura dell'applicazione.

Questi modelli permettono anche **viste per team**, visto che ogni servizio è sviluppato da un team diverso. È utile soprattutto quando ci sono smell tra microservizi gestiti da team diversi: i team possono accorgersene e mettersi d'accordo per risolverli.

Graphical representation

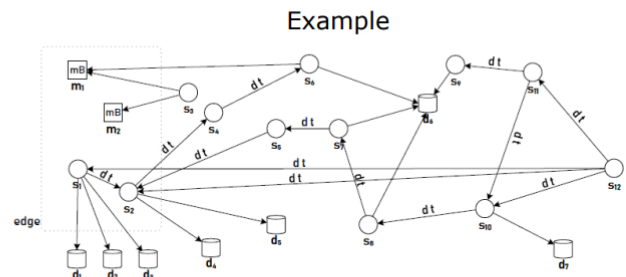
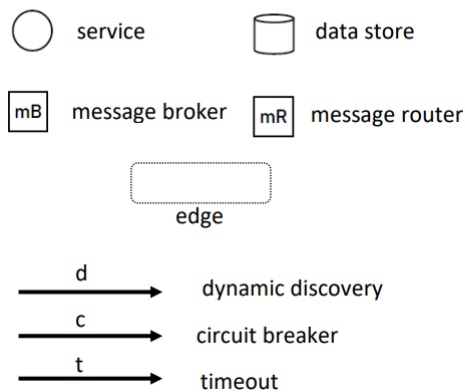


Figura 6.12: Modellare l'architettura di un'applicazione.

Esistono strumenti che ricevono questi modelli e restituiscono gli smell presenti. Uno è stato sviluppato all'Università di Pisa: **MicroFreshener** ([GitHub](#)). Per sistemi grandi può essere difficile ottenere a mano il modello dell'architettura, quindi sono stati creati strumenti (**MicroMiner** e **MicroTOM**) che lo ricavano analizzando i manifest K8s e facendo analisi dinamica dell'applicazione in esecuzione (l'orchestrazione dei container, infatti, cambia il comportamento dell'applicazione). La Fig. 6.13 mostra la toolchain completa.

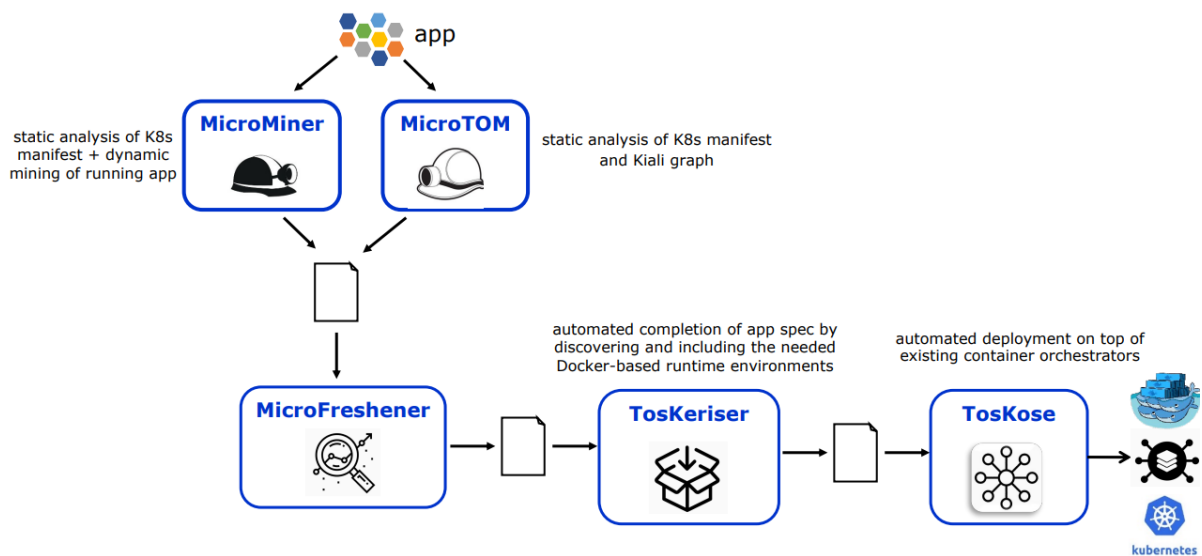


Figura 6.13: Toolchain per i microservizi.

Dopo aver analizzato gli smell e applicato le correzioni in **MicroFreshener**, il risultato si può **installare automaticamente** sull'orchestratore di container già esistente (nella toolchain, con **TosKeriser** e **TosKose**).

Capitolo 7

Sicurezza e privacy

Capitolo originale: 7 — Security and Privacy • Antonio Brogi
Fonte: Appunti di Emiliano Sescu (github.com/Faxatos), cap. 7

La sicurezza del software è una priorità sia per chi sviluppa il prodotto sia per chi lo usa, perché un attacco può causare danni a entrambi. La Fig. 7.1 mostra i tipi principali di minaccia; alcuni attacchi li combinano (ad esempio un **ransomware** colpisce l'integrità dei dati, ma di conseguenza anche la disponibilità del sistema).

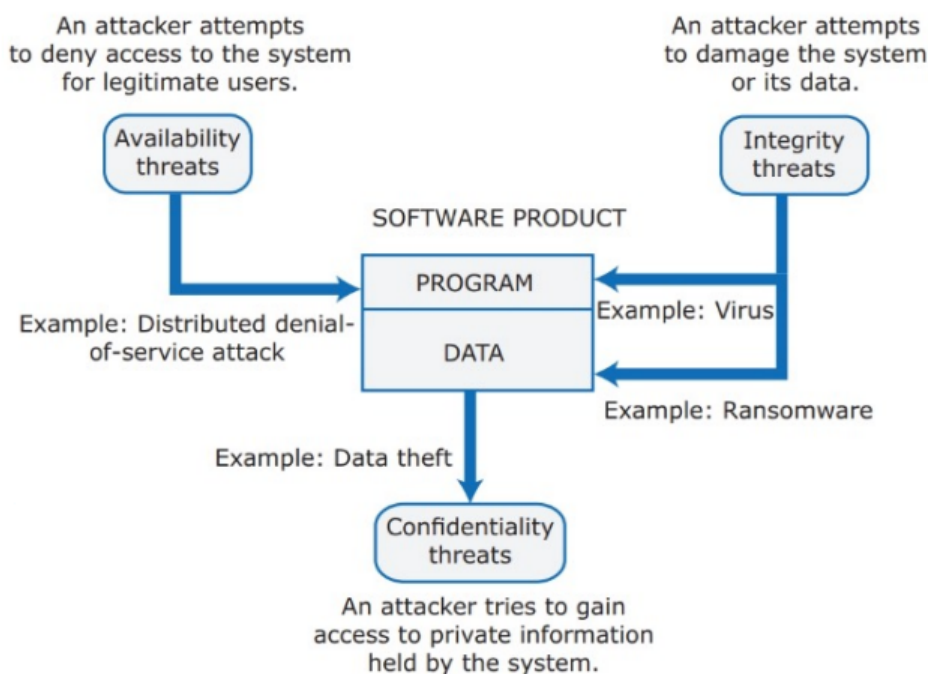


Figura 7.1: I tipi principali di minacce alla sicurezza.

Suggerimento — Le tre minacce in breve

- **Disponibilità** (*availability*): impedire agli utenti legittimi di usare il sistema.
- **Integrità** (*integrity*): danneggiare il sistema o i suoi dati.
- **Riservatezza** (*confidentiality*): accedere a informazioni private.

La sicurezza riguarda **tutto il sistema**: il software applicativo dipende dal sistema operativo, dal web server, dal runtime del linguaggio, dal database, dai framework, dagli strumenti, ecc. Un attacco può colpire qualsiasi livello dell'infrastruttura, a partire dalla rete.

Alcune attività di gestione del sistema per mantenerlo sicuro:

- **Standard e procedure di autenticazione e autorizzazione**, perché tutti gli utenti abbiano un'autenticazione forte e permessi di accesso impostati bene.

- **Gestione dell'infrastruttura:** tenere configurato bene il software di base e applicare subito gli aggiornamenti di sicurezza che correggono le vulnerabilità.
- **Monitoraggio regolare degli attacchi,** per accorgersene presto e attivare strategie di difesa che ne limitino gli effetti.
- **Politiche di backup,** per avere copie integre di programmi e dati da ripristinare dopo un attacco.

7.1 Attacchi e difese

7.1.1 Injection

Un attacco di **injection** avviene quando un utente malintenzionato usa un campo di input valido per inserire **codice malevolo** o **comandi per il database** che danneggiano il sistema.

- **Buffer overflow:** una delle classi di attacco più comuni. L'attaccante costruisce con cura una stringa di input che contiene istruzioni eseguibili e sovrascrive la memoria. Se viene sovrascritto l'indirizzo di ritorno di una funzione, il controllo passa al codice malevolo. Succede ad esempio con sistemi operativi e librerie scritti in C/C++, che non controllano se gli accessi agli array restano nei limiti.
- **SQL injection:** possibile quando l'input dell'utente diventa parte di un comando SQL.

La contromisura più semplice è **controllare la validità dell'input**.

Esempio — SQL injection

Se il codice costruisce la query come "SELECT * FROM users WHERE name = '" + input + "'" e l'utente scrive ' OR '1'='1, la condizione diventa sempre vera e la query restituisce tutti gli utenti. Oltre a validare l'input, la difesa standard è usare **query parametrizzate** (*prepared statement*).

7.1.2 Session hijacking

Molte applicazioni usano il concetto di **sessione**: un periodo in cui l'autenticazione dell'utente con l'applicazione web resta valida. Di solito si realizza con dei **cookie di sessione** (token) che il server dà al client e che il client rimanda a ogni richiesta HTTP, così l'utente non deve autenticarsi di nuovo a ogni interazione. La sessione si chiude quando l'utente fa logout o quando scade (*timeout*).

Nel **session hijacking** (furto di sessione) l'attaccante si procura un cookie di sessione valido per **fingersi un utente legittimo**. Può ottenerlo con un attacco di cross-site scripting o osservando il traffico (facile su Wi-Fi non protette e con dati non cifrati). L'attacco può essere:

- **attivo:** l'attaccante compie azioni sul server al posto dell'utente;
- **passivo:** l'attaccante si limita a osservare il traffico client-server in cerca di informazioni utili (password, numeri di carta di credito, ...).

Difese: **cifrare il traffico** client-server (es. HTTPS), usare l'**autenticazione a più fattori** per confermare azioni nuove e potenzialmente dannose, usare **timeout di sessione** abbastanza brevi.

7.1.3 Cross-site scripting (XSS)

Il **cross-site scripting** è un'altra forma di injection: l'attaccante aggiunge **codice JavaScript malevolo** alla pagina web che il server manda al client. Lo script viene eseguito quando la pagina è mostrata nel browser della vittima, con l'obiettivo di rubare informazioni del cliente o dirottarlo su un altro sito. Può anche rubare i cookie, rendendo possibile il session hijacking.

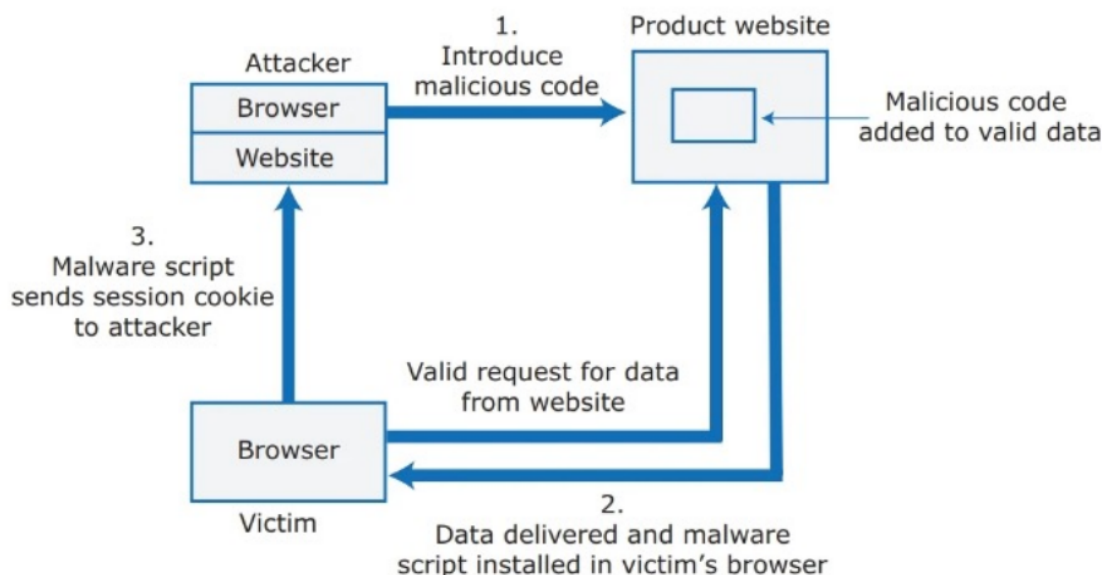


Figura 7.2: Un attacco di cross-site scripting.

Difese: **validare l'input** (dei form), **controllare i dati presi dal database** prima di inserirli nella pagina generata, usare il comando HTML di **"encode"** (così le informazioni aggiunte alla pagina non sono eseguibili).

7.1.4 Denial of Service (DoS)

Gli attacchi **Denial of Service** (DoS) vogliono rendere il sistema **inutilizzabile** per l'uso normale. Di solito si usano molti computer distribuiti (spesso infettati e controllati dall'attaccante) che mandano centinaia di migliaia di richieste a un'applicazione web (**DDoS**, *Distributed DoS*). Lo scopo è sabotare il fornitore del servizio o chiedere un riscatto.

Contromisure:

- software specializzati che **rilevano e scartano** i pacchetti in arrivo, evitando di essere sommersi dalle richieste;
- **blocchi temporanei** degli utenti (es. bloccare un utente dopo ripetuti tentativi di login falliti con la sua email);
- **tracciamento degli indirizzi IP** (es. applicare il blocco solo se i tentativi falliti arrivano da indirizzi IP insoliti).

7.1.5 Brute force

L'attaccante ha solo una parte delle informazioni (es. un nome utente valido ma non la password) e prova ripetutamente a indovinare la parte mancante, ad esempio con un generatore di stringhe che crea tutte le combinazioni possibili di lettere e numeri.

Difese: convincere (cioè obbligare) gli utenti a usare **password lunghe**, che non siano nel dizionario né parole comuni. Un ulteriore livello di sicurezza è l'**autenticazione a due fattori**.

7.2 Autenticazione e autorizzazione

7.2.1 Autenticazione

Definizione — Autenticazione

L'**autenticazione** serve a garantire che gli utenti del sistema siano **davvero chi dicono di essere**.

Ci sono tre approcci principali:

- **Basata sulla conoscenza** (*knowledge-based*): l'utente fornisce informazioni segrete e personali scelte alla registrazione (es. una password). Ha diversi punti deboli: password poco sicure; **phishing** (l'utente clicca un link in una email che porta a un sito falso che raccoglie login e password); stessa password usata ovunque; password dimenticate spesso (serve un meccanismo di recupero, che è una possibile vulnerabilità se le credenziali sono state rubate). Si può rendere più sicura obbligando a usare password forti e aggiungendo domande personali per il recupero.
- **Basata sul possesso** (*possession-based*): l'utente ha un **dispositivo fisico** collegato al sistema che genera o mostra informazioni note al sistema (es. un codice inviato al numero di telefono, o un dispositivo dedicato che genera codici usa e getta).
- **Basata sugli attributi** (*attribute-based*): usa una caratteristica **biometrica** unica dell'utente (impronta digitale, volto).

Se il sistema conserva informazioni riservate, la buona pratica è l'**autenticazione a più fattori** (*multi-factor*, es. password e poi un codice ricevuto sul cellulare), che aggiunge un livello di sicurezza.

Di solito gli sviluppatori non scrivono l'autenticazione da zero ma usano librerie e toolkit già pronti (es. [OAuth](#)). Anche così, costruire un sistema di autenticazione sicuro e affidabile richiede molto lavoro. Per questo spesso l'autenticazione si **delega** a un **sistema di identità federata**, cioè a un servizio esterno (es. Google, Facebook), come in Fig. 7.3. Questi servizi sono collaudati e probabilmente molto più sicuri di qualsiasi sistema fatto in casa. In più il fornitore del prodotto non deve gestire un proprio database di password e può ottenere altre informazioni sull'utente (se l'utente acconsente).

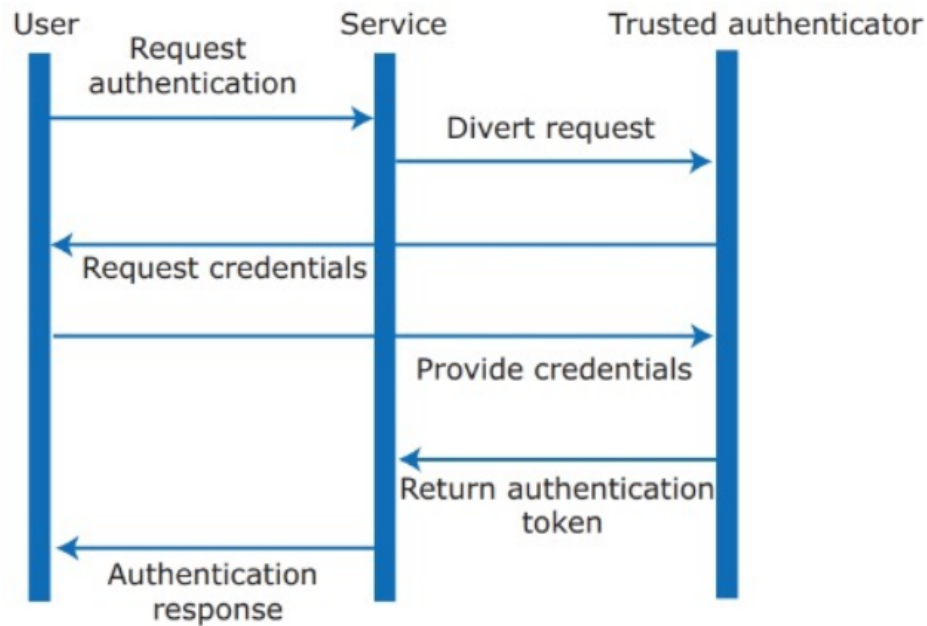


Figura 7.3: Diagramma di sequenza dell'identità federata.

Sui **dispositivi mobili** le regole cambiano, perché scrivere password sulla tastiera del telefono è scomodo. Un'alternativa è installare un **token di autenticazione** sul dispositivo, ma se il telefono viene rubato o perso qualcun altro può accedere al prodotto. Un'alternativa più sicura sono i **certificati digitali** personali degli utenti, emessi da enti fidati.

7.2.2 Autorizzazione

Definizione — Autorizzazione

Mentre l'autenticazione verifica **chi** è l'utente, l'**autorizzazione** controlla **a quali risorse** l'utente può accedere.

Nei prodotti multiutente serve un **controllo degli accessi**, con una politica che rispetti le regole di protezione dei dati e limiti l'accesso ai dati personali (anche per evitare cause legali in caso di furto di dati).

Un modo diffuso per implementare la politica di accesso sono le **Access Control List (ACL)**:

- gli utenti sono divisi in **gruppi**, e questo riduce molto la dimensione delle ACL;
- gruppi diversi possono avere diritti diversi su risorse diverse;
- le **gerarchie di gruppi** permettono di assegnare diritti a sottogruppi o a singoli utenti.

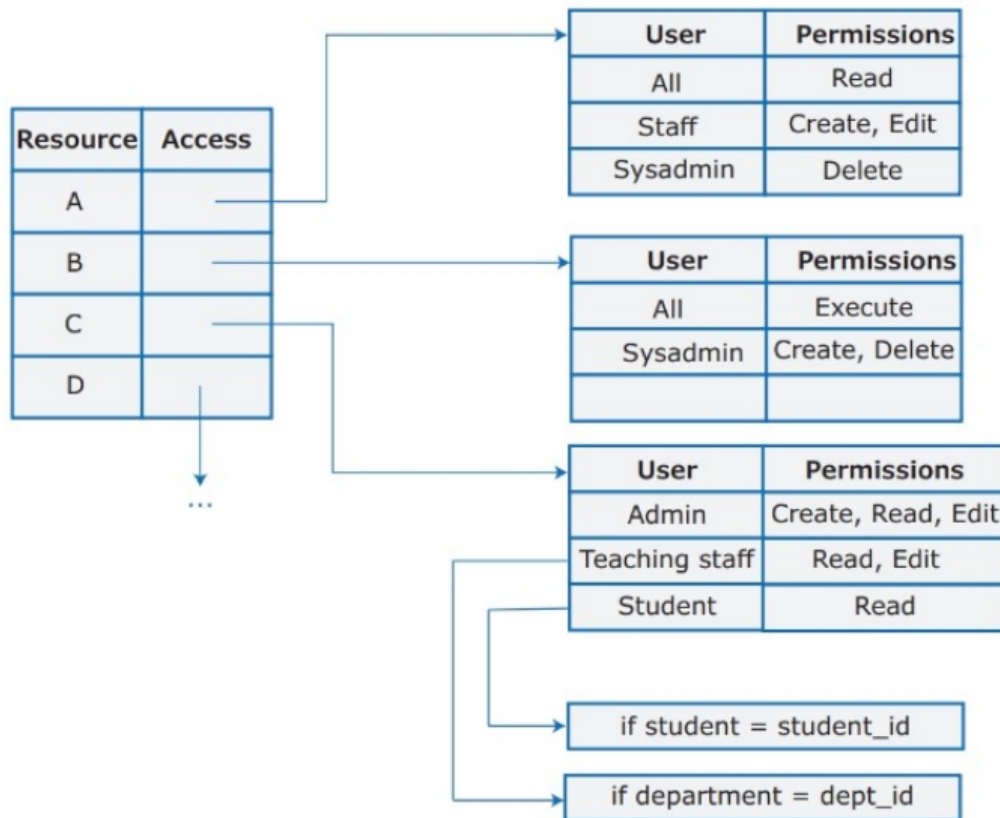


Figura 7.4: Esempio di Access Control List.

7.3 Crittografia

La **crittografia** (*encryption*) rende un testo illeggibile applicandogli una trasformazione algoritmica. Il testo viene cifrato con una chiave segreta, viaggia su canali non sicuri e, quando arriva a destinazione, viene **decifrato** con la stessa chiave o con un'altra.

Le tecniche moderne sono considerate “praticamente inattaccabili” con la tecnologia di oggi, anche se la storia insegna che una cifratura apparentemente inattaccabile può diventare attaccabile con nuove tecnologie (cosa succederà quando i computer quantistici saranno in commercio?).

Esistono due grandi categorie:

- **Crittografia simmetrica:** è la più antica (usata da secoli) ma si usa ancora oggi. Usa **la stessa chiave** per cifrare e decifrare. Il suo punto debole è come **condividere la chiave** in modo sicuro.
- **Crittografia asimmetrica:** usa **chiavi diverse** per cifrare e decifrare (una **pubblica** e una **privata**). È il metodo più sicuro, ma anche il più costoso in termini di calcolo.

Suggerimento — Il meglio dei due mondi

Di solito si **inizia** la comunicazione con la crittografia asimmetrica solo per scambiarsi in modo sicuro una **chiave simmetrica**, che poi si usa per il resto della comunicazione (più veloce).

7.3.1 TLS

Il **Transport Layer Security (TLS)** è un protocollo di sicurezza molto diffuso per garantire privacy e sicurezza dei dati nelle comunicazioni su Internet. L'uso principale è cifrare la comunicazione tra applicazioni web e server, ad esempio quando un browser carica un sito. Usato sopra HTTP dà **HTTPS**, ormai lo standard per i siti web.

TLS serve anche a **verificare l'identità** dei web server tramite i **certificati digitali** che il server manda al client. I certificati sono emessi da enti fidati di verifica dell'identità (**CA, Certificate Authority**). La Fig. 7.5 mostra un tipico handshake client-server.

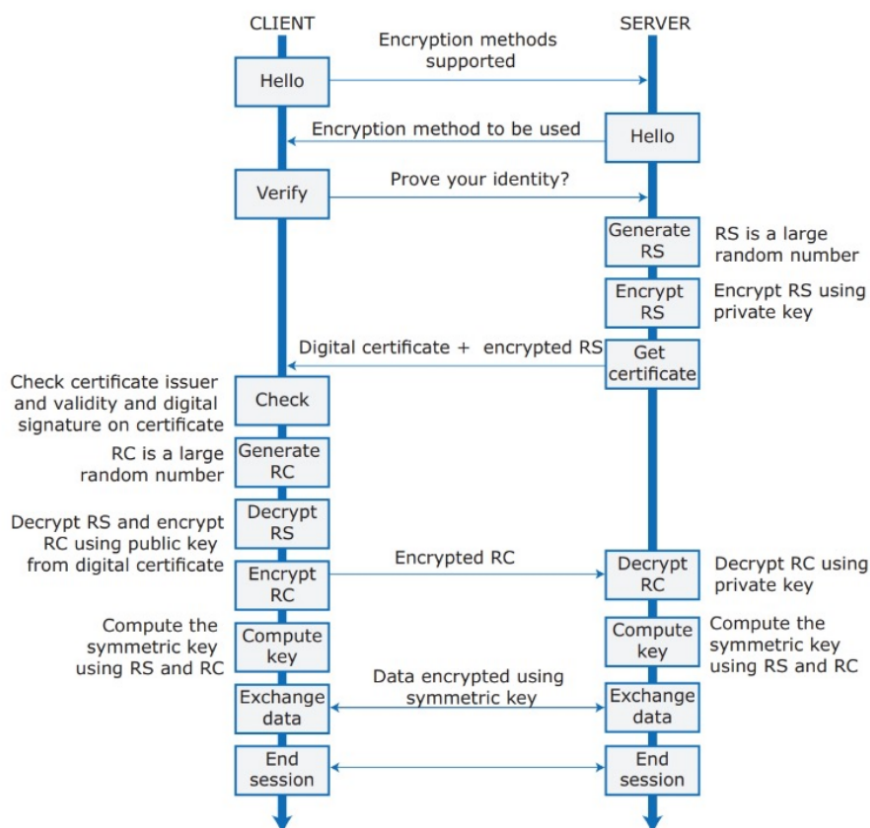


Figura 7.5: Interazione TLS tra client e server per generare la chiave simmetrica con cui scambiare i dati.

7.3.2 Quando e dove cifrare

I dati degli utenti si dividono in tre categorie:

- **Dati in transito** (*in transit*): vanno **sempre cifrati**, per avere comunicazioni private.
- **Dati a riposo** (*at rest*, cioè salvati): vanno **sempre cifrati**, così in caso di attacco nessuno ottiene le informazioni degli utenti in chiaro.
- **Dati in uso** (*in use*, cioè in elaborazione): cifrarli e decifrarli **rallenta** i tempi di risposta del sistema.

La cifratura si può fare a quattro livelli:

- **Applicazione:** l'applicazione decide quali dati cifrare e li decifra subito prima di usarli. Costa in prestazioni (la sicurezza non è gratis) e richiede una gestione delle chiavi.

- **Database:** il DBMS può cifrare l'intero database quando viene chiuso e decifrarlo quando viene riaperto, oppure cifrare solo singole tabelle o colonne.
- **File:** il sistema operativo cifra i singoli file quando vengono chiusi e li decifra quando vengono riaperti.
- **Supporto (media):** il sistema operativo cifra i dischi quando vengono smontati e li decifra quando vengono rimontati (utile per portatili rubati o persi).

Tutti questi livelli richiedono una **chiave**. Le leggi sulla protezione dei dati possono richiedere di conservare copie dei dati per anni in modo sicuro: se si perdono le chiavi, i dati cifrati diventano **inaccessibili per sempre!** Le chiavi vanno cambiate periodicamente e il database deve conservare più versioni delle chiavi con la loro data. Per semplificare si usano i **Key Management System (KMS)**, che garantiscono che le chiavi siano generate, conservate e usate in modo sicuro e solo da utenti autorizzati (Fig. 7.6).

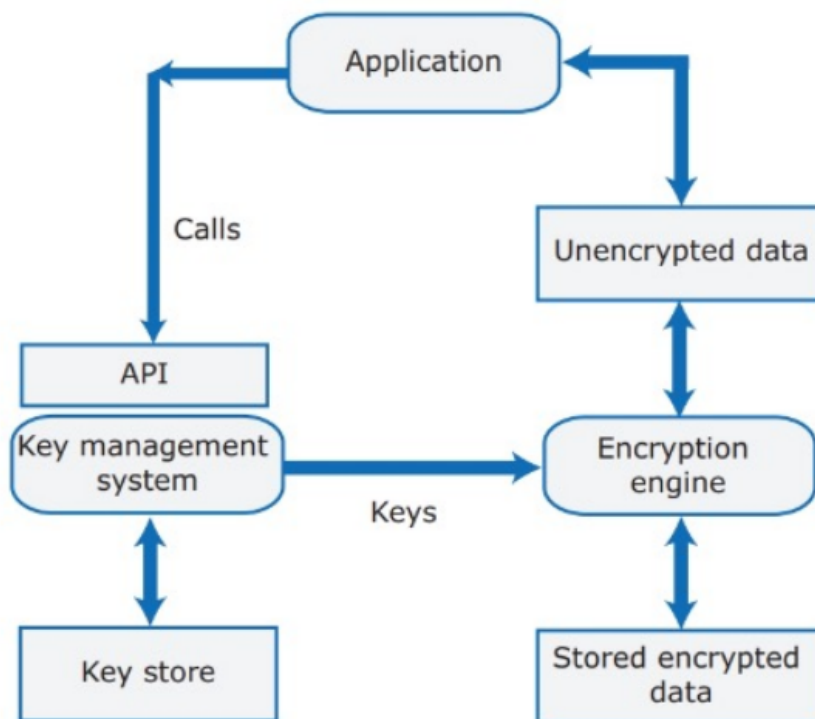


Figura 7.6: Schema d'uso di un KMS.

7.4 Privacy

Definizione — Privacy

La **privacy** è un concetto sociale che riguarda la raccolta, la diffusione e l'uso corretto delle informazioni personali possedute da terzi.



Figura 7.7: Le leggi sulla protezione dei dati.

Principi di protezione dei dati (in Europa sono alla base del **GDPR**):

- **Consapevolezza e controllo:** gli utenti devono sapere quali dati vengono raccolti mentre usano il prodotto e devono avere il controllo sulle informazioni personali raccolte.
- **Scopo:** bisogna dire agli utenti **perché** si raccolgono i dati, e non usarli per altri scopi.
- **Consenso:** serve sempre il consenso dell'utente prima di comunicare i suoi dati ad altri.
- **Durata:** non si devono tenere i dati più del necessario. Se un utente cancella l'account, vanno cancellati anche i suoi dati personali.
- **Conservazione sicura:** i dati vanno conservati in modo che non possano essere alterati o mostrati a persone non autorizzate.
- **Accesso e correzione:** gli utenti devono poter sapere quali dati personali conserviamo e poter correggere eventuali errori.
- **Luogo:** non si devono conservare i dati in Paesi con leggi sulla protezione dei dati più deboli, a meno di un accordo esplicito che garantisca regole più forti.

Ci sono anche motivi di **business** per curare la privacy:

- se il sistema non rispetta le leggi sulla protezione dei dati, il fornitore rischia cause legali o di non poter vendere il prodotto;
- se il prodotto è venduto ad aziende, i clienti aziendali possono pretendere garanzie sulla privacy (per non essere a rischio con i loro utenti);
- una fuga o un uso scorretto dei dati dei clienti può danneggiare la reputazione del fornitore.

Quali dati raccogliere dipende dalle funzionalità del prodotto e dal modello di business. Altri consigli:

- non raccogliere informazioni personali che non servono;
- definire una **privacy policy** che dica come vengono raccolte, conservate e gestite le informazioni personali e sensibili;
- dire chiaramente se si usano i dati degli utenti per la pubblicità mirata o per servizi pagati da altre aziende;
- se il prodotto ha funzioni social con cui gli utenti condividono informazioni, assicurarsi che capiscano come controllare cosa condividono.

7.5 Security smell

7.5.1 Le sfide della sicurezza nei microservizi

Vediamo le caratteristiche degli attacchi e della sicurezza tipiche dei microservizi:

- **Più ampia è la superficie d'attacco, più alto è il rischio:** i microservizi si scambiano dati con chiamate remote, quindi ci sono potenzialmente moltissimi punti d'ingresso. La sicurezza dell'applicazione dipende dall'**anello più debole**: basta un solo punto d'ingresso debole per compromettere tutto.
- **Controlli di sicurezza distribuiti peggiorano le prestazioni:** ogni microservizio deve fare i suoi controlli di sicurezza, magari collegandosi a un servizio remoto che rilascia i token. Controlli ripetuti e distribuiti rallentano il sistema. Un ripiego è **fidarsi della rete** e saltare i controlli nei singoli microservizi, ma la vera soluzione (e la tendenza del settore) è la **rete zero-trust** (non ci si fida di nessuno per default), tenendo d'occhio le prestazioni complessive.
- **Stabilire la fiducia tra microservizi richiede automazione:** la comunicazione tra servizi deve avvenire su canali protetti. Se i servizi usano certificati, ogni microservizio deve ricevere un certificato (e la relativa chiave privata) per autenticarsi con gli altri, e chi riceve deve saper validare il certificato di chi chiama. Bisogna quindi creare la fiducia tra microservizi, anche revocando e rinnovando i certificati. Con centinaia di microservizi serve per forza l'**automazione**.
- **Tracciare le richieste tra più microservizi è difficile:** i **log** si possono aggregare in metriche che mostrano lo stato del sistema (es. media di accessi non validi all'ora) e fanno scattare allarmi. Le **tracce** seguono una richiesta da quando entra nel sistema a quando esce. A differenza del monolite, una richiesta può attraversare molti microservizi, quindi è difficile collegare tra loro i pezzi della stessa richiesta.
- **I container complicano la gestione di credenziali e policy:** i container sono server **immutabili** (ottimo per semplificare il deploy e scalare in orizzontale), cioè non cambiano stato dopo l'avvio. Ma ogni servizio deve mantenere una lista dinamica di client autorizzati e un insieme dinamico di policy di accesso: l'unico modo per aggiornarle è tenere le credenziali nel file system del container e **iniettarle all'avvio**.
- **La distribuzione rende difficile condividere il contesto utente:** a una richiesta che si muove nel sistema si vogliono associare molte informazioni (es. se è una richiesta premium o no). Il problema è far sì che il microservizio che riceve si fidi del contesto utente passato da chi chiama (se qualcuno lo manipola, è un attacco). Una soluzione diffusa è il **JSON Web Token (JWT)**, un token di sicurezza per condividere il contesto utente tra microservizi.
- **Le responsabilità di sicurezza sono divise tra team diversi:** team indipendenti possono usare tecnologie diverse (buone pratiche di sicurezza, strumenti di test di sicurezza statici e dinamici, ...). Le organizzazioni spesso usano un approccio **ibrido**: un team di sicurezza centrale più esperti di sicurezza dentro i singoli team.

7.5.2 Smell e refactoring

Definizione — Security smell

Un **security smell** è una decisione architetturale di uso comune che segnala **possibili violazioni della sicurezza** in un'applicazione a microservizi.

Come per gli architectural smell (vedi [06 - Architettura a microservizi](#)), si vuole capire come rilevarli e risolverli tramite refactoring, sulla base di una multivocal review. Le proprietà di sicurezza considerate sono:

- **Confidenzialità** (*confidentiality*): quanto il sistema garantisce che i dati siano accessibili solo a chi è autorizzato.

- **Integrità** (*integrity*): quanto il sistema impedisce modifiche non autorizzate a programmi o dati.
- **Autenticità** (*authenticity*): quanto si può dimostrare che l'identità di un soggetto o di una risorsa è quella dichiarata.

Uno smell può violare più proprietà. Ecco gli smell, le proprietà violate e i refactoring:

- **Controllo degli accessi insufficiente** (*insufficient access control*) — viola la **confidenzialità**. Alcuni servizi non controllano gli accessi, e questo porta al problema del **confused deputy** (un attaccante ottiene dati a cui non dovrebbe accedere, usando un servizio che ha più permessi di lui). I permessi del client vanno verificati al momento della richiesta, senza aggiungere latenza e colli di bottiglia con chiamate continue a un servizio centrale. **Refactoring**: usare **OAuth 2.0**, un framework basato su token per il controllo degli accessi delegato.
- **Microservizi accessibili pubblicamente** (*publicly accessible microservices*) — viola la **confidenzialità**. Alcuni microservizi sono raggiungibili direttamente dai client esterni: ognuno deve controllare autenticazione e autorizzazione a ogni richiesta (più costi di manutenzione) e le credenziali sono più esposte. **Refactoring**: il più citato è un **API gateway** che, da dietro un firewall, gestisce autenticazione, autorizzazione, limitazione del traffico e validazione del contenuto dei messaggi.
- **Privilegi non necessari ai microservizi** (*unnecessary privileges*) — viola **confidenzialità** e **integrità**. A volte i microservizi hanno livelli di accesso, permessi o funzionalità che non servono alle loro funzioni di business, spesso perché i programmatori copiano e incollano schemi simili. Più risorse esposte senza motivo vuol dire superficie d'attacco più ampia. **Refactoring**: seguire il **principio del privilegio minimo** (*Least Privilege Principle*: il codice deve avere solo i permessi necessari per il suo compito, e niente di più).
- **Crittografia "fatta in casa"** (*home-made crypto code*) — viola **confidenzialità**, **integrità** e **autenticità**. Usare codice crittografico scritto da sé può essere peggio che non cifrare affatto, per il falso senso di sicurezza che dà. **Refactoring**: usare librerie crittografiche ampiamente testate dalla comunità e aggiornate regolarmente (evitando algoritmi sperimentali).
- **Dati esposti non cifrati** (*non-encrypted data exposure*) — viola **confidenzialità**, **integrità** e **autenticità**. L'applicazione espone per errore dati sensibili (es. salvati senza cifratura o con protezioni vulnerabili), e un intruso può leggerli o modificarli, credenziali comprese. **Refactoring**: **cifrare tutti i dati sensibili a riposo**; devono restare sempre cifrati ed essere decifrati solo quando servono. La maggior parte dei DBMS supporta la cifratura automatica, ma si può cifrare anche a livello di applicazione, sistema operativo e cache. Attenzione: cifrare consuma risorse, quindi va fatto solo sui dati davvero critici.
- **Segreti scritti nel codice** (*hardcoded secrets*) — viola **confidenzialità**, **integrità** e **autenticità**. Segreti come chiavi API, client secret e credenziali non vanno mai messi nel codice o nelle variabili d'ambiente, perché possono essere esposti per errore (es. un gestore di eccezioni che manda informazioni a una piattaforma di logging). **Refactoring**: cifrare i segreti a riposo, non salvare le credenziali insieme all'applicazione o nei repository del codice, non passare i segreti tramite variabili d'ambiente.
- **Comunicazione tra servizi non protetta** (*non-secured service-to-service communications*) — viola **confidenzialità**, **integrità** e **autenticità**. Senza canali sicuri i dati sono esposti ad attacchi **man-in-the-middle**, intercettazione (*eavesdropping*) e manomissione (*tampering*). **Refactoring**: usare il **mutual TLS (mTLS)**, che cifra i dati in transito e ne garantisce integrità e riservatezza, e permette a ogni microservizio di identificare con certezza l'altro (**autenticazione reciproca**).
- **Traffico non autenticato** (*unauthenticated traffic*) — viola l'**autenticità**. I microservizi

devono autenticarsi tra loro (soprattutto quando si passano il contesto utente). Altrimenti sono esposti a manomissione dei dati, denial of service o aumento dei privilegi. **Refactoring:** il **mutual TLS**, oppure **OpenID Connect**, che usa JWT con le informazioni dell'utente autenticato; i microservizi verificano l'identità dell'utente con i server di autorizzazione.

- **Autenticazione multipla** (*multiple user authentication*) — viola l'**autenticità**. È una tentazione far autenticare gli utenti da punti diversi, ma ogni punto d'accesso è un possibile varco per un intruso. **Refactoring:** il **Single Sign-On** (un solo punto d'ingresso), che semplifica anche il salvataggio dei log e i controlli. Si ottiene con un **API gateway** e **OpenID Connect** (per condividere il contesto utente).
- **Autorizzazione centralizzata** (*centralized authorization*) — viola l'**autenticità**. L'autorizzazione si può fare al bordo dell'applicazione (nell'API gateway) e/o in ogni microservizio. Se si fa **solo** al bordo, il punto centrale diventa un **collo di bottiglia**, e c'è il rischio di **confused deputy**: i microservizi si fidano del gateway solo perché è il gateway. **Refactoring:** **autorizzazione decentralizzata**, passando un **token di accesso** (es. JWT) con ogni richiesta a un microservizio e dando accesso solo se il token è valido.

7.5.3 Fare refactoring o no?

Va deciso **caso per caso**: risolvere uno smell può peggiorare altre proprietà! Esempio: passare all'autorizzazione decentralizzata rispetta i principi di design dichiarati e garantisce le proprietà di sicurezza, ma quella centralizzata è più facile da mantenere e ha prestazioni migliori.

Per vedere questi compromessi aiuta modellare le **interdipendenze tra obiettivi "soft"** (*soft goal*) come grafi, che si possono anche analizzare in modo automatico (Fig. 7.8).

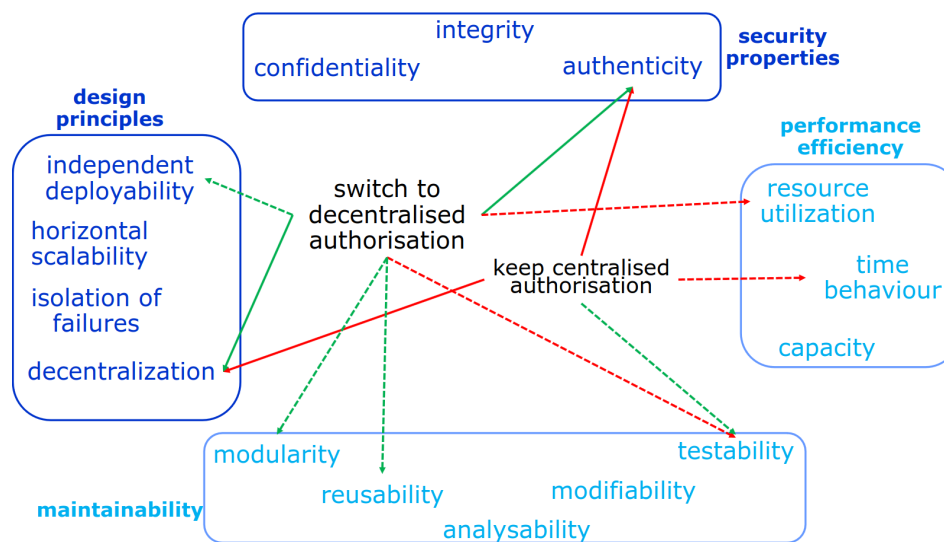


Figura 7.8: Confronto tra autorizzazione centralizzata e decentralizzata.

Capitolo 8

Business Process Modeling

Capitolo originale: 8 — Business Process Modeling • Antonio Brogi
Fonte: Appunti di Emiliano Sescu (github.com/Faxatos), cap. 8

Il **Business Process Management** (BPM) è il metodo sistematico con cui si analizzano i processi di business esistenti di un'organizzazione e si introducono miglioramenti per rendere il flusso di lavoro più efficace ed efficiente (IBM ci ha fatto una fortuna).

Definizione — Processo, modello, istanza

- Un **processo di business** è un insieme di attività di business che rappresentano i passi necessari per raggiungere un obiettivo di business.
- Un **modello di processo** (Fig. 8.1) è formato da un insieme di modelli di attività e dai **vincoli di esecuzione** tra di esse.
- Un'**istanza di processo** è un caso concreto nell'operatività di un'azienda, formato da istanze di attività.

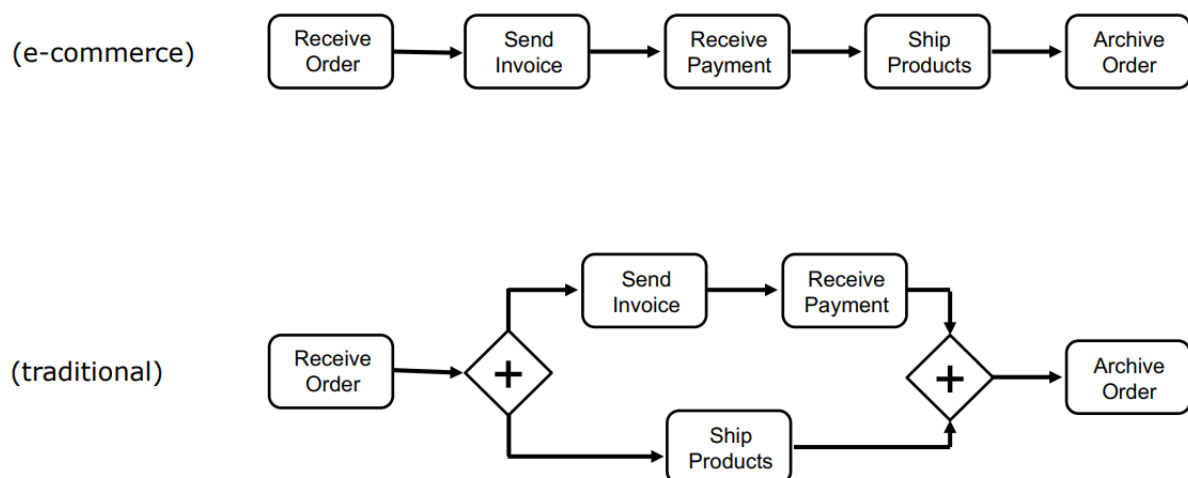


Figura 8.1: Esempio di modello di processo di business.

8.1 BPMN

La **Business Process Model and Notation** (BPMN) è la notazione grafica standard per modellare i processi di business (Fig. 8.2).

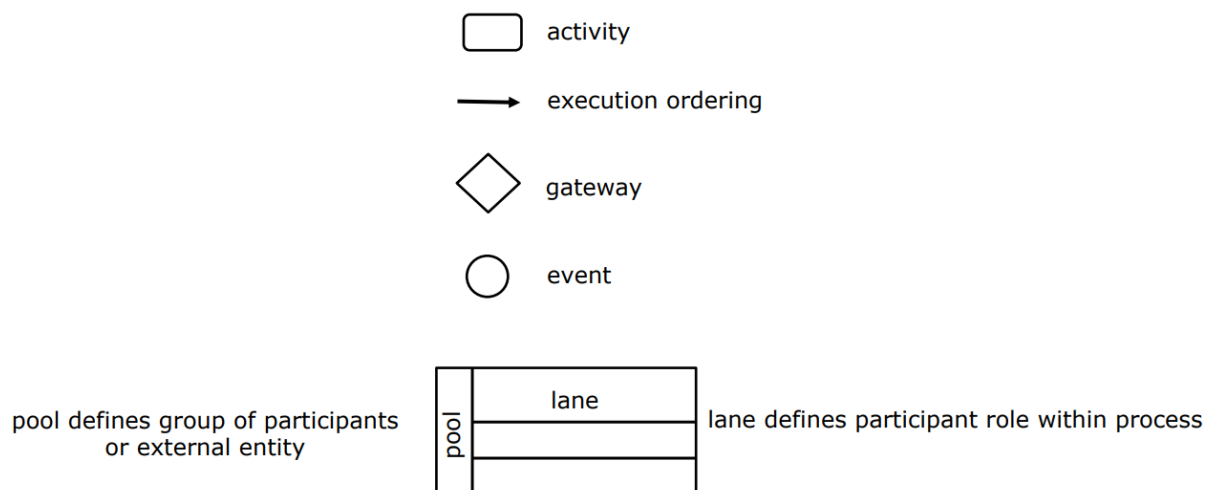


Figura 8.2: La notazione grafica per il business process modeling.

La Fig. 8.3 mostra un esempio completo, con il funzionamento di tre tipi di **gateway**:

- **parallelo (AND)**: attiva tutti i rami in uscita e, in ingresso, aspetta che arrivino tutti;
- **esclusivo (XOR)**: sceglie **uno solo** dei rami in uscita;
- **inclusivo (OR)**: attiva **uno o più** rami in base a una condizione su ciascuno; in ingresso aspetta tutti i rami che erano stati attivati.

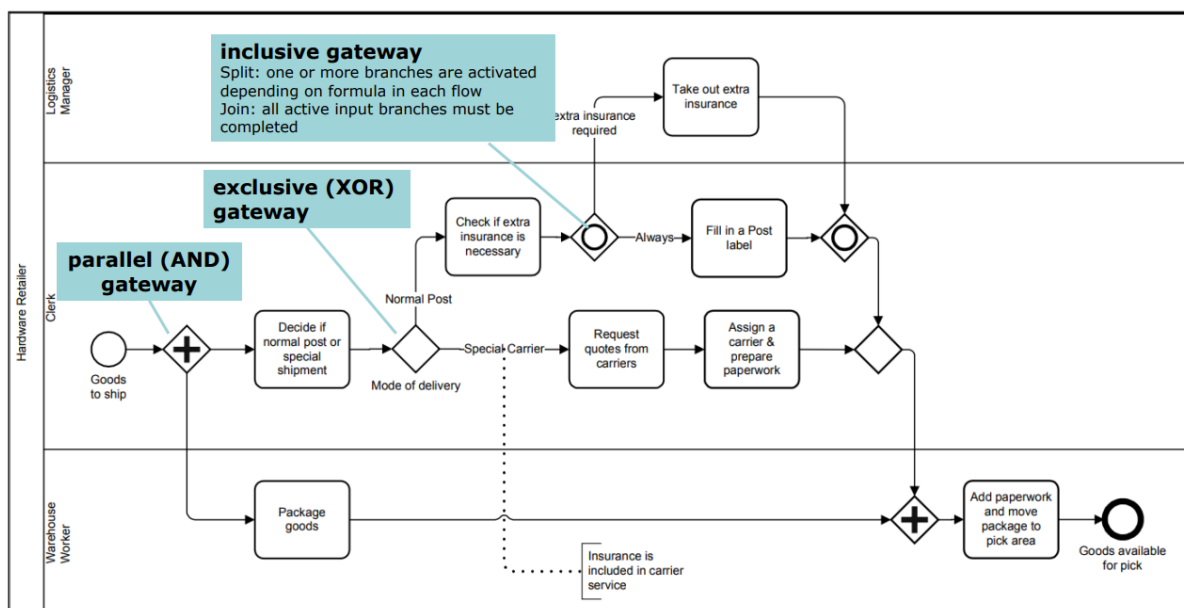


Figura 8.3: Esempio: processo di spedizione di un rivenditore di hardware.

La Fig. 8.4 mostra come più processi possono interagire tramite **message flow** (freccie tratteggiate tra pool diversi). C'è un processo principale e uno secondario. Quando si raggiunge un **terminate event**, tutte le attività del processo terminano subito. Nel processo principale si vedono anche un **timer event** e un **event-based gateway** (che aspetta un evento per scegliere il ramo).

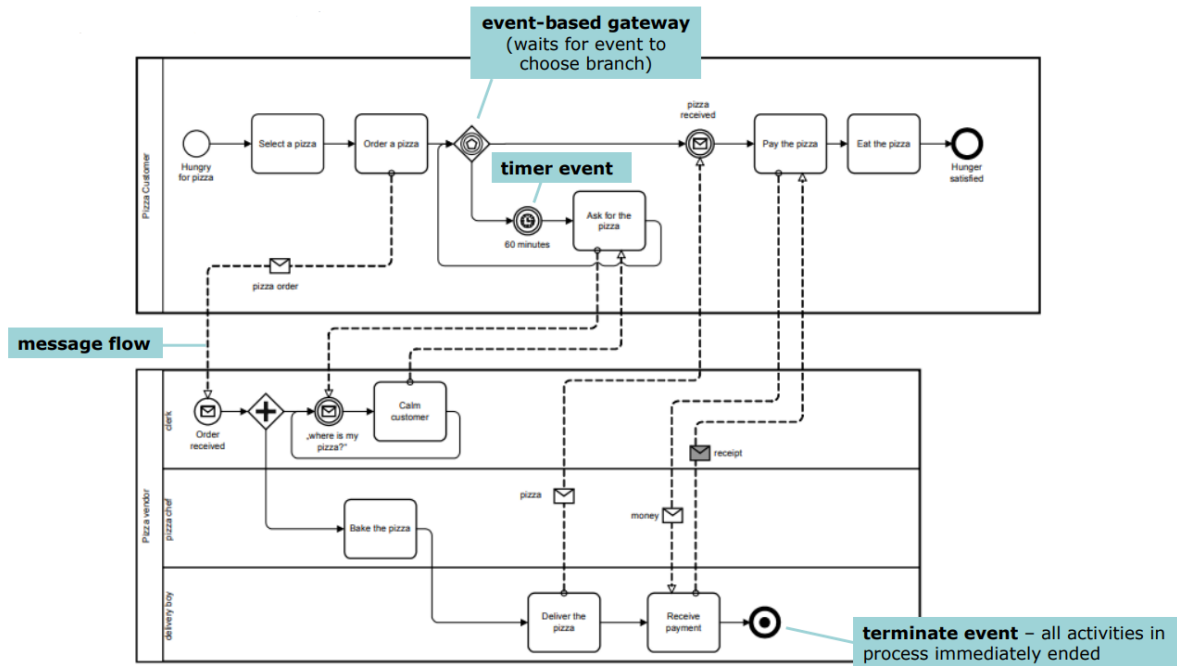


Figura 8.4: Esempio di collaborazione B2B (pizza).

L'ultimo esempio (Fig. 8.5) mostra la notazione dei **sotto-processi** (*sub-process*) e degli **eventi agganciati** (*attached event*) al bordo di un'attività:

- **error** → **interrompente**: se scatta, l'attività viene interrotta e si segue il ramo dell'evento;
- **escalation** → **non interrompente**: se scatta, si avvia il ramo dell'evento ma l'attività continua.

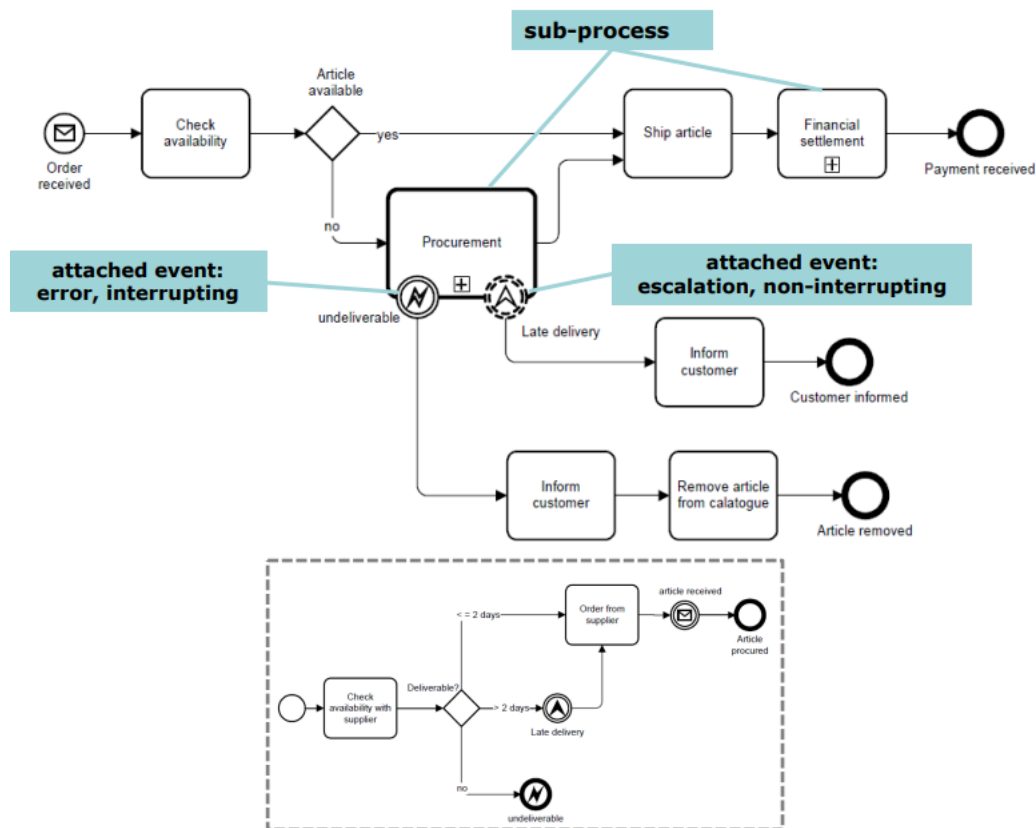


Figura 8.5: Esempio: evasione dell'ordine e approvvigionamento.

Nota — Collegamento con il corso di BPM

BPMN, reti di Petri e soundness sono trattati in dettaglio negli appunti di Business Process Modeling: [07a - EPC e BPMN](#), [04 - Petri Nets](#), [12 - Soundness](#).

8.2 Workflow net

Nel BPM è fondamentale poter **dimostrare proprietà** dei modelli di processo.

Le **workflow net** sono un'estensione delle **reti di Petri** e sono una delle tecniche più note per specificare processi di business in modo **formale e astratto**. La rappresentazione grafica delle reti di Petri facilita la comunicazione tra stakeholder diversi, e le proprietà del processo si possono analizzare formalmente con vari strumenti disponibili.

La Fig. 8.6 mostra come una rete di Petri renda leggibile un modello di processo, traducendo l'esempio di Fig. 8.1.

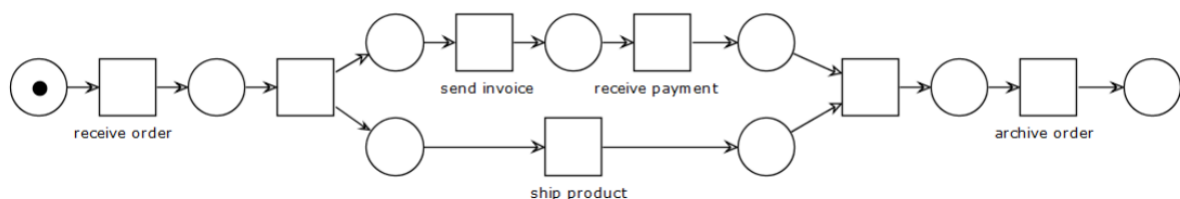


Figura 8.6: La rete di Petri dell'esempio precedente (Fig. 8.1).

8.2.1 Reti di Petri

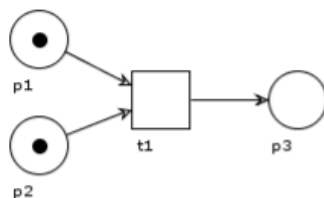
Una **rete di Petri** è formata da:

- **transizioni** (quadrati), che modellano le **attività**;
- **posti** (*place*, cerchi) e **archi** orientati tra posti e transizioni, che modellano i **vincoli di esecuzione**.

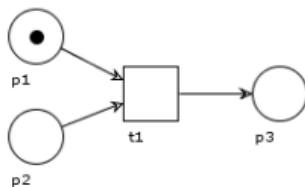
La dinamica del sistema è rappresentata dai **token** (i pallini): come sono distribuiti nei posti determina lo **stato** del sistema modellato.

Definizione — Regola di scatto

- Una transizione **può scattare** (è *abilitata*) se c'è **un token in ognuno dei suoi posti di input** (Fig. 8.7).
- Quando scatta, **toglie un token** da ogni posto di input e **aggiunge un token** a ogni posto di output (Fig. 8.8).



t1 can fire



t1 cannot fire

Figura 8.7: Reti di Petri: la regola di input.

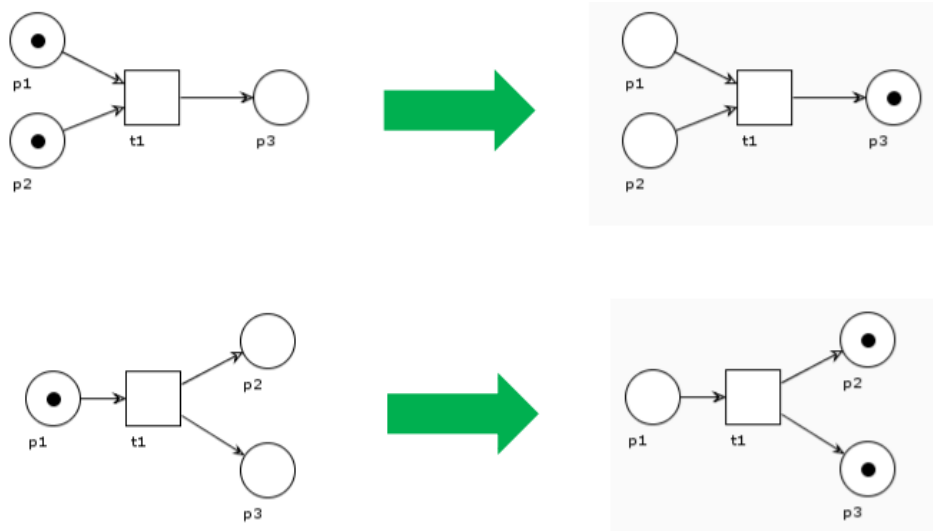


Figura 8.8: Reti di Petri: la regola di output.

8.2.2 Workflow net e soundness

Le workflow net arricchiscono le reti di Petri con notazioni che semplificano la rappresentazione dei processi di business. Come le reti di Petri, si concentrano sul **flusso di controllo** del processo:

- le **transizioni** rappresentano le **attività**;
- i **posti** rappresentano le **condizioni**;
- i **token** rappresentano le **istanze di processo**.

La Fig. 8.9 mostra alcuni pattern di composizione utilizzabili con le reti di Petri.

Definizione — Workflow net

Una rete di Petri è una **workflow net** se e solo se:

1. c'è un **unico posto sorgente** (*source*, *i*) senza archi in ingresso;
2. c'è un **unico posto pozzo** (*sink*, *o*) senza archi in uscita;
3. ogni posto e ogni transizione si trova su **qualche cammino** dal posto iniziale a quello finale.

Definizione — Soundness

Una workflow net è **sound** (corretta) se e solo se:

1. ogni esecuzione della rete che parte dallo stato iniziale (un token nel posto sorgente, nessun altro token) **arriva prima o poi allo stato finale** (un token nel posto pozzo, nessun altro token);
2. ogni transizione compare in **almeno un'esecuzione** della rete.

Attenzione — Cosa dice (e cosa non dice) la soundness

Le workflow net **astraggono dai dati**: le scelte che dipendono dai dati diventano scelte “alla cieca”. Di conseguenza l'analisi può considerare **più rami del necessario**.

- Una rete **non sound** va presa solo come un **avvertimento** di possibili problemi a

runtime: l'analisi di un ramo che in realtà non verrà mai eseguito può rendere la rete non sound, anche se l'applicazione non lo eseguirà mai.

- Al contrario, l'analisi di un ciclo può non accorgersi che l'applicazione non terminerà mai: una rete **sound non garantisce** che l'applicazione termini sempre.

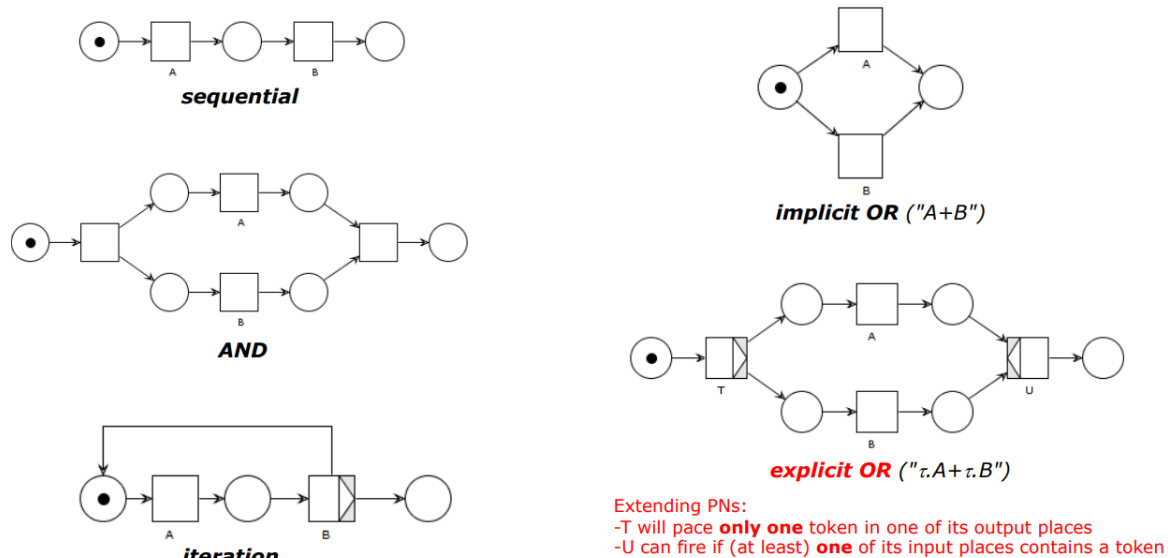


Figura 8.9: Pattern di composizione delle reti di Petri.

8.2.3 Verificare la soundness

Come si stabilisce in modo formale (e automatico) se una rete è sound? Servono prima due proprietà:

Definizione — Liveness

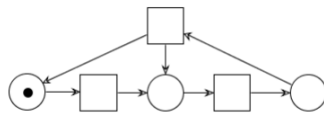
Una rete di Petri (PN, M) è **viva** (*live*) se e solo se per ogni stato raggiungibile M' e per ogni transizione t esiste uno stato M'' raggiungibile da M' in cui t è abilitata. In parole semplici: da qualunque punto si arrivi, ogni transizione può ancora scattare prima o poi.



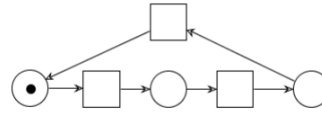
Esempio di rete non viva (a sinistra) e viva (a destra).

Definizione — Boundedness

Una rete di Petri (PN, M) è **limitata** (*bounded*) se e solo se per ogni posto p esiste un $n \in \mathbb{N}$ tale che, in ogni stato raggiungibile M' , il numero di token in p è minore di n . In parole semplici: nessun posto può accumulare infiniti token.



not bounded



bounded

Esempio di rete non limitata (a sinistra) e limitata (a destra).

Teorema — Soundness tramite liveness e boundedness

Una workflow net N è **sound** se e solo se $(\bar{N}, \{i\})$ è **viva e limitata**, dove $\bar{N}$ è la rete N a cui si aggiunge una transizione che va dal posto pozzo o al posto sorgente i .

Suggerimento — Perché si aggiunge la transizione da o a i

“Cortocircuitando” la rete, il processo può ripartire dopo aver finito. Così la domanda “ogni esecuzione termina correttamente e ogni attività è utile?” diventa una domanda su proprietà classiche (liveness e boundedness) che i tool sanno già verificare.

Capitolo 9

Testing

Capitolo originale: 9 — Testing • Antonio Brogi

Fonte: Appunti di Emiliano Sescu (github.com/Faxatos), cap. 9

Definizione — Testing

Il **testing** è il processo sistematico con cui si esegue un programma usando dati che simulano l'input dell'utente, per verificare che il software funzioni come previsto e rispetti i requisiti.

Il testing serve a osservare in modo metodico il comportamento del programma. Il principio di base è **confrontare il comportamento atteso con quello reale** durante l'esecuzione:

- se il comportamento osservato è quello atteso, il test **passa**;
- se si discosta da quello atteso, il test **fallisce**.

Alcuni tipi di test:

- **Test funzionale** (*functional testing*): verifica che l'applicazione svolga le sue funzioni come previsto. Si valutano le singole feature per controllare che rispettino i requisiti, guardando input, output e comportamento del sistema in varie condizioni.
- **Test con gli utenti** (*user testing*, o test di usabilità): valuta il software dal punto di vista dell'utente finale, per verificare che sia facile e intuitivo da usare e che risponda ai bisogni di chi lo userà. Spesso coinvolge utenti reali, per trovare problemi di usabilità, difetti di design o punti da migliorare.
- **Test di prestazioni e di carico** (*performance and load testing*): il test di prestazioni misura reattività, velocità ed efficienza in condizioni diverse. Il test di carico, un suo sottoinsieme, valuta come si comporta il sistema con livelli simulati o previsti di utenti contemporanei. Aiutano a trovare colli di bottiglia, misurare la scalabilità e assicurare buone prestazioni con carichi diversi.
- **Test di sicurezza** (*security testing*): cerca vulnerabilità e punti deboli per proteggere il sistema da minacce e accessi non autorizzati. Valuta la resistenza ad attacchi, furti di dati e manipolazioni, e aiuta a garantire che i dati sensibili siano protetti e che si rispettino gli standard di sicurezza.

Attenzione — Il limite del testing

Il testing può mostrare la **presenza** di bug, ma **mai la loro assenza** (Dijkstra): non sostituisce la verifica formale del software.

Il resto del capitolo approfondisce il test funzionale e il test di sicurezza.

9.1 Test funzionale

Il **test funzionale** è un insieme completo di test che garantiscono che **tutto il codice venga eseguito almeno una volta**, per verificare il comportamento complessivo del sistema. Il testing deve iniziare **dal giorno in cui si comincia a scrivere codice**, e i test automatici semplificano il ciclo sviluppo-test.

Il test funzionale è un'attività **a fasi**: il processo è organizzato in fasi distinte (Fig. 9.1). Le fasi non sono rigidamente in sequenza e spesso si sovrappongono. Inoltre il **test di regressione** si fa continuamente, in ogni fase, per controllare che modifiche o nuove feature non introducano difetti in funzionalità già testate. Dividere il lavoro in fasi aiuta a trovare e risolvere i problemi ai diversi livelli dello sviluppo.

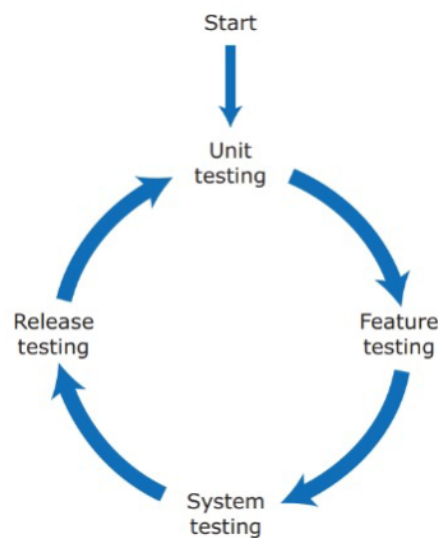


Figura 9.1: Il ciclo di vita del test funzionale.

9.1.1 Unit testing

L'obiettivo dello **unit testing** è testare le **unità del programma** (es. funzioni, metodi) in **isolamento**.

Definizione — Principio dello unit test

Se un'unità di programma si comporta come previsto per un insieme di input che hanno alcune caratteristiche in comune, si comporterà allo stesso modo per un insieme più grande di input con le stesse caratteristiche.

Esempio — Esempio

Se un programma si comporta correttamente con gli input 1, 5, 17, 45, 99, si può concludere che elaborerà correttamente anche tutti gli altri interi tra 1 e 99.

Per questo è importante individuare le **partizioni di equivalenza**: insiemi di input che vengono trattati allo stesso modo da un pezzo di codice. Poi si testa il programma con diversi input presi da ogni partizione. I programmatori sbagliano spesso proprio sui **limiti** delle partizioni, quindi conviene usare i valori di confine come input.

Linee guida per gli unit test:

- **Testa i casi limite:** se la partizione ha un limite inferiore e superiore (lunghezza delle stringhe, numeri, ...), scegli input ai bordi dell'intervallo.
- **Forza gli errori:** scegli input che facciano generare al sistema tutti i messaggi di errore, e input che dovrebbero produrre output non validi.
- **Riempi i buffer:** scegli input che facciano traboccare tutti i buffer di input.
- **Ripeti:** ripeti più volte lo stesso input o la stessa serie di input.
- **Overflow e underflow:** se il programma fa calcoli numerici, scegli input che producano numeri molto grandi o molto piccoli.
- **Non dimenticare null e zero:** se il programma usa puntatori o stringhe, testa sempre con puntatori e stringhe nulli; con le sequenze, testa la sequenza vuota; con i numeri, testa sempre lo zero.
- **Tieni il conto:** con liste e trasformazioni di liste, conta gli elementi di ogni lista e controlla che i conti tornino dopo ogni trasformazione.
- **Uno è diverso:** se il programma lavora su sequenze, testa sempre anche sequenze con un solo elemento.

Esempio — Partizioni e limiti in pratica

Una funzione accetta un'età tra 18 e 65. Le partizioni sono: minore di 18 (non valida), 18-65 (valida), maggiore di 65 (non valida). Oltre a un valore "normale" per ogni partizione (es. 10, 40, 80), si testano i confini: 17, 18, 65, 66.

9.1.2 Feature testing

Una feature implementa una funzionalità utile per l'utente. L'obiettivo del **feature testing** è verificare che quella funzionalità sia implementata come previsto e risponda ai bisogni reali degli utenti. Le feature di solito sono realizzate da più unità che interagiscono, quindi ci sono due tipi di test:

- **Test di interazione:** verificano le interazioni tra unità (di solito scritte da sviluppatori diversi). Possono anche scoprire bug nelle singole unità sfuggiti agli unit test.
- **Test di utilità** (*usefulness tests*): verificano che la feature faccia quello che gli utenti probabilmente vogliono. Il **Product Manager** dovrebbe partecipare alla loro progettazione.

I test delle feature si ricavano da **scenari** o **user story**.

Esempio — Da user story a test

User story — Registrazione: "Come utente, voglio poter accedere senza creare un nuovo account, così non devo ricordare un altro login e un'altra password."

Test — Schermata di login iniziale:

- verificare che, quando l'utente clicca su "Accedi con Google", venga mostrata correttamente la schermata che chiede le credenziali Google;
- verificare che il login vada a buon fine se l'utente ha già fatto l'accesso a Google.

9.1.3 System testing

L'obiettivo del **system testing** è testare il sistema **nel suo insieme**, per:

- scoprire interazioni inattese o indesiderate tra le feature;

- capire se le feature lavorano bene insieme per supportare ciò che gli utenti vogliono davvero fare;
- assicurarsi che il sistema funzioni come previsto nei diversi ambienti in cui verrà usato;
- testare reattività, throughput, sicurezza e gli altri attributi di qualità.

Un consiglio per creare i test di sistema è usare scenari e user story per individuare i **percorsi completi** (*end-to-end*) degli utenti, perché si vuole testare il flusso dei dati attraverso moduli diversi.

9.1.4 Release testing

Il **release testing** testa un sistema pronto per essere rilasciato ai clienti, nel suo **ambiente operativo reale** (non in un ambiente di test). L'obiettivo è **decidere se il sistema è abbastanza buono da essere rilasciato**, non trovare bug.

Serve perché preparare un sistema per il rilascio vuol dire impacchettarlo per il deploy, installare software e librerie necessari e configurare i parametri, e in questi passaggi si possono fare errori. Se il software è nel cloud, si può usare un processo di **rilascio continuo** automatico.

9.2 Test di sicurezza

Gli obiettivi del **test di sicurezza** sono:

- trovare le **vulnerabilità** che un attaccante potrebbe sfruttare;
- fornire prove convincenti che il sistema è **abbastanza sicuro**.

Trovare vulnerabilità è più difficile che trovare bug, perché bisogna testare qualcosa che il software **NON** deve fare (potenzialmente infiniti test). I normali test funzionali possono non rivelare le vulnerabilità, e anche il software su cui si appoggia il prodotto (sistema operativo, librerie, database, ...) può contenerne.

Un test di sicurezza completo richiede conoscenze specialistiche (es. coinvolgere esperti esterni per il **penetration testing**, che costa molto). Di solito le aziende usano un **approccio basato sul rischio**: individuano i principali rischi di sicurezza del prodotto e scrivono test che dimostrino che il prodotto se ne protegge. Alcuni test si possono automatizzare, altri richiedono per forza controlli manuali del comportamento e dei file.

Esempi di rischi di sicurezza:

- un attaccante non autorizzato entra nel sistema usando credenziali autorizzate;
- una persona autorizzata accede a risorse che le sono vietate;
- il sistema di autenticazione non riconosce un attaccante non autorizzato;
- un attaccante accede al database con una SQL injection;
- gestione scorretta delle sessioni HTTP;
- i cookie di sessione HTTP finiscono in mano a un attaccante;
- dati riservati non cifrati;
- chiavi di cifratura finite in mano a un potenziale attaccante.

Suggerimento — Pensa come un attaccante

Quando si testa la sicurezza bisogna ragionare come un **attaccante**, non come un utente normale: provare apposta a fare le cose sbagliate e ripetere le azioni più volte.

9.3 Automazione dei test

I **test automatici** sono molto usati nelle aziende di prodotto, perché testare il sistema a mano è noioso e soggetto a errori. I test eseguibili controllano che il software restituisca il risultato atteso per certi dati di input (Fig. 9.2).

Definizione — Struttura di un test automatico (Arrange-Act-Assert)

1. **Arrange** (prepara): si prepara il sistema per eseguire il test (si definiscono i parametri del test).
2. **Action** (esegui): si chiama l'unità da testare con quei parametri.
3. **Assert** (verifica): si dichiara cosa deve essere vero se il test va a buon fine.

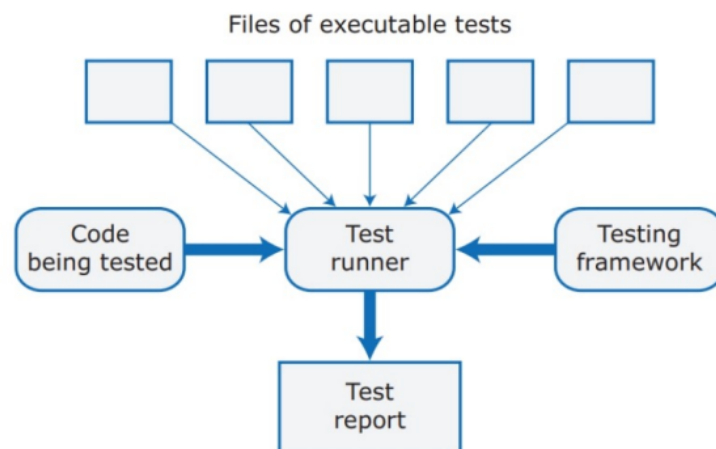


Figura 9.2: Schema del test runner.

Esempio — Un test con Arrange-Act-Assert (Python)

```

def test_sconto_10_percento():
    carrello = Carrello(totale=100)      # Arrange
    carrello.applca_sconto(0.10)        # Action
    assert carrello.totale == 90         # Assert
  
```

Esistono framework di test per tutti i linguaggi più usati. Anche il **codice dei test può contenere bug**: per ridurre il rischio conviene scrivere test il più semplici possibile e revisionare i test insieme al codice che testano.

Gli **unit test** sono i più facili da automatizzare. Buoni unit test riducono (ma non eliminano) il bisogno di feature test, ed è un bene perché i **test tramite interfaccia grafica** (GUI) sono costosi da automatizzare (Fig. 9.3): conviene testare tramite le **API**. Per controllare che una feature abbia funzionato come previsto servono più asserzioni.

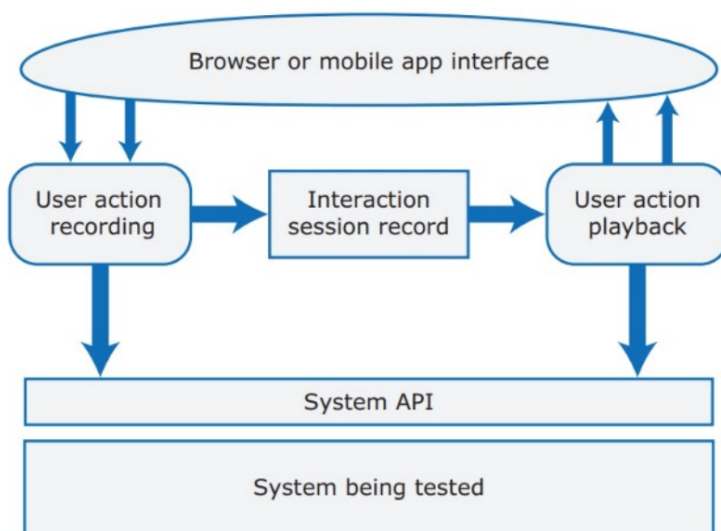


Figura 9.3: Il ciclo del test basato su GUI.

9.4 Test-Driven Development

Alcuni framework Agile, come l'Extreme Programming, sono **test-driven**: gli sviluppatori scrivono **prima i test eseguibili** e **poi il codice** che li fa passare. La Fig. 9.4 mostra come il **Test-Driven Development (TDD)** si inserisce nel ciclo di sviluppo.

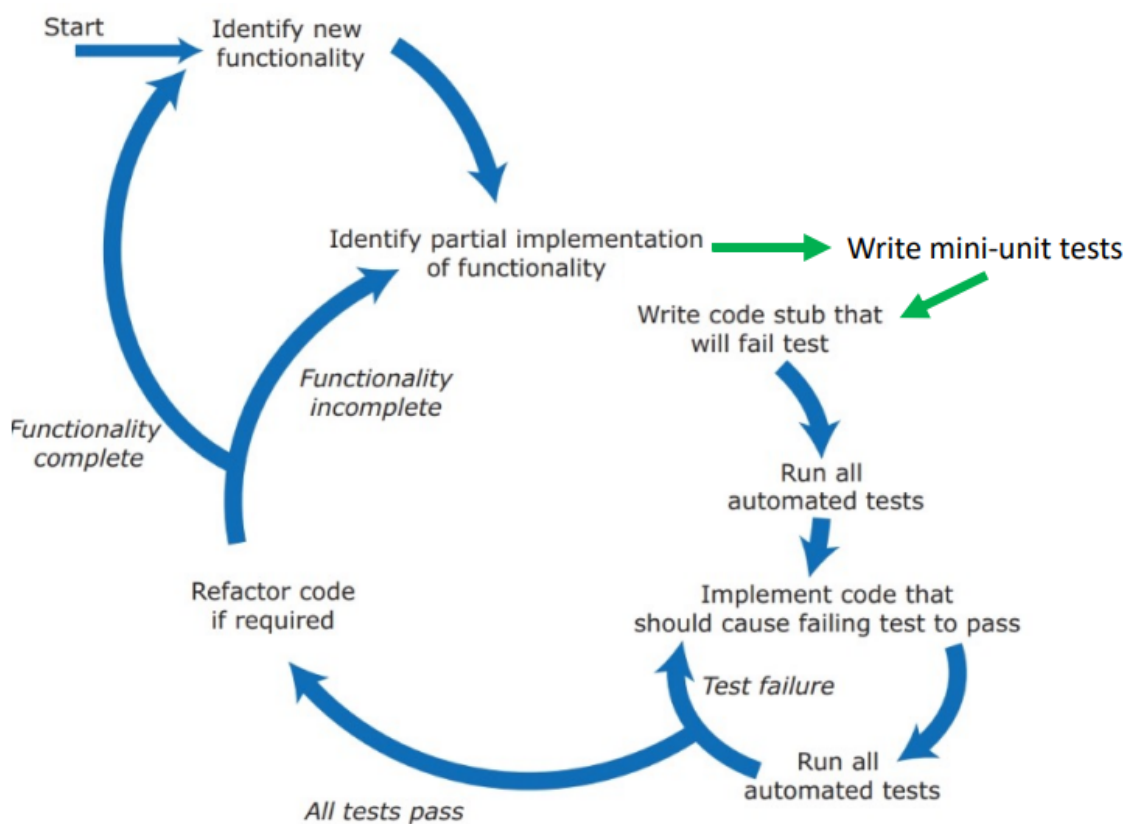


Figura 9.4: Il ciclo del Test-Driven Development.

Suggerimento — Il ciclo in tre parole: red, green, refactor

Si scrive un test che fallisce (**red**), si scrive il codice minimo per farlo passare (**green**), si migliora il codice mantenendo i test verdi (**refactor**).

Vantaggi:

- **Approccio sistematico**: i test sono legati chiaramente a parti del codice, quindi non ci sono parti non testate.
- I test aiutano a **capire il codice**.
- **Debug semplificato** e incrementale.
- **Codice più semplice** (punto discutibile).

Svantaggi:

- è difficile applicare il TDD al **test di sistema**;
- scoraggia i **cambiamenti radicali** al programma;
- porta a concentrarsi sui test invece che sul problema da risolvere;
- porta a pensare troppo ai dettagli di implementazione invece che alla struttura generale del programma;
- è difficile scrivere test con “**dati sbagliati**”.

9.5 Code review

Il testing ha dei limiti:

- Gli sviluppatori testano il codice in base a **come hanno capito** cosa deve fare. Se hanno capito male, l'errore sarà sia nel codice sia nei test.
- I test possono non coprire tutto il codice scritto (il TDD sposta il problema sull'incompletezza del codice).
- I test non dicono molto su altre qualità del programma (leggibilità, struttura, facilità di evoluzione).

Per questo le **code review** (revisioni del codice) completano il testing. La Fig. 9.5 mostra come lavora un revisore.

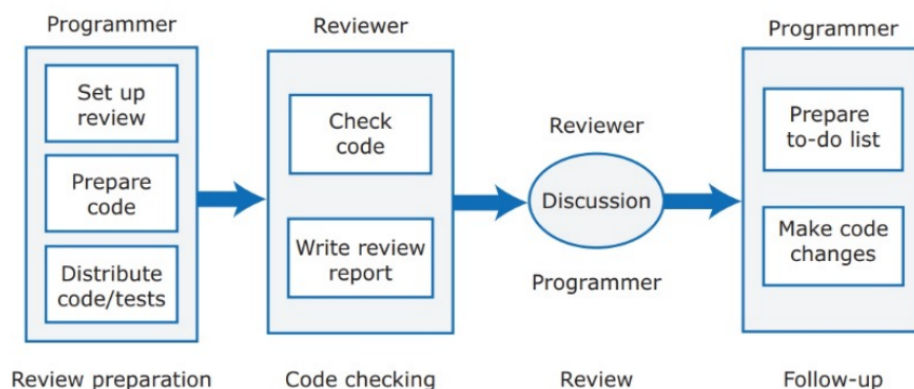


Figura 9.5: Il ciclo delle code review.

Spesso c'è **un solo revisore** (dello stesso team DevOps o di un altro), che può commentare anche la leggibilità e la comprensibilità del codice. Ogni sessione di review dovrebbe riguardare **200-400 righe di codice** e può partire in automatico dai commit sui repository condivisi.

Il revisore usa spesso una **checklist** con punti generali o specifici del linguaggio. Esempi:

- (generale) I nomi di variabili e funzioni sono significativi?
- (generale) Sono stati considerati tutti gli errori sui dati, con i relativi test?
- (generale) Tutte le eccezioni sono gestite esplicitamente?
- (Python) Si usano parametri di default nelle funzioni?
- (Python) I tipi sono usati in modo coerente?
- (Python) Il livello di indentazione è corretto?

Capitolo 10

DevOps

Capitolo originale: 10 — DevOps • Antonio Brogi

Fonte: *Appunti di Emiliano Sescu (github.com/Faxatos), cap. 10*

Nell'ingegneria del software project-based c'erano due team separati:

- un **team di sviluppo**, che si occupava di tutto lo sviluppo (design, requisiti, specifiche, prototipi, test, integrazione) fino a consegnare un software testato e pronto al rilascio;
- un **team operativo** (*operations*), che installava e manteneva il sistema (supporto agli utenti ed eventuali modifiche al software).

I problemi di questo modello erano i ritardi di comunicazione tra i team, strumenti diversi, competenze diverse e spesso la difficoltà a capire i problemi dell'altro team. In più, per correggere bug urgenti o vulnerabilità di sicurezza servivano giorni, perché il team operativo non aveva sviluppato il software.

Tre fattori hanno permesso il cambiamento:

- l'**ingegneria del software Agile** ha ridotto i tempi di sviluppo, e il processo di rilascio tradizionale è diventato un collo di bottiglia;
- **Amazon** ha riprogettato il suo software a (micro)servizi, affidando sviluppo e supporto di ogni servizio **allo stesso team**;
- è diventato possibile rilasciare software come **SaaS** su cloud pubblici o privati.

Definizione — DevOps

Il **DevOps** (*Development + Operations*) unisce sviluppo, deploy e supporto in **un unico team**.

I principi del DevOps sono:

- **Tutti sono responsabili di tutto**: tutti i membri del team condividono la responsabilità di sviluppare, rilasciare e supportare il software.
- **Tutto ciò che si può automatizzare va automatizzato**: tutte (o quasi) le attività di test, deploy e supporto devono essere automatiche.
- **Prima misura, poi cambia**: il DevOps deve essere guidato dai dati raccolti sul sistema e sul suo funzionamento.

Vantaggi del DevOps:

- **Deploy più veloce** (il vantaggio principale): i ritardi di comunicazione tra persone si riducono drasticamente, e si passa in produzione in ore invece che in giorni o settimane.
- **Meno rischi**: ogni release aggiunge piccoli incrementi di funzionalità, quindi ci sono meno probabilità di interazioni impreviste tra feature e di guasti.
- **Riparazioni più rapide**: non bisogna scoprire quale team deve risolvere il problema e aspettare che lo faccia.

- **Team più produttivi:** i team DevOps sono più produttivi di team che si occupano di attività separate.

Un team DevOps mette insieme competenze diverse: ingegneria del software, UX design, sicurezza, infrastruttura, rapporto con i clienti e altro. Il suo successo si basa su:

- una **cultura di rispetto reciproco e condivisione:** tutti partecipano agli scrum e alle altre riunioni, e ognuno è incoraggiato a condividere le proprie competenze e a impararne di nuove;
- il **supporto degli sviluppatori al software che hanno scritto:** se un servizio si guasta nel weekend, è lo sviluppatore che deve rimetterlo in funzione; se non è disponibile, lo fa un altro membro del team. I team DevOps puntano a riparare i guasti il più in fretta possibile, **non a cercare un colpevole.**

10.1 Gestione del codice

Durante lo sviluppo di un prodotto si scrivono decine di migliaia di righe di codice e di test automatici, organizzate in centinaia di file. Si usano decine di librerie e servono vari programmi per creare ed eseguire il codice: senza un supporto automatico è impossibile tenere traccia delle modifiche.

Definizione — Sistema di gestione del codice

Un **code management system** è un software che supporta le pratiche per gestire un codice che evolve nel tempo. Deve garantire che le modifiche di sviluppatori diversi **non interferiscano** tra loro e aiutare a creare **versioni diverse** del prodotto. Deve anche semplificare la creazione del prodotto eseguibile dai sorgenti e l'esecuzione dei test automatici.

La Fig. 10.1 mostra i componenti: in alto la pipeline **CI/CD** (*Continuous Integration / Continuous Delivery-Deployment*), in basso le **misurazioni DevOps**, al centro il sistema di gestione del codice con le sue funzionalità principali.

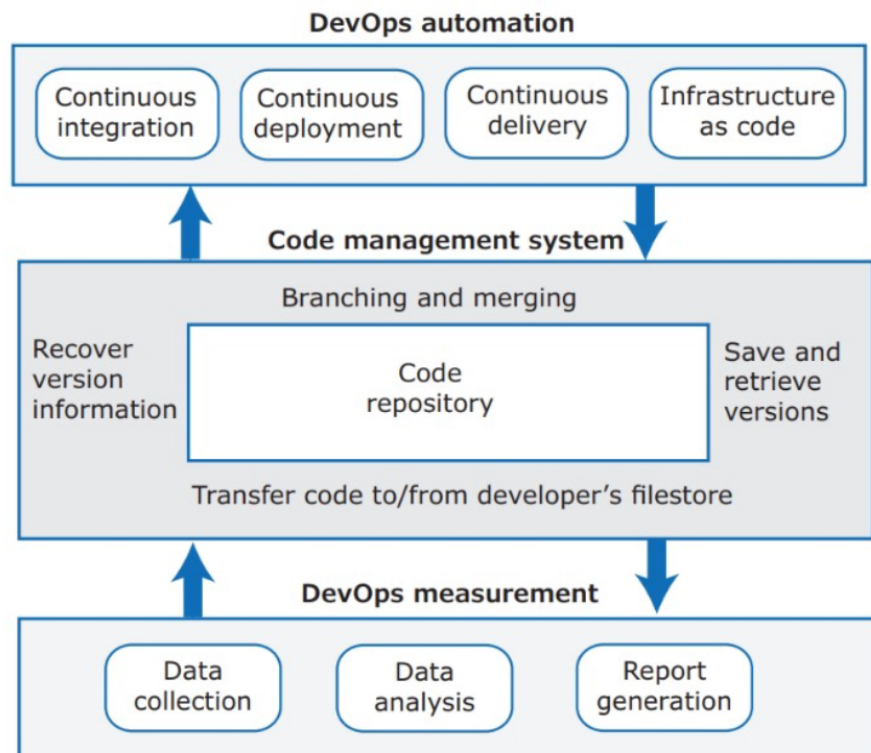


Figura 10.1: La struttura di un sistema DevOps.

Funzionalità di un sistema di gestione del codice:

- **Identificazione di versioni e release:** ogni versione di un file inviata al sistema riceve un identificatore unico (i file gestiti non vengono mai sovrascritti) e si può recuperare con l'identificatore o con altri attributi del file.
- **Registrazione della storia delle modifiche:** chi invia una modifica deve aggiungere una nota che ne spiega il motivo, così gli altri capiscono perché è nata una nuova versione.
- **Sviluppo indipendente:** più sviluppatori possono lavorare sullo stesso file contemporaneamente. Ogni invio crea una nuova versione, quindi i file non vengono mai sovrascritti da modifiche successive.
- **Supporto ai progetti:** gli sviluppatori scaricano il codice nel loro spazio personale, ci lavorano e lo rimandano al sistema condiviso. Si possono creare **branch** paralleli per lavorare in contemporanea, e le modifiche fatte in branch diversi si possono unire (**merge**). Tutti i file di un progetto si possono scaricare insieme.
- **Gestione dello spazio:** meccanismi efficienti evitano di tenere più copie di file che differiscono di poco (oggi meno importante, perché lo spazio costa poco).

10.1.1 Sistemi centralizzati e distribuiti

All'inizio si usavano sistemi **centralizzati**, ma i vantaggi dei sistemi **distribuiti** (es. **Git**, creato per il kernel Linux) li hanno resi la soluzione migliore:

- **Resilienza:** ognuno ha la propria copia del repository (e può lavorare anche offline). Se il repository condiviso si danneggia o viene attaccato, si continua a lavorare e lo si ripristina dalle copie.
- **Velocità:** fare commit è veloce, perché si può fare in locale (senza trasferire dati in rete) oltre che sul branch comune.

- **Flessibilità:** avendo ognuno la propria copia, sperimentare in locale è molto più semplice e sicuro.

In Git ogni sviluppatore ha sul proprio computer un **clone privato** del repository condiviso, e c'è un **repository di progetto condiviso** (sul server dell'azienda o nel cloud, su servizi come GitHub o GitLab). La Fig. 10.2 mostra alcuni comandi Git e l'aspetto dei branch.

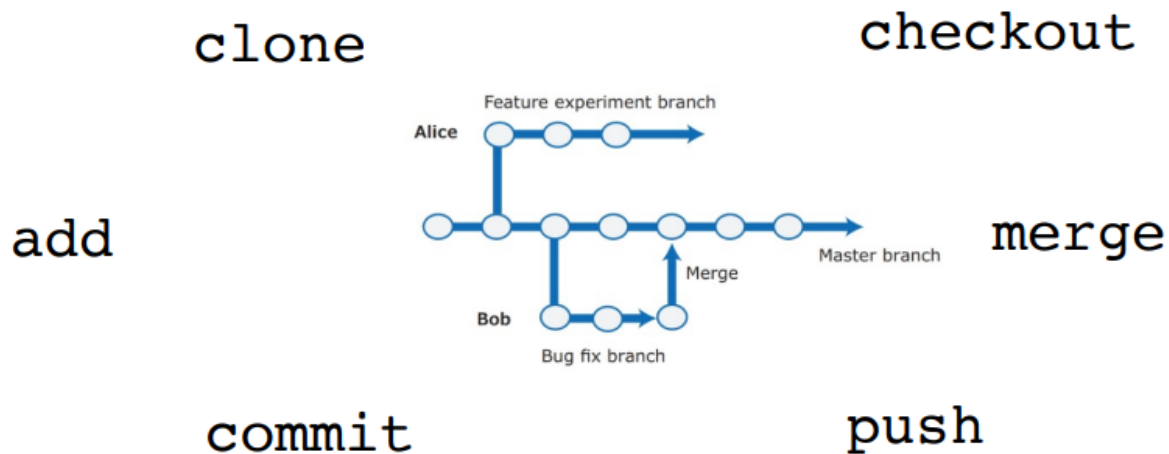


Figura 10.2: Esempio di branch Git.

Suggerimento — I comandi Git della figura

- **clone:** copia in locale un repository remoto.
- **add:** prepara le modifiche da salvare (le mette nella *staging area*).
- **commit:** salva le modifiche preparate come nuova versione nel repository locale.
- **push:** invia i commit locali al repository condiviso.
- **checkout:** passa a un altro branch o a un'altra versione.
- **merge:** unisce le modifiche di un branch in un altro.

Nello sviluppo open source con Git di solito c'è un gruppo di persone che decide quali modifiche accettare. GitHub usa un meccanismo chiamato **Webhooks** per avviare gli strumenti di automazione DevOps quando il repository del progetto viene aggiornato.

10.2 Automazione DevOps

“Tutto ciò che si può automatizzare va automatizzato”: vediamo nel dettaglio questo principio del DevOps. Le forme di automazione sono:

- **Continuous integration:** ogni volta che uno sviluppatore fa commit di una modifica sul branch principale (*master*), si costruisce e si testa una versione eseguibile del sistema.
- **Continuous delivery:** costruita la nuova versione, il software eseguibile viene testato in un ambiente che simula quello operativo del prodotto.
- **Continuous deployment:** una nuova release del sistema viene resa disponibile agli utenti ogni volta che si fa una modifica al branch principale.
- **Infrastructure as code:** modelli leggibili dalle macchine dell'infrastruttura su cui gira il prodotto (rete, server, router, ...) vengono usati da strumenti di gestione della configurazione per costruire la piattaforma di esecuzione. Il modello include anche il software da installare (compilatori, librerie, DBMS, ...).

10.2.1 Continuous integration

Serve la continuous integration perché, se il sistema si integra raramente, i problemi sono **difficili da isolare** e correggerli rallenta lo sviluppo. Integrare il sistema (*system building*) non vuol dire solo compilare, ma anche:

1. installare il software del database e creare il database con lo schema giusto;
2. caricare nel database i dati di test;
3. collegare il codice compilato con le librerie e gli altri componenti usati;
4. controllare che i servizi esterni usati siano attivi;
5. spostare i file di configurazione nelle posizioni giuste (e cancellare quelli vecchi);
6. eseguire i test di sistema per controllare che l'integrazione sia riuscita.

Come mostra la Fig. 10.3, ogni volta che si fa push di una modifica sul repository condiviso si crea e si testa una versione integrata del sistema. Finito il push, il repository manda un messaggio al **server di integrazione**, che costruisce una nuova versione del prodotto.

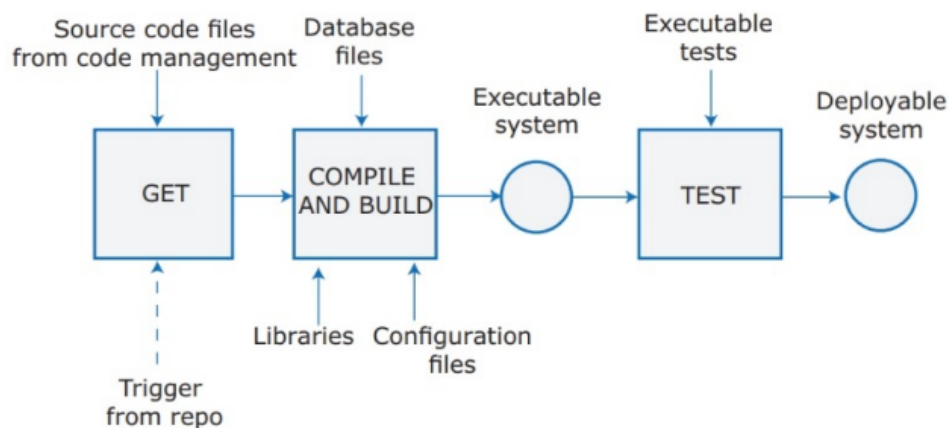


Figura 10.3: La pipeline di continuous integration.

Suggerimento — Integrate twice

È buona pratica **integrare due volte**: prima sulla macchina locale dello sviluppatore, poi facendo push sul repository del progetto, che avvia il server di integrazione (Fig. 10.4). Si fa così perché mettere una **build rotta** nel repository di progetto è una cosa molto grave.

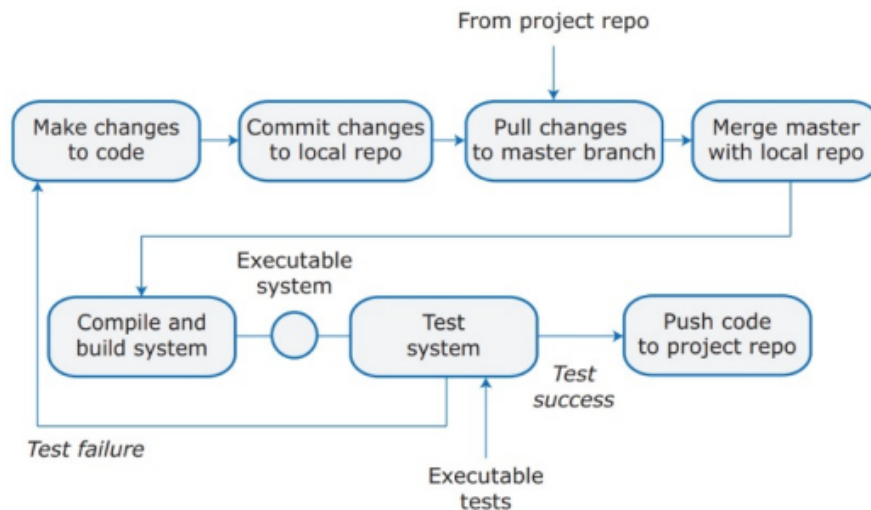


Figura 10.4: La pipeline “integrate twice”.

Vantaggi della continuous integration:

- **Trovare e correggere i bug è più veloce:** se si fa una piccola modifica e un test di sistema fallisce, il problema è quasi certamente nel nuovo codice.
- **C'è sempre un sistema funzionante** a disposizione del team, per provare idee e fare demo al management e ai clienti.
- **Cultura della qualità** nel team: nessuno vuole essere quello che ha “rotto la build”, quindi tutti controllano bene il proprio lavoro prima del push.

Analizzare tutto il codice a ogni commit è molto pesante: per questo gli strumenti di integrazione **ripetono le operazioni solo se i file da cui dipendono sono cambiati** (es. ricompilano solo i file sorgente modificati). Per capire se una dipendenza è cambiata si possono usare le **date di modifica** dei file.

10.2.2 Continuous delivery e deployment

Con la continuous integration e la gestione del codice si può creare una versione eseguibile del sistema, costruendolo e testandolo prima sul computer dello sviluppatore e poi sul server di integrazione.

Quando però lo si installa nell'**ambiente di produzione reale**, qualcosa può andare storto, perché la produzione è **diversa dall'ambiente di sviluppo** (il server può avere un'organizzazione del file system diversa, permessi diversi, applicazioni installate diverse, ...).

La **continuous delivery** garantisce che il sistema modificato sia pronto per i clienti eseguendo test delle feature nell'ambiente di produzione (per controllare che l'ambiente non causi guasti), test di sistema e **test di carico** (per vedere come si comporta il software quando aumentano gli utenti). Il modo più semplice per creare una replica dell'ambiente di produzione sono i **container**.

Come mostra la Fig. 10.5:

- per la **delivery**, dopo i primi test di integrazione si configura un ambiente di test “a fasi” (*staged*) e si lanciano i test di accettazione del sistema (funzionalità, carico, prestazioni);
- per il **deployment**, software e dati vengono trasferiti sui server di produzione, si passa alla nuova versione del sistema e si riavvia il processo. È fondamentale gestire anche i client ancora collegati alla vecchia versione.

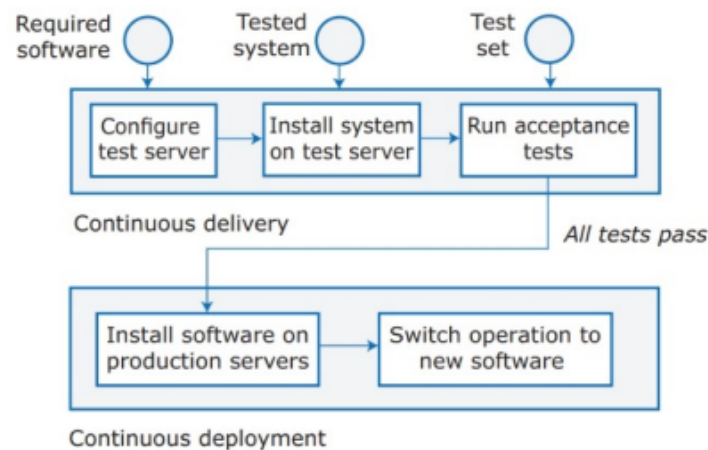


Figura 10.5: Schema di continuous delivery e continuous deployment.

Suggerimento — Delivery vs deployment

Con la **continuous delivery** il software è sempre **pronto** per essere rilasciato (ma la decisione di rilasciarlo può restare manuale). Con il **continuous deployment** ogni modifica che passa i test viene **rilasciata automaticamente** agli utenti.

Vantaggi del continuous deployment:

- **Costi ridotti:** la pipeline di deploy è completamente automatica.
- **Problemi risolti più in fretta:** un problema probabilmente riguarda una piccola parte del sistema, e la sua causa è evidente.
- **Feedback dei clienti più rapido:** si rilasciano le nuove feature appena sono pronte e si usa il feedback degli utenti per migliorarle.
- **A/B testing:** con molti clienti e più server, si può installare la nuova versione solo su alcuni server e usare un load balancer per mandarci una parte dei clienti, così si misura come vengono usate le nuove feature.

Attenzione — Non rilasciare proprio tutto

Probabilmente conviene non rilasciare ogni singola modifica:

- piccole modifiche possono contenere feature incomplete, che è meglio non mostrare ai concorrenti prima di averle finite;
- i clienti possono irritarsi se il software cambia di continuo, soprattutto l'interfaccia;
- si possono voler sincronizzare i rilasci con i cicli di business noti (es. l'inizio dell'anno scolastico per il mercato dell'istruzione).

Molti strumenti di continuous integration (come **Jenkins** e **Travis**) si possono usare anche per la continuous delivery e il deployment. Si possono integrare con strumenti di gestione della configurazione dell'infrastruttura, anche se per il software nel cloud spesso è più semplice usare i **container** insieme agli strumenti di CI.

10.2.3 Infrastructure as code

Gestire a mano un'infrastruttura con decine o centinaia di server è **costoso e soggetto a errori**. I server (fisici o virtuali) hanno configurazioni diverse e software diversi: quando esce una nuova versione di un software, alcuni server vanno aggiornati e altri no, per via di dipendenze

da versioni vecchie. Tenere traccia a mano del software installato su ogni server è difficilissimo e spesso non si fa (es. le modifiche d'emergenza non sempre vengono documentate).

L'idea è automatizzare l'aggiornamento dei server usando un **modello dell'infrastruttura leggibile dalle macchine**. Gli strumenti di **Configuration Management** (CM, come **Puppet**, **Chef** e **Ansible**) installano automaticamente software e servizi sui server secondo la definizione dell'infrastruttura. L'infrastruttura è rappresentata come dati, per questo si parla di **Infrastructure as Code** (IaC, Fig. 10.6). Quando serve un cambiamento si aggiorna il modello, e lo strumento CM applica le modifiche a tutti i server.

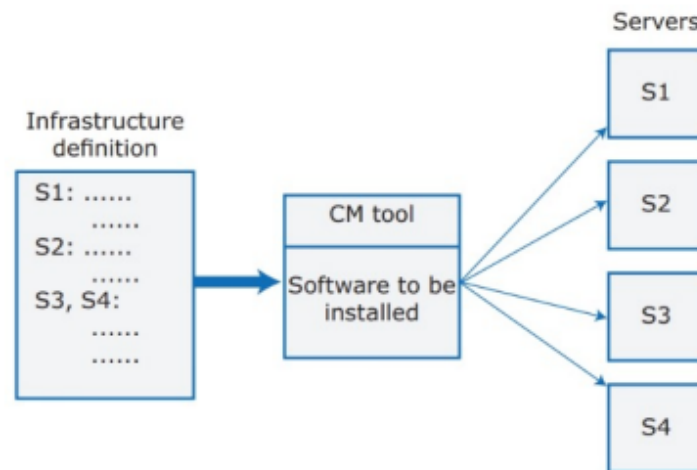


Figura 10.6: Schema dell'infrastructure as code.

Vantaggi dell'infrastructure as code:

- **Visibilità:** l'infrastruttura è definita in un modello a sé che tutto il team DevOps può leggere, capire e revisionare.
- **Riproducibilità:** le installazioni avvengono sempre nella stessa sequenza e producono sempre lo stesso ambiente. Non serve che qualcuno si ricordi l'ordine delle operazioni (i computer sono più affidabili delle persone in questo).
- **Affidabilità:** l'automazione evita i piccoli errori che gli amministratori fanno quando ripetono le stesse modifiche su più server.
- **Ripristino:** il modello dell'infrastruttura si può versionare e salvare nel sistema di gestione del codice. Se una modifica crea problemi, si torna facilmente a una versione precedente e si reinstalla l'ambiente che si sa funzionare.

I **container** sono un modo molto efficace per rilasciare prodotti nel cloud, perché rendono semplice avere **ambienti di esecuzione identici**: per ogni tipo di server si definisce l'ambiente necessario e si costruisce un'immagine; quando si aggiorna il software si crea una nuova immagine. Per scalabilità, resilienza e orchestrazione si può usare un sistema di gestione dei container come **Kubernetes** (vedi [05 - Software cloud-based](#)).

10.3 Misurazioni DevOps

Per migliorare continuamente il processo DevOps, cioè rilasciare più in fretta software di qualità migliore, bisogna **misurare e analizzare** dati sul prodotto e sul processo. Ci sono vari tipi di misurazioni:

- **Misure di processo:** dati sui processi di sviluppo, test e deploy.

- **Misure di servizio:** dati su prestazioni, affidabilità e gradimento del software da parte dei clienti.
- **Misure d'uso:** dati su **come i clienti usano** il prodotto (invece di mostrare pop-up che chiedono se il prodotto è piaciuto); aiutano a trovare problemi nel software stesso.
- **Misure di successo del business:** dati su quanto il prodotto contribuisce al successo complessivo dell'azienda. È difficile isolare il contributo del DevOps, perché il successo potrebbe dipendere dal DevOps oppure da una gestione migliore.

Esempio — Metriche tipiche

- **Processo:** frequenza dei deploy, percentuale di deploy falliti, tempo medio di ripristino dopo un guasto.
- **Servizio:** disponibilità, tempi di risposta, numero di segnalazioni dei clienti.
- **Uso:** numero di utenti attivi, feature più e meno usate, durata delle sessioni.

Misurare il software e il suo sviluppo è complesso: bisogna scegliere metriche che diano informazioni utili e trovare modi affidabili per raccoglierle e analizzarle. Alcune misure (es. la **soddisfazione dei clienti**) vanno ricavate da altre metriche (es. il **numero di clienti che tornano**).

La misurazione va automatizzata il più possibile: gli sviluppatori devono **strumentare** il software perché raccolga dati su se stesso e usare un sistema di monitoraggio per le prestazioni e la disponibilità. Per raccogliere dati in automatico si può:

- usare strumenti di continuous integration come **Jenkins** per i dati su deploy, test superati, ecc.;
- usare i servizi di monitoraggio dei provider cloud, come **Amazon CloudWatch**, per disponibilità e prestazioni;
- raccogliere i dati forniti dai clienti tramite i sistemi di gestione delle segnalazioni (*issue management*);
- aggiungere al prodotto della **strumentazione** per raccogliere dati sulle prestazioni e su come lo usano i clienti. Di solito i **file di log** sono la scelta migliore: si registrano più eventi possibile e si usano strumenti di analisi dei log per capire come viene usato il software.

Capitolo 11

Paradigmi emergenti

Capitolo originale: 11 — Emerging Paradigms in Computing • Antonio Brogi
Fonte: Appunti di Emiliano Sescu (github.com/Faxatos), cap. 11

11.1 Cloud-Edge Continuum

Definizione — Cloud-Edge Continuum

Il **Cloud-Edge Continuum** unisce i punti di forza dell'**Edge Computing** (elaborare i dati vicino a dove vengono prodotti) e del **Cloud Computing**, estendendo i servizi cloud fino all'**Internet of Things (IoT)**. Il risultato è un'infrastruttura **distribuita ed eterogenea**.

Vantaggi principali:

- **Potenza di calcolo:** si possono usare sia le risorse del cloud sia i dispositivi edge.
- **Connettività:** comunicazione migliore tra dispositivi, con trasferimento di dati più fluido.
- **Bassa latenza:** elaborare i dati vicino alla loro fonte riduce i ritardi e rende il sistema più reattivo.

Le applicazioni del futuro saranno soprattutto **a microservizi in container**, eseguite su un'infrastruttura continua Cloud-Edge. Gestirle però è difficile, perché sia l'infrastruttura sia le applicazioni cambiano di continuo:

- **Cambiamenti dell'infrastruttura:** il carico dei nodi può spostarsi, latenza e banda possono variare, i nodi possono entrare o uscire dalla rete all'improvviso, e possono esserci interruzioni temporanee di connessione.
- **Cambiamenti delle applicazioni:** il codice e i requisiti cambiano e richiedono adattamenti rapidi.

Per questo il deploy delle applicazioni va **gestito di continuo**, anche dopo il primo avvio.

11.1.1 Monitoraggio

Serve un monitoraggio efficace sia delle applicazioni sia dell'infrastruttura: un sistema **leggero, tollerante ai guasti** e capace di adattarsi ai cambiamenti.

Un metodo efficace è il **continuous reasoning**: si analizzano sistemi grandi concentrandosi sui **cambiamenti recenti** e **riusando i risultati precedenti** dove possibile. Nel continuum bisogna considerare insieme i cambiamenti dell'applicazione e dell'infrastruttura per decidere se sostituire, spostare, riavviare o scalare i servizi.

Un esempio recente è **FogBrainX** ([GitHub](https://github.com)), che confronta le differenze nelle specifiche dell'applicazione e nei dati monitorati dell'infrastruttura. Aiuta a decidere dove collocare i servizi (*placement*):

- adattandosi ai cambiamenti dell'infrastruttura (es. risorse dei nodi o qualità della rete), che possono richiedere di spostare dei servizi;
- adattandosi ai cambiamenti nei requisiti dei servizi (software, hardware, IoT) o nelle esigenze di comunicazione, che possono richiedere aggiornamenti;
- gestendo l'aggiunta o la rimozione di servizi o di requisiti di comunicazione nelle specifiche dell'applicazione.

Prese le decisioni, FogBrainX le passa a **FogArm**, che esegue i comandi di gestione sull'infrastruttura Cloud-IoT.

11.1.2 Gestione decentralizzata

Per la gestione decentralizzata ci sono due approcci interessanti:

- **Gestione osmotica** (*osmotic management*): i servizi dell'applicazione si adattano in base alle risorse disponibili e alle esigenze dell'applicazione. Le politiche di gestione possono disinstallare, spostare o scalare le applicazioni in tempo reale.
- **Gestione decentralizzata** ispirata al comportamento dei **batteri**:
 - ogni istanza dell'applicazione ha un proprio **agente di gestione** (una specie di mini-unità di gestione);
 - **regole semplici** fanno scattare azioni (es. disinstallare o replicare) in base ai dati monitorati;
 - dai comportamenti locali **emerge** un comportamento globale flessibile, utile per infrastrutture grandi, anche se più complesso da gestire.

11.2 Quantum Software Engineering

Definizione — Quantum Software Engineering (QSE)

La **Quantum Software Engineering** applica solidi principi di ingegneria allo sviluppo, all'esercizio e alla manutenzione del **software quantistico** e della sua documentazione. L'obiettivo è creare software quantistico affidabile, efficiente sui computer quantistici e con costi contenuti.

Il **Manifesto di Talavera** per la QSE indica alcuni principi importanti, tra cui:

- la QSE deve essere compatibile con diversi linguaggi e tecnologie di programmazione quantistica;
- bisogna integrare calcolo **classico e quantistico**, usando i computer quantistici soprattutto per i compiti specifici in cui sono migliori (es. i problemi di fattorizzazione).

11.2.1 Quantum Broker

Un esempio pratico di QSE è il **Quantum Broker**, che risponde alla domanda: “**Quale computer quantistico devo usare per eseguire il mio algoritmo?**”. È utile soprattutto per chi non conosce bene i vari provider quantistici.

Anche dopo aver scelto un provider e avviato gli algoritmi, possono esserci problemi:

- **Disponibilità**: cosa succede se il computer quantistico diventa non disponibile durante l'esecuzione?

- **Requisiti:** come bilanciare costo, tempo e accuratezza?
- **Personalizzazione:** come adattare il modo in cui si sceglie il computer quantistico alle proprie esigenze?

Definizione — Shot e distribuzione degli shot

Un circuito quantistico ha un risultato **probabilistico**, quindi lo si esegue molte volte: ogni esecuzione è uno **shot**. Mettendo insieme i risultati di tutti gli shot si ottiene la distribuzione statistica dei risultati. La **distribuzione degli shot** consiste nel dividere gli shot di un circuito tra **più computer quantistici** e poi combinare i dati per avere il quadro completo.

Dato un circuito quantistico e dei requisiti specifici, il quantum broker sceglie il miglior insieme di computer quantistici su cui distribuire gli shot. Per ogni computer scelto individua i **compilatori** più adatti e il **numero di shot** da eseguire. I vantaggi sono:

- **Resilienza migliore:** dividere il lavoro tra sistemi quantistici diversi aumenta la tolleranza ai guasti.
- **Alta personalizzazione:** si può controllare molto nel dettaglio come vengono gestiti i calcoli.
- **Distribuzioni parziali:** ogni computer produce una distribuzione parziale, e le parziali si combinano per ottenere una visione completa dell'esecuzione del circuito, con la possibilità di metodi di unione più sofisticati.